Practicumhandleiding Technische Natuurkunde

Leerjaar 2

Hogeschool van Amsterdam

Opleiding Technische Natuurkunde

25 oktober 2026

Inhoudsopgave

Practica periode 1 . . . 3
Opzetten Python voor practicum . . . 13
Practicum: Bepaling van de brandpuntsafstand van een onbekende lens . . . 22
Practicum: Spectrometer bouwen . . . 30
Practica periode 2 . . . 40
Practicum: Open-loop regeling van een aeropendulum . . . 41
Practicum: PID-regeling van een aeropendulum . . . 51
Practicum: efficiëntie en vermogen van zonnecellen . . . 61
Practica periode 3 . . . 66
Practicum: SEM-EDX . . . 67
Practicum: bepaling van de verhouding e/m_e met een elektronenbundel . . . 74
Practicum: Electronenbundel – Elektronendiffractie op grafiet . . . 80
Practicum: apparaat communicatie met Python . . . 85
Practica periode 4 . . . 101
Practicum: Trekbankproef I . . . 102
Practicum: Trekbankproef II . . . 106
Practicum: Hardheid en Kerfslag . . . 111
Practicum: Viscositeit met de Ubbelohde viscometer . . . 118
Practicum: Digitale viscosimeter en niet-Newtoniaanse vloeistoffen . . . 122
Python voor het practicum . . . 127
Python-tips . . . 128
Code snippets . . . 153

Practica periode 1

Optica

Introductie

Algemeen & leerdoelen practicum

In het eerste jaar van de opleiding Technische Natuurkunde waren practica voornamelijk ondersteunend aan de natuurkundetheorie. De experimenten waren relatief kort en gericht op het zichtbaar en meetbaar maken van concepten uit de theorielessen.
In het tweede jaar verschuift deze focus. De practica worden georganiseerd rond bredere natuurkundige thema's, zoals *moderne optica*, en richten zich sterker op het ontwikkelen van experimentele vaardigheden. Het doel is om steeds zelfstandiger te leren werken als technisch natuurkundige.
De belangrijkste leerdoelen zijn:

- Professioneel, zorgvuldig en veilig leren werken in een laboratoriumomgeving;
- Vaardigheid en routine ontwikkelen in experimentele handelingen en het gebruik van gangbare meetapparatuur;
- Zelfstandig experimentele problemen herkennen, analyseren en oplossen;
- Meetgegevens verantwoord analyseren, onzekerheden beoordelen en resultaten fysisch interpreteren;
- Effectief samenwerken bij het uitvoeren, analyseren en documenteren van experimenten.

Naast deze overkoepelende leerdoelen, zijn er ook per practicum specifieke leerdoelen. Je rond het practicum af als je voldoet aan deze specifieke leerdoelen.

Begeleiding

Op het practicum leer je experimenteren. Dat gaat niet vanzelf. Naarmate je verder in het tweede jaar komt, krijg je een grotere vrijheid om naar eigen inzicht en keuze te experimenteren. Je wordt begeleid door praktijkinstructeurs, de practicumdocent en (soms) externen. Zit je met een probleem, kom je ergens niet uit, vraag dan vooral hulp. Bedenk dat een vraag geen teken van onvermogen is, maar een kenmerk van het nemen van initiatief, een belangrijke vaardigheid.



Praktijkinstructeur Bert Buisma.



Practicumdocent Joris Busink.

- Praktijkinstructeur (Bert Buisman): Bert zal ieder practicum starten met een veiligheidsinstructie. Hij is het eerste aanspreekpunt voor veilig én zorgvuldig gebruik van (practicum) apparatuur. Ben je te laat of afwezig? Meldt dit dan bij Bert.
- Practicum docent (Joris Busink): Joris zal ieder practicum ook inleiden en je meenemen in de achtergrond, theorie en toepassingen van het practicum. Hij is op de practicumzaal het éérste aanspreekpunt bij inhoudelijke vragen over het practicum, voorbereiding én de leerstof. Hij controleert en beoordeeld de voorbereiding en de verrichtte werkzaamheden tijdens het practicum.
- Externe docenten: externe docenten begeleiden vaak zéér specifieke practica (bijvoorbeeld een proef over de trekbank). Zij zullen dan ook het eerste aanspreekpunt zijn voor inhoudelijke vragen én/of veilig werken.

Huisregels practicum

Op het practicum zijn een aantal basis regels noodzakelijk. De bedoeling van deze regels is dat er op een soepele en prettige manier op het practicum gewerkt kan worden, zonder dat er chaos ontstaat, materialen zoekraken of apparatuur onnodig snel defect raakt. De practicumstaf draagt zorg voor een goed verloop van het practicum en zal je, indien nodig, aanspreken op onderstaande regels.

Aanwezigheid

- Het practicum heeft een verplichte aanwezigheid voor álle dagdelen. Te laat komen of een sessie missen is niet toegestaan.
- Het practicumrooster verloopt via inschrijving. Via Brightspace zul je je moeten aanmelden voor een practicum. Soms zijn deze practica één sessie (bijvoorbeeld op maandag van 12:00-14:30), maar af en toe zullen we ook langere sessies houden (bijvoorbeeld **alle** practica sessies van week 1 & 2). Mocht het zo zijn dat op het moment van inschrijven er géén sessies meer zijn waarbij jij aanwezig kan zijn, neem dat contact op met de practica staffleden. Mogelijk kan er dan nog iets geregeld worden, échter, het blijft jouw eigen verantwoordelijkheid om aanwezig te zijn.

Te-laat-regeling

We begrijpen dat het kan voorkomen dat je trein vertraging heeft, je band lek is of zich een andere onverwachte situatie voordoet. Échter, we hebben beperkt tijd en we raden je aan om bij een moeilijk OV-traject b.v. een trein eerder te pakken.

- Ben je minder dan 15 minuten te laat, laat dit dan weten aan de practica stafleden via Teams/ Mail. Bij herhaaldelijk <15 minuten te laat zal een gesprek volgen.
- Ben je meer dan 15 minuten te laat, maar minder dan 30 minuten, dan zal de aanwezig practicum staflid hierover een besluit nemen. Afhankelijk van het type practicum mag je dan wel/niet meedoen.
- Ben je meer dan 30 minuten te laat? Dan geldt dat je het practicum niet kan afronden en zal je deze sessie moeten herkansen.

Afwezigheid & inhalen

- Afwezigheid in verband met b.v. ziekte, géén openbaar vervoer óf persoonlijke omstandigheden kunnen natuurlijk voorkomen. Bij afwezigheid van één sessie zul je deze practicumsessie kunnen inhalen aan het eind van de periode (week 9). Meld je afwezigheid bij de practica stafleden.
- Mis je meer dan één practica-sessie dan kun je het practicum niet afronden.
- In uitzonderlijke gevallen kan er een uitzondering gemaakt worden op de herkansingsregeling.

Opruimen en werkplek

Een overzichtelijke werkplek is belangrijk voor zowel de veiligheid als de betrouwbaarheid van experimenteel werk.

- Eten en drinken is niet toegestaan in de practicumzaal en in ruimtes met optische of andere gevoelige experimentele opstellingen.
- Jassen, tassen en andere losse voorwerpen worden buiten de directe experimentele werkruimte opgeborgen.
- Houd de werkplek tijdens het experiment overzichtelijk en zorg dat kabels, slangen en apparatuur geen onnodig struikel- of beschadigingsgevaar opleveren.
- Breng de opstelling na afloop van het experiment terug in de afgesproken toestand.
- Zet apparatuur uit wanneer deze niet meer gebruikt wordt.
- Wanneer de opstelling tijdens een volgende practicumdag verder wordt gebruikt, overleg je met de practicumbegeleider welke onderdelen opgebouwd mogen blijven staan.

Voorbereiding, digitaal labjournaal & afronding practicum

Voorbereiding

Veel van de practica verwachten voorbereidende werkzaamheden. Deze voorbereiding zal in de meeste gevallen minder dan één uur moeten kosten. Het maken van de voorbereiding is essentieel om ervoor te zorgen dat het practica op tijd afkomt. We hebben per practicumsessie maar 2.5 uur, we hebben deze tijd écht nodig!

- Géén voorbereiding = géén toegang!
- De voorbereiding mag alleen óf in duo's gemaakt worden. De voorbereiding moet in **allebei** de digitale labjournaals staan!
- Iedere student moet de voorbereiding hebben gedaan. Jouw practicumpartner mag dus niet 'voor jullie samen' de voorbereiding hebben gedaan.
- De deadline voor de voorbereiding is **2 uur** voor het betreffende practicum. Mocht je het later in je labjournaal zetten, dan loop je het risico dat je niet wordt toegelaten bij het practicum.
- Bij het maken van de voorbereiding mag géén gebruik gemaakt worden van AI. Mocht je vastlopen bij sommige opdrachten, informeer dan de practicum docent óf geef het aan in je digitale labjournaal. Het motto is: Het is niet erg om vast te lopen, het is wel erg om een AI-antwoord op te schrijven en dit zelf niet te kunnen reproduceren!

Digitale labjournaal

Tijdens het practica maken wij gebruik van het digitale labjournaal. Alhoewel de praktijk reeds vaak nog met pen en papier werkt, zullen wij dit toch digitaal doen. In het labjournaal zet je onder andere in:

- De voorbereidende werkzaamheden voor ieder practicum;
- De aantekeningen die je tijdens de les maakt (uitleg, theorie, bronnen, e.d.);
- De gemaakte figuren, dataverwerkingsstappen e.d.;
- De eindconclusie en discussie;
- Alle overige details die relevant zijn tijdens het practicum.

In de praktijk zal je moeten oefenen met het bijhouden van een labjournaal, je wordt dan ook per periode gecontroleerd of jouw labjournaal voldoet aan de door ons gestelde eisen. Voor ons is het belangrijkst dat:

- Het labjournaal is overzichtelijk, de experimentele handelingen zijn per practica goed te volgen;
- De voorbereiding, belangrijkste figuren en einddiscussie & conclusie zijn aanwezig.
- Het is bijgehouden *tijdens* het practicum.

Afronding

Je rond het practicum af als geldt dat:

- Je bent alle practica lessen aanwezig geweest **én**
- Je hebt voldaan aan alle leerdoelen van het practicum **én**
- Je hebt jouw opgeleverde werk laten zien aan de practicum docent **én**
- Je hebt jouw werkplek opgeruimd.

Fraude, plagiaat en gebruik van AI

Werkplannen, labjournaals, codes en andere individuele producten moeten het eigen werk van de student weergeven. Het overnemen van tekst, berekeningen, afbeeldingen, meetresultaten of andere inhoud van een medestudent zonder correcte verantwoording is niet toegestaan.

- Individuele producten moeten zelfstandig worden geschreven en uitgewerkt.
- Samenwerken tijdens het uitvoeren van een experiment is toegestaan en vaak noodzakelijk, maar de individuele verwerking en verslaglegging moet voldoen aan de eisen die bij het betreffende practicumonderdeel worden gesteld.
- Bronnen moeten zorgvuldig en volgens de voorgeschreven wijze worden vermeld. Het zonder bronvermelding overnemen van tekst, afbeeldingen, code, berekeningen of andere inhoud geldt als plagiaat.
- **Het is niet toegestaan om generatieve AI te gebruiken tijdens het practicum!** Het overtreden van deze regel kan in het uiterste geval leiden tot verwijdering uit de practicumzaal.
- Wanneer twijfel bestaat of een ingeleverd product daadwerkelijk het eigen werk van de student is, kan de student worden gevraagd het werk mondeling toe te lichten.
- Bij een vermoeden van fraude of plagiaat wordt gehandeld volgens de geldende onderwijs- en examenregeling en de fraude- en plagiaatregeling van de Technische Natuurkunde opleiding aan de Hogeschool van Amsterdam.

Veiligheid op het practicum

Tijdens de practica van Technische Natuurkunde werk je met uiteenlopende apparatuur en fysische verschijnselen. Hierbij kun je onder andere te maken krijgen met elektrische spanningen en stromen, lasers, chemicaliën, hoge en lage temperaturen, bewegende onderdelen, druksystemen en ioniserende straling.

Veilig werken is onderdeel van professioneel handelen als technisch natuurkundige. Van iedere student wordt daarom verwacht dat deze de veiligheidsinstructies kent en opvolgt en bij twijfel eerst overlegt met de practicumbegeleider.

Algemene veiligheidsregels

Tijdens het practicum gelden de volgende algemene regels:

- Volg altijd de instructies van de practicumbegeleider.
- Lees vooraf de veiligheidsinstructies van het betreffende practicum.
- Gebruik apparatuur alleen voor het doel waarvoor deze bedoeld is.
- Controleer een opstelling voordat deze wordt ingeschakeld.
- Breng geen wijzigingen aan in een ingeschakelde elektrische, mechanische of optische opstelling als dit een risico kan opleveren.
- Houd de werkplek overzichtelijk en zorg dat looppaden vrij blijven.
- Meld defecte apparatuur, beschadigde kabels en andere onveilige situaties direct bij de practicumbegeleider.
- Weet waar de nooduitgangen, brandmelders, blusmiddelen en andere veiligheidsvoorzieningen zich bevinden.
- Meld ieder ongeval of incident bij de practicumbegeleider, ook wanneer er geen direct letsel lijkt te zijn ontstaan.

Elektriciteit

Elektrische stroom door het lichaam kan leiden tot spierverkramping, brandwonden, ademhalingsproblemen en hartritmestoornissen. Daarnaast kan kortsluiting leiden tot zeer hoge stromen, warmteontwikkeling, vonkvorming of een vlamboog. Een elektrische schok kan bovendien een secundair ongeval veroorzaken, bijvoorbeeld doordat iemand valt.

Houd je daarom aan de volgende regels:

- Schakel de voeding uit voordat je een elektrische schakeling opbouwt of wijzigt.
- Controleer de schakeling voordat de spanning wordt ingeschakeld.
- Gebruik geschikte meetsnoeren en connectoren en controleer deze op beschadigingen.
- Voorkom kortsluiting en controleer vooraf de ingestelde stroombegrenzing van een voeding.
- Raak geen ongeïsoleerde geleiders aan wanneer niet zeker is dat deze spanningsloos zijn.
- Raak geen delen aan waarop een wisselspanning hoger dan 50 V of een gelijkspanning hoger dan 120 V kan staan.
- Werk bij hogere spanningen uitsluitend volgens de instructies van de practicumbegeleider en met de daarvoor bestemde apparatuur en beveiligingen.
- Verwijder of onderbreek nooit de beschermingsaarde van apparatuur.

Let erop dat ook relatief lage spanningen gevaar kunnen opleveren wanneer grote stromen beschikbaar zijn. Hoge stromen kunnen bijvoorbeeld kabels, componenten en accu's sterk verhitten en brand veroorzaken.

Lasers

Een laserbundel kan een hoge vermogensdichtheid hebben. Zelfs lasers met een relatief laag vermogen kunnen oogletsel veroorzaken doordat het oog de bundel op het netvlies focust.

Bij het werken met lasers gelden daarom de volgende regels:

- Kijk nooit rechtstreeks in een laserbundel.

- Kijk ook niet in een laserbundel met optische hulpmiddelen zoals een lens, microscoop, telescoop of verrekijker.
- Houd de laserbundel waar mogelijk laag en parallel aan het tafelblad en voorkom dat deze op ooghoogte loopt.
- Voorkom dat een laserbundel de werktafel kan verlaten.
- Gebruik geschikte bundelstoppers en, waar nodig, afscherming aan de randen van de werktafel.
- Houd rekening met reflecties aan spiegels, glas, metalen onderdelen en andere glanzende oppervlakken.
- Doe horloges en andere sterk reflecterende sieraden af wanneer deze onbedoelde reflecties kunnen veroorzaken.
- Laat geen losse opticacomponenten of gereedschappen in of nabij het pad van de laserbundel liggen.
- Schakel de laser uit of scherm de bundel af voordat grote wijzigingen aan de optische opstelling worden uitgevoerd, wanneer je de werkplek verlaat of wanneer je iets van de grond moet oprapen.
- Houd rekening met andere aanwezigen in de practicumruimte en waarschuw hen wanneer met een laser wordt gewerkt.

Voor practica waarbij aanvullende laserveiligheidsmaatregelen nodig zijn, worden deze vooraf door de practicumbegeleider toegelicht.

Mechanische opstellingen

Tijdens het practicum werken we soms met krachtige mechanische opstellingen, zoals motoren en trekbanken. Bewegende onderdelen en onderdelen waarop grote krachten worden uitgeoefend kunnen beknellings- of verwondingsgevaar opleveren.

- Houd handen, kleding, haar en losse voorwerpen uit de buurt van bewegende onderdelen.
- Zet een aandrijving uit voordat je de mechanische opstelling wijzigt.
- Controleer of draaiende of bewegende onderdelen goed bevestigd zijn voordat een motor of andere aandrijving wordt ingeschakeld.
- Gebruik aanwezige beschermkappen en verwijder deze niet tijdens bedrijf.
- Houd voldoende afstand tot onderdelen die onder grote mechanische spanning staan of bij breuk kunnen losschieten.

Chemische en schadelijke stoffen

Tijdens enkele practica wordt gewerkt met chemicaliën of andere stoffen die bij verkeerd gebruik risico's kunnen opleveren. Hoewel op het natuurkundepracticum in principe niet met sterk gevaarlijke chemicaliën wordt gewerkt, moet contact met huid en ogen zoveel mogelijk worden voorkomen.

Bij sommige experimenten kunnen daarnaast materialen worden gebruikt die schadelijke stoffen bevatten, zoals lood. Vermijd direct contact met onbeschermd lood en was na aanraking altijd je handen.

- Draag bij het werken met chemicaliën altijd een labbril.
- Draag handschoenen wanneer dit bij het practicum wordt aangegeven.
- Pipetteer nooit met de mond en proef stoffen nooit.
- Ruik niet rechtstreeks aan onbekende stoffen of chemicaliën.
- Laat verpakkingen en voorraadflessen niet onnodig openstaan en berg deze na gebruik op de daarvoor bestemde plaats op.
- Ruim gemorste stoffen direct en volgens de instructies van de practicumbegeleider op.
- Was na het werken met chemicaliën, lood of andere schadelijke stoffen je handen.

Bij twijfel over een stof of de benodigde beschermingsmiddelen overleg je altijd eerst met de practicumbegeleider.

Cryogene vloeistoffen en gassen

Bij sommige natuurkundige experimenten kunnen sterk gekoelde of vloeibaar gemaakte gassen worden gebruikt, zoals vloeibare stikstof. Deze stoffen hebben een zeer lage temperatuur en kunnen bij contact met de huid ernstig koudeletsel veroorzaken.

Daarnaast verdampen cryogene vloeistoffen snel. Het gevormde gas neemt een veel groter volume in dan de vloeistof. Hierdoor kan in een afgesloten systeem een gevaarlijke overdruk ontstaan. In een onvoldoende geventileerde ruimte kan het verdampende gas bovendien de zuurstofconcentratie verlagen.

- Gebruik cryogene vloeistoffen uitsluitend volgens de instructies van de practicumbegeleider.
- Voorkom direct huidcontact met cryogene vloeistoffen en met sterk afgekoelde, niet-geïsoleerde oppervlakken en leidingen.
- Gebruik de voorgeschreven persoonlijke beschermingsmiddelen bij het overgieten of hanteren van cryogene vloeistoffen.
- Werk met cryogene vloeistoffen uitsluitend in voldoende geventileerde ruimten.
- Sluit een vat met cryogene vloeistof nooit gasdicht af.
- Voorkom dat verontreinigingen of ongewenste stoffen in cryogene vloeistoffen of opslagvaten terechtkomen.

Brand en calamiteiten

Voor het ontstaan van brand zijn een brandbare stof, zuurstof en een ontstekingsbron nodig. Ontstekingsbronnen kunnen bijvoorbeeld open vuur, hete onderdelen, soldeerbouten en elektrische vonken zijn.

Voorkom daarom dat brandbare materialen in contact kunnen komen met hete onderdelen, vonken of andere ontstekingsbronnen. Houd de werkplek vrij van onnodige brandbare materialen en zorg bij het werken met vluchtige of brandbare stoffen voor voldoende ventilatie of afzuiging.

Bij een alarm, brand, letsel of een andere calamiteit:

- stop het experiment indien dit veilig mogelijk is;
- waarschuw direct de practicumbegeleider en aanwezigen in de omgeving;
- activeer bij brand de brandmelder;
- volg de instructies van de practicumbegeleider en BHV;
- bel bij een ernstige calamiteit het interne alarmnummer **020–595 3333**;
- probeer een brand niet zelfstandig te bestrijden wanneer dit risico voor jezelf of anderen oplevert;
- sluit bij een brandend gas, indien dit veilig mogelijk is, de gastoevoer af;
- sluit bij brand ramen en deuren wanneer dit veilig mogelijk is;
- volg bij een ontruiming de aangegeven vluchtroute;
- gebruik geen lift tijdens een ontruiming;
- ga naar de aangegeven verzamelplaats.

Weet voordat je met een experiment begint waar de dichtstbijzijnde brandmelder, nooduitgang en blusmiddelen zich bevinden.

Eerste hulp en ongevallen

Meld ieder ongeval en ieder lichamelijk letsel direct bij de practicumbegeleider.

Bij ernstig lichamelijk letsel moet het interne alarmnummer **020–595 3333** worden gebeld. Geef daarbij in ieder geval de volgende informatie door:

- de locatie van het ongeval;
- de toestand van het slachtoffer;
- je naam;
- of EHBO en/of een ambulance nodig is.

Breng jezelf bij het verlenen van hulp niet in gevaar en volg de aanwijzingen van de practicumbegeleider, BHV of hulpdiensten.

Veiligheid

1 Zowel Bert als Joris zijn in bezit van een BHV-diploma. Zorg ervoor dat bij een calamiteit zij op de hoogte zijn.

Planning Periode 1

Tabel 1: Planning practica blok 1.

kalender week	Periode week	Ma 12:00-14:30	Ma 14:30-17:00	Vrij 15:20-17:50	Activiteit Bijeenkomst 1	Activiteit Bijeenkomst 2	Extra bijeenkomst
Blok 1							
36	1	31-aug	31-aug	4-sep	Gr. 1 Python	Gr. 2 Python	-
37	2	7-sep	7-sep	11-sep	Gr. 1 Optica	Gr. 2 Optica	-
38	3	14-sep	14-sep	18-sep	Groep 1: Bouwen spectrometer[1]		-
39	4	21-sep	21-sep	25-sep	Groep 1: Experimenteren spectrometer[1]		-
40	5	28-sep	28-sep	2-okt	Groep 2: Bouwen spectrometer[1]		-
41	6	5-okt	5-sep	9-okt	Groep 2: Experimenteren spectrometer[1]		-
42	7	**Herfstvakantie**					
43	8	19-okt	19-okt	23-okt	-		
44	9	26-okt	26-okt	30-okt	Herkansing op afspraak		
45	10	**Tentamens**					
Blok 2							

[1] Studenten worden geacht alle bijeenkomsten bij te wonen.

Opzetten Python voor practicum

Leerdoelen

- ☐ Pythonontwikkelomgeving opzetten in VSCode.
- ☐ Leren werken met virtuele omgevingen in Python.
- ☐ Basis van projectstructuur.

Veiligheid

We gaan programmeren, hier zijn geen veiligheidskwesties aan gebonden. Probeer alleen zoveel mogelijk zelfstandig te programmeren. Het is toegestaan om in dit practicum generatieve AI te gebruiken, gebruik het verstandig:

1. Gebruik het als uitlegmachine, niet als codegenerator
2. Gebruik het om eigen code te laten verbeteren, maar niet herschrijven
3. Gebruik het niet voor volledige oplossingen, snap je iets niet? Vraag het de docent.

Introductie

Python is een van de meest gebruikte programmeertalen binnen de natuurkunde en andere technische vakgebieden. Het wordt gebruikt voor het verwerken en visualiseren van meetgegevens, het uitvoeren van numerieke berekeningen en het aansturen van experimenten. Tijdens de opleiding zullen we Python daarom regelmatig gebruiken, bijvoorbeeld om meetdata te analyseren en grafieken te maken. Python zelf is echter alleen de programmeertaal. Voor praktisch gebruik zijn daarnaast verschillende *packages* nodig. Zo gebruiken we `NumPy` voor numerieke berekeningen, `Matplotlib` voor het maken van grafieken en `SciPy` voor wetenschappelijke berekeningen. Naarmate je meer projecten uitvoert, zul je steeds meer van dergelijke packages tegenkomen.

Hierbij ontstaat een praktisch probleem: niet ieder project gebruikt dezelfde packages of dezelfde versies daarvan. Een nieuwere versie van een package kan bijvoorbeeld anders werken dan een oudere versie. Om te voorkomen dat verschillende projecten elkaar in de weg zitten, werken we daarom met *virtuele omgevingen* (*environments*). Een environment kun je zien als een afzonderlijke Python-installatie met zijn eigen verzameling packages. Voor ieder project kan zo een geschikte en reproduceerbare werkomgeving worden gemaakt.

In dit practicum gebruiken we **Miniforge** om Python, environments en packages te beheren. Miniforge is een relatief compacte Python-distributie die gebruikmaakt van het `conda`-systeem. Miniforge gebruikt standaard packages uit het `conda-forge`-ecosysteem. Hierdoor hoeven packages en hun onderlinge afhankelijkheden niet handmatig geïnstalleerd en bijgehouden te worden.

Voor het daadwerkelijk schrijven en uitvoeren van Python-code gebruiken we **Visual Studio Code (VS Code)**. VS Code is een programmeeromgeving waarin je onder andere Python-scripts en Jupyter notebooks kunt schrijven, uitvoeren en beheren. VS Code bevat daarbij niet zelf “de Python”: we koppelen VS Code aan de Python-interpreter uit de environment die we met Miniforge hebben gemaakt.

In dit practicum ga je daarom stap voor stap een complete Python-werkomgeving opzetten. Je installeert Miniforge en VS Code, maakt vanuit de opdrachtprompt een eigen environment aan, installeert hierin enkele veelgebruikte wetenschappelijke packages en koppelt deze environment vervolgens aan VS Code. Tot slot controleer je met een eenvoudig Python-programma of alles correct is geïnstalleerd.

Voorbereidingsopdracht

1. Neem een laptop mee.
2. Zorg voor een stabiele wifi-verbinding met de HvA.

Python installatie

Opdracht 1 — Oude Anaconda-installatie verwijderen

Voordat we een nieuwe Python-omgeving installeren, verwijderen we eerst eventuele oude installaties van Anaconda. Hiermee voorkomen we dat Windows of VS Code later per ongeluk een oude Python-installatie gebruikt.

1. Open in Windows **Instellingen** → **Apps** → **Geïnstalleerde apps**.
2. Zoek naar **Anaconda** of **Anaconda3**. Indien Anaconda geïnstalleerd is, verwijder deze installatie.
3. Open vervolgens de Verkenner en ga naar je gebruikersmap:
   `C:\Users\<gebruikersnaam>\`
4. Controleer of hier nog een map `anaconda3` aanwezig is. Verwijder deze map indien deze na het verwijderen van Anaconda is achtergebleven.
5. Controleer ook of de volgende verborgen map aanwezig is:
   `C:\Users\<jouw_naam>\anaconda3`
   `C:\Users\<jouw_naam>\miniconda3`
   `C:\Users\<jouw_naam>\.conda`
   `C:\Users\<jouw_naam>\.anaconda`
   `C:\Users\<jouw_naam>\.condarc`
   `C:\ProgramData\Anaconda3`
   Verwijder deze mappen indien deze uitsluitend afkomstig zijn van je oude Anaconda-installatie.
6. Start de computer opnieuw op.

Let op! Heb je eerder zelf environments gemaakt met bestanden die je nog nodig hebt? Verwijder deze dan niet voordat je hebt gecontroleerd of je de bestanden hebt opgeslagen.

Opdracht 2 — Controleren of Anaconda verwijderd is

We controleren nu of Windows niet meer naar de oude Anaconda-installatie verwijst.

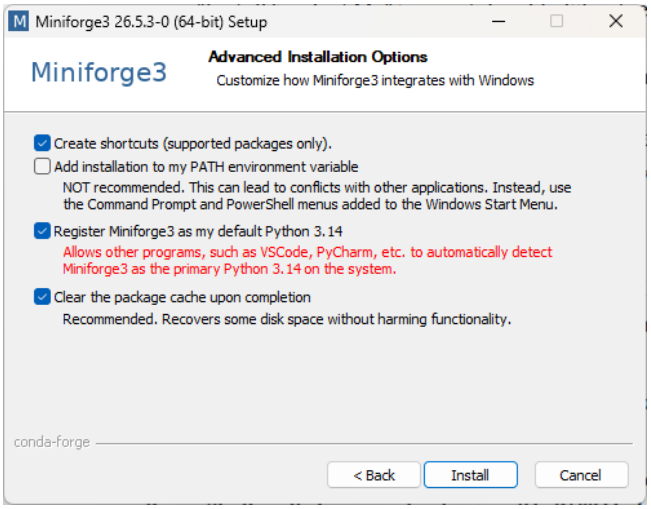
1. Open de Windows **Opdrachtprompt**. Zoek hiervoor in het startmenu naar `cmd` (Command Prompt/Opdrachtenprompt).
2. Voer het volgende commando uit:
   `where conda`
3. Voer vervolgens uit:
   `where python`
4. Controleer de uitvoer. Er mag geen pad meer zichtbaar zijn waarin `Anaconda` of `anaconda3` staat.

Het is geen probleem als Windows meldt dat `conda` niet kan worden gevonden. Op dit moment hebben we immers nog geen nieuwe conda-installatie geïnstalleerd.

Opdracht 3 — Miniforge installeren

We gebruiken **Miniforge** voor het installeren en beheren van Python en de benodigde packages.

1. Ga naar de officiële Miniforge-pagina op GitHub:
   https://conda-forge.org/download/
2. Download de meest recente **Miniforge3**-installer voor jouw besturingssysteem.
3. Kies de installer die past bij je computer. Voor de meeste Windows-computers is dit:
   `Miniforge3-Windows-x86_64.exe`
4. Start het installatiebestand en doorloop de installatie, zie Fig. 2.
5. Gebruik de standaard installatiemap, tenzij je een goede reden hebt om hiervan af te wijken.
6. Rond de installatie af en open daarna vanuit het startmenu de **Miniforge Prompt**.



Figuur 2: Miniforge installatie.

Opdracht 4 — Miniforge controleren

We controleren of Miniforge en het programma `conda` correct zijn geïnstalleerd.

1. Open de **Miniforge Prompt**.
2. Voer het volgende commando uit:
   ```
   conda --version
   ```
3. Als de installatie correct is verlopen, verschijnt een versienummer van conda.
4. Controleer vervolgens welke Python-installatie op dit moment wordt gebruikt:
   ```
   where python
   ```
5. Bekijk het weergegeven pad. Dit pad hoort nu naar je installatie van **Miniforge3** te verwijzen.

Opdracht 5 — Miniforge toevoegen aan PowerShell

Om Conda-commando's rechtstreeks vanuit PowerShell en de terminal van VS Code te kunnen gebruiken, moet Miniforge eenmalig aan PowerShell worden toegevoegd.

1. Open via het Windows-startmenu de **Miniforge Prompt**.
2. Voer het volgende commando uit:
   ```
   conda init powershell
   ```

   Hiermee wordt PowerShell zo ingesteld dat het commando `conda` beschikbaar is.
3. Sluit de Miniforge Prompt en open vervolgens **PowerShell**.
4. PowerShell kan het uitvoeren van het Conda-script blokkeren vanwege de ingestelde beveiliging. Krijg je een melding dat het uitvoeren van scripts is uitgeschakeld (*running scripts is disabled on this system*)? Voer dan in PowerShell het volgende commando uit:
   ```
   Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
   ```

   Bevestig de wijziging wanneer daarom wordt gevraagd door `Y` te typen en op *Enter* te drukken. Door `CurrentUser` te gebruiken is hiervoor geen administratoraccount nodig.
5. Sluit vervolgens alle geopende PowerShell-vensters en **VS Code volledig af**. Start daarna PowerShell opnieuw.
6. Controleer of Conda nu beschikbaar is:
   ```
   conda --version
   ```

Wanneer een versienummer wordt weergegeven, bijvoorbeeld `conda 25.x.x`, is Miniforge correct aan PowerShell toegevoegd.

Opdracht 6 — Een Python-environment maken

We installeren Python niet rechtstreeks als één algemene installatie voor alle projecten. In plaats daarvan maken we binnen Miniforge een afzonderlijke *environment*. Hierdoor kunnen verschillende projecten ieder hun eigen Python-versie en packages gebruiken.
In de vorige opdracht hebben we Conda beschikbaar gemaakt in PowerShell. We gebruiken daarom vanaf nu **PowerShell** voor het uitvoeren van Conda-commando's. De **Miniforge Prompt** kan hiervoor eventueel ook worden gebruikt.

1. Open **PowerShell**.
2. Maak een nieuwe environment met de naam `practicum`:
   ```
   conda create -n practicum python
   ```
3. Bevestig de installatie wanneer daarom wordt gevraagd. Dit doe je door `y` te typen en vervolgens op *Enter* te drukken.
4. Activeer vervolgens de nieuwe environment:
   ```
   conda activate practicum
   ```
5. Controleer de geïnstalleerde Python-versie:
   ```
   python --version
   ```
   Je ziet nu aan het begin van de opdrachtregel `(practicum)` staan. Dit betekent dat de environment `practicum` actief is.
6. Bekijk vervolgens welke environments op je computer aanwezig zijn:
   ```
   conda env list
   ```
   Naast `practicum` zou op dit moment in ieder geval de `base`-environment aanwezig moeten zijn. De actieve environment wordt aangegeven met een sterretje (`*`). De uitvoer ziet er bijvoorbeeld als volgt uit:
   ```
   # conda environments:
   #
   # * -> active
   # + -> frozen
   base                  C:\Users\<jouw_naam>\AppData\Local\miniforge3
   practicum          *   C:
   \Users\<jouw_naam>\AppData\Local\miniforge3\envs\practicum
   ```

Opdracht 7 — Wetenschappelijke packages installeren

Een nieuwe Python-environment bevat nog niet automatisch alle packages die we tijdens de practica nodig hebben. We installeren daarom een aantal veelgebruikte wetenschappelijke packages.
Ga verder in **PowerShell** vanuit de vorige opdracht. De **Miniforge Prompt** kan eventueel ook worden gebruikt.

1. Controleer eerst of de environment `practicum` actief is. Aan het begin van de opdrachtregel moet `(practicum)` staan.
   Is dit niet het geval, activeer de environment dan met:
   ```
   conda activate practicum
   ```

2 Installeer vervolgens NumPy, SciPy, Matplotlib, Pandas en Jupyter (ipykernel):
`conda install numpy scipy matplotlib pandas ipykernel`

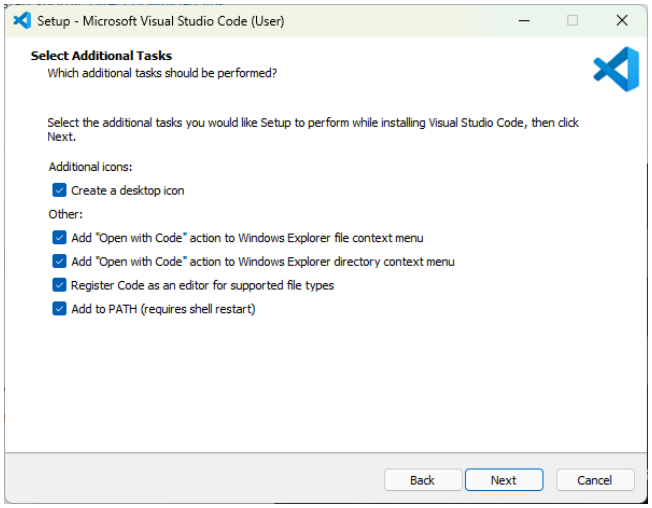
3 Bevestig de installatie wanneer daarom wordt gevraagd door `y` te typen en op *Enter* te drukken.
4 Bekijk na de installatie welke packages in de actieve environment aanwezig zijn:
`conda list`

Vraag: Zoek in de lijst de packages `numpy`, `scipy` en `matplotlib`. Welke versies zijn geïnstalleerd?

Opdracht 8 — Visual Studio Code installeren

Tot nu toe hebben we Python en de benodigde packages geïnstalleerd. We hebben echter nog geen programma waarin we gemakkelijk Python-code kunnen schrijven. Hiervoor gebruiken we **Visual Studio Code (VS Code)**.

1 Ga naar de officiële website van Visual Studio Code: https://code.visualstudio.com/
2 Download en installeer de Windows-versie van VS Code, zie Fig. 3.
3 Start VS Code na de installatie.
4 Open links in VS Code het onderdeel **Extensions**.
5 Zoek naar de extensies **Python, Jupyter, Prettier, Rainbow CSV, Spreadsheet Viewer** en installeer deze.



Figuur 3: VScode installatie.

Mappenstructuur

Mappenstructuur

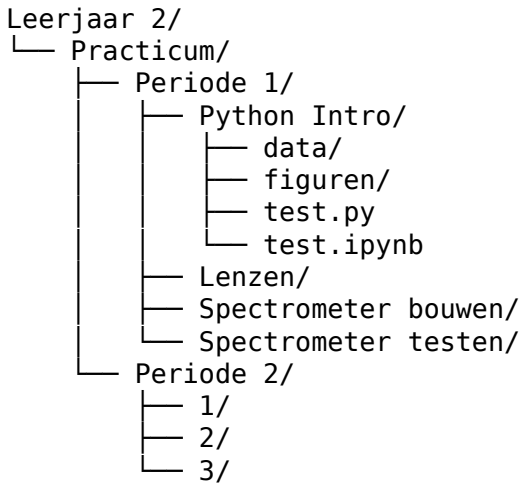
Tijdens practica verzamel je verschillende bestanden, zoals Python-scripts, meetdata en figuren. Het is daarom belangrijk om vanaf het begin een duidelijke **mappenstructuur** te gebruiken. Zo blijven bestanden makkelijk terug te vinden en voorkom je dat verschillende projecten of versies door elkaar raken.
Maak voor ieder practicumserie of project een **aparte hoofdmap** en verdeel deze onder per practicum. Een goede mappenstructuur is een belangrijk onderdeel van zorgvuldig en reproduceerbaar werken.

Opdracht 9 — mappenstructuur

In deze opdrachtg aan we aan de slag om een goede structuur neer te zetten en deze te laden vanuit VScode.

1. Open in VScode **File** → **Open Folder…** → **Documenten**.
2. Creëer een nieuwe map op de locatie die handig is bijvoorbeeld op de **one-drive**.
3. Maak de mappenstructuur na uit Fig. 4.

Ieder practicum dat je start zal je beginnen met **Open Folder…** en dan de juiste practica kiezen. Zo zorg je ervoor dat je altijd je data, figuren en gebruikte scripts kunt terugvinden.



Figuur 4: Boomdiagram voor apparaatcommunicatie.

Test Python

Een **Jupyter Notebook** is een document waarin Python-code in afzonderlijke blokken, zogenaamde *cellen*, kan worden uitgevoerd. Je kunt ze herkennen aan de extensie **.ipynb**[1]. Het grote voordeel hiervan is dat je code stap voor stap kunt uitvoeren en het resultaat direct onder de betreffende cel verschijnt. Naast code kunnen notebooks ook tekst, formules en figuren bevatten.

Dit maakt notebooks bijzonder geschikt voor practica en data-analyse: je kunt meetgegevens inlezen, berekeningen uitvoeren en grafieken maken zonder telkens het volledige programma opnieuw uit te voeren. Tijdens de practica zullen we Jupyter Notebooks daarom regelmatig gebruiken.

Opdracht 10 — Test Jupyter Notebook (.ipynb)

In de vorige opdrachten hebben we de basis gelegd voor een overzichtelijke mappenstructuur en hebben we de benodigde software geïnstalleerd. Nu gaan we controleren of alles correct werkt.

1. Open in VS Code via **File** → **Open Folder…** de map **Practicum Python Intro**.
2. Open het bestand `script.ipynb` en voeg met `+ Code` een nieuwe codecel toe.
3. Neem de onderstaande code over en voer de cel uit met `Shift+Enter`.
4. Als er nog geen Python-omgeving is geselecteerd, vraagt VS Code om een *kernel* te kiezen. Selecteer hier de eerder aangemaakte `practicum_omgeving`. Deze kan soms wat verstopt zitten.
5. Je kunt de actieve omgeving ook controleren of wijzigen via `Select Kernel` rechtsboven in het notebook. Controleer of hier `practicum_omgeving` is geselecteerd.
6. Controleer of de uitvoer van de code overeenkomt met wat je verwacht.

script.ipynb

```python
print("Hello world")

import numpy as np

a = np.array([1, 2, 3, 4])

print("Array a:", a)
print("Array a^2:", a**2)
```

Naast Jupyter Notebooks wordt Python veel gebruikt in de vorm van **Python-scripts**. Dit zijn tekstbestanden met de extensie **.py**. In tegenstelling tot een Jupyter Notebook bestaat een Python-script niet uit losse cellen: bij het uitvoeren wordt het programma in principe van boven naar beneden uitgevoerd.

Bij een Jupyter Notebook hebben we hiervoor een *kernel* geselecteerd. Voor een Python-script moet VS Code weten welke **Python-interpreter** gebruikt moet worden. Het is belangrijk om hier onze eerder gemaakte `practicum_omgeving` te kiezen. Anders kan het gebeuren dat het script met een andere Python-installatie wordt uitgevoerd waarin bijvoorbeeld onze packages niet zijn geïnstalleerd.

Opdracht 11 — Test Python-script (.py)

1. Zorg dat de map **Practicum Python Intro** nog geopend is in VS Code.
2. Open het bestand met de naam `script.py`.
3. Selecteer de juiste Python-interpreter. Druk hiervoor op `Ctrl+Shift+P` en zoek naar: `Python: Select Interpreter`

[1] Zorg ervoor dat extensies zichtbaar zijn bij bestanden. Kun je dit niet zien, spreek dan even de practicumbegeleider aan.

4. Selecteer de eerder gemaakte `practicum_omgeving`. Als deze niet direct zichtbaar is, kijk dan bij de overige beschikbare Python-omgevingen.
5. Neem onderstaande code over in `script.py` en sla het bestand op.
6. Voer het programma uit met de knop **Run Python File** rechtsboven in VS Code.
7. Onderaan VS Code wordt nu een **Terminal** geopend. Controleer of het programma zonder foutmeldingen wordt uitgevoerd.

script.py

```python
import sys
import numpy as np

print("Hello world")

a = np.array([1, 2, 3, 4])

print("Array a:", a)
print("Array a^2:", a**2)

print("\nPython wordt uitgevoerd vanuit:")
print(sys.executable)
```

Opdracht 12 — Controleer de Python-omgeving

Bekijk de laatste regel van de uitvoer van je programma. Hier staat het pad naar het bestand `python.exe` waarmee het programma is uitgevoerd.

1. Controleer of in dit pad de naam `practicum_omgeving` voorkomt.
2. Controleer ook linksonder in VS Code welke Python-versie en environment geselecteerd zijn.
3. Staat hier niet `practicum_omgeving`? Selecteer dan opnieuw de juiste interpreter via `Python: Select Interpreter` en voer `script.py` opnieuw uit.

Afronding

1. Heb je alle opdrachten afgerond? Laat dit dan controleren door de practicumbegeleider.
2. Lukt het niet om alles op tijd af te ronden, zorg ervoor dat je dit dan thuis afrond! Mocht je er thuis niet uitkomen, spreek dan de practicumbegeleider even aan, diegene zal dan met je meekijken.

Appendix: Materialen Lijst Practica

- Laptop met stabiele internetverbinding.

Practicum: Bepaling van de brandpuntsafstand van een onbekende lens

Leerdoelen

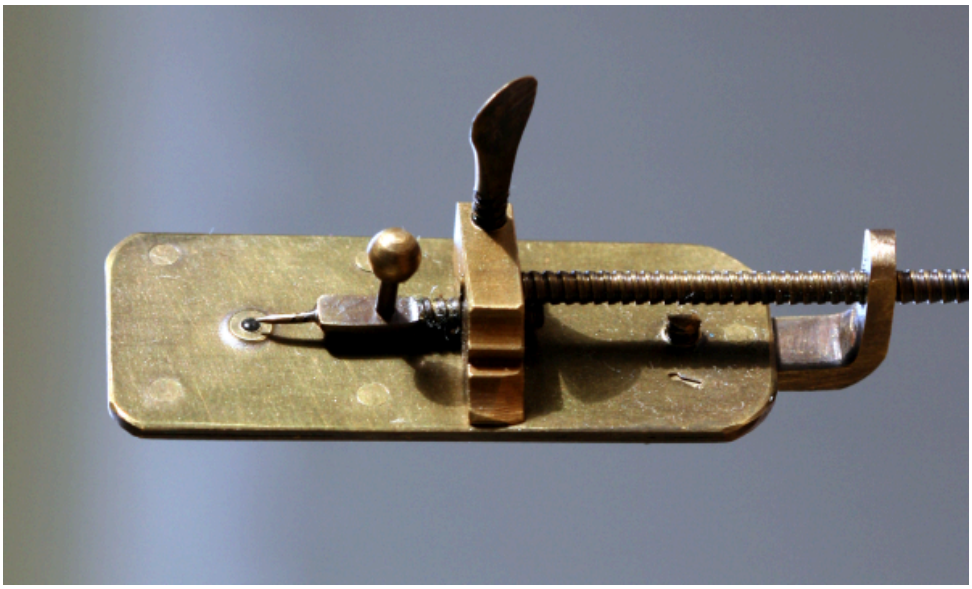
- ☐ Begrip van het belang van het gebruik van een (digitaal) labjournaal bij natuurkundig onderzoek.
- ☐ Zelfstandig kunnen bijhouden van een (digitaal) labjournaal in natuurkundepractica.
- ☐ Begrip van en toepassen van de lenzenformule.
- ☐ Basisvaardigheid in datamanipulatie en visualisatie in Python.

Veiligheid

1. We werken dit practicum met lenzen en LED lampen. Ga voorzichtig om met de lenzen en pas op voor de LED lampen, deze kunnen heet worden en mogen niet warm opgeborgen worden.
2. De LED lamp kan maximaal 12 volt aan!

Introductie

Al sinds de oudheid gebruiken mensen lenzen om beelden te vergroten of scherp te stellen. De oude Grieken en Romeinen kenden al glaskogeltjes die het licht bundelden, en in de middeleeuwen verschenen de eerste brillen. In de 17e eeuw werden brillen en lenzen steeds verfijnder, en in die tijd begon men microscopen en telescopen te ontwikkelen. Een van de bekendste Nederlanders, Antoni van Leeuwenhoek, bouwde met zelfgeslepen lenzen ontzettend sterke microscopen[2], zie Fig. 5.



Figuur 5: Een replica van één van de eerste microscopen van Antoni van Leeuwenhoek.

Met deze microscopen werden voor het eerst bacteriën, spiervezels en bloedcellen ontdekt. Zijn werk markeert het begin van de microscopie, en laat zien hoe een relatief eenvoudige lens tot grote wetenschappelijke ontdekkingen kan leiden.

De werking van een lens is in essentie eenvoudig: een lens buigt lichtstralen af doordat het licht bij de overgang van lucht naar glas (of andersom) van richting verandert. Hierdoor kunnen lichtstralen die uit één punt van een voorwerp komen, weer in één punt samenkomen: het beeldpunt. Hoe sterk een lens lichtstralen bundelt of spreidt, wordt bepaald door de *brandpuntsafstand* f.

De wiskundige relatie tussen voorwerpafstand v, beeldafstand b en brandpuntsafstand f staat bekend als de *lenzenformule*:

$$\frac{1}{f} = \frac{1}{v} + \frac{1}{b}.$$

In dit practicum ga je deze relatie zelf onderzoeken. Door voor verschillende voorwerpsafstanden v de bijbehorende beeldafstanden b te meten, kun je de brandpuntsafstand van een onbekende lens afleiden. Op deze manier volg je in zekere zin de voetsporen van Van Leeuwenhoek: met een lens de wereld op een nieuwe manier zichtbaar maken.

Voorbereidingsopdracht 1

1. Neem een laptop mee.
2. Zorg voor een goede installatie van Python & Jupyter notebooks, zie practicumhandleiding week 1.

[2] Het vakmanschap van Antoni van Leeuwenhoek was zo uitzonderlijk, dat het tot voor kort een raadsel bleef hoe hij zulke krachtige microscopen kon bouwen. Pas enkele jaren geleden is dit mysterie deels ontrafeld in de nucleaire reactor in Delft (zie dit artikel).

Opdracht 1(a)

Meet voor verschillende voorwerpsafstanden v de bijbehorende beeldafstanden b en vergrotingen.

1. Noteer bij elke meting ook de meetfout σ_b.
2. Leg uit hoe je de grootte van σ_b hebt bepaald. Hoe kun je deze nauwkeuriger bepalen?
3. Noteer deze gegevens in je Digitaal Labjournaal.

Tip! Gebruik de onderstaande code en plak deze in je labjournaal:

Data invoeren

```
import numpy as np

# meetwaarden
b = np.array([])
v = np.array([])
b_err = np.array([])
```

Opdracht 1(b)

Maak in Python een grafiek waarin je b uitzet tegen v. Voeg duidelijke labels en eenheden toe.
Tip! Gebruik de onderstaande code en plak deze in je labjournaal:

Grafiek maken

```
import matplotlib.pyplot as plt

# plotten
plt.errorbar(v, b, yerr=b_err, fmt='o')
plt.xlabel("v (m)")
plt.ylabel("b (m)")
plt.show()
```

Zoals je kunt zien in het figuur is er een duidelijk, maar soms ook verwarrend, verband tussen de beeldafstand en de voorwerpsafstand. De lenzenformule

$$\frac{1}{f} = \frac{1}{v} + \frac{1}{b}, \tag{1}$$

geeft echter niet direct het verband aan tussen b en v.

Opdracht 1(c)

1. Herschrijf de lensvergelijking zodat b wordt uitgedrukt als functie van v en f.
2. Gebruik onderstaande Python-code en vul deze correct aan.
3. Noteer je afleiding in je Digitaal Labjournaal.

Functie maken

```
def beeldafstand(v, f):
    return (...)
```

Opdracht 1(d)

1. Maak een array die loopt van 0 tot 1 meter in stapjes van 1 cm.
2. Teken de functie van de *beeldafstand* voor verschillende waarden van de brandpuntsafstand f. Welke waarde komt het meest overeen met de onbekende lens?

Lineaire span

```
v_fit = np.linspace(0, 1, 101) # in meters
# Gebruik voor elke gekozen f alleen v_fit > f (reeel beeld).
# Bij v = f is de beeldafstand niet eindig.
```

Het tekenen van verschillende waarden van *f* levert een eerste schatting op, maar houdt geen rekening met meetonzekerheden. Grotere afstanden betekenen niet automatisch nauwkeurigere metingen: schat de onzekerheid per meting. We gebruiken hier *lineariseren*: door zowel de beeldafstand als de voorwerpsafstand te inverteren, ontstaat een lineair verband tussen de nieuwe variabelen. Lineariseren is niet hetzelfde als inverteerbaarheid. Bij deze transformatie moeten ook de onzekerheden worden omgerekend.

Opdracht 1(e)

1. Maak een nieuwe grafiek waarbij je $1/b$ uitzet tegen $1/v$.
2. Schrijf de lenzenformule om in de vorm
$$y = ax + c, \qquad y = \frac{1}{b}, \quad x = \frac{1}{v}. \tag{2}$$
3. Maak een nieuwe functie genaamd *lineair*, met als invoer `inverse_v = 1/v` en parameter `f`. De uitvoer is `1/b`.
4. Wat valt je op aan de grafiek? Wat is de betekenis van het snijpunt met de y-as?
5. Voeg nu enkele lijnen toe met verschillende waarden van de brandpuntsafstand f.

Functie maken

```
def lineair(inverse_v, f):
    return (...)
```

Zoals je misschien hebt opgemerkt, ziet de grafiek er nu aanzienlijk beter uit. De vraag blijft echter wat het snijpunt met de y-as betekent en welke waarde hier precies bij hoort. Om dit op een natuurkundig correcte manier uit te werken, gebruiken natuurkundigen vaak modellen. In dit geval is dat natuurlijk de functie $lineair$.
Nu moeten we ons model nog toetsen (*fitten*!) aan de gemeten data. Dit is in principe een optimalisatieprobleem: we moeten de fitwaarde *f* zo kiezen dat de afwijkingen tussen model en meting zo klein mogelijk zijn[3]. Hiervoor maken we gebruik van de rekenkracht van Python, met de wetenschappelijke module **scipy**.

Data fitten

```
from scipy.optimize import curve_fit

# Schatting van parameters
p0 = [1] # eerste schatting van f, 1 meter

# aanroepen fit
popt, pcov = curve_fit(lineair, 1/v, 1/b, sigma=(...), p0=p0,
                       absolute_sigma=True)

# resultaat printen
print("$f$ =", popt, "+-", np.sqrt(np.diag(pcov)))
```

[3] Het maakt niet uit als deze stap nog onduidelijk is; het is een trucje dat tijdens wiskunde 2.1 verder wordt uitgelegd.

Opdracht 1(f)

1. Bereken de fout op $\frac{1}{b}$, gebruik hiervoor de relatieve fout van b. Als dit niet lukt, sla deze stap dan over.
2. Bepaal f door middel van een fit, gebruik hiervoor de *scipy*-module. Mocht het bepalen van de fout in de vorige opdracht niet gelukt zijn, verwijder dan de argumenten `sigma=(...)` en `absolute_sigma=True`. De onzekerheid wordt dan geschat uit de residuen, onder de aanname van gelijke spreiding in `1/b`.
3. Wat is de gevonden waarde voor *f*, en welke onzekerheid hoort hierop?

Afronding

1. Zet de gevonden figuren in je digitaal labjournaal en maak een duidelijk onderschrift waarin je uitlegt wat de figuren laten zien.
2. Vul de **Resultaat- en Discussie**-sectie aan in je labjournaal. Hierin moet de uiteindelijke waarde van f (met onzekerheid) staan en een korte discussie of deze waarde **realistisch** en **betrouwbaar** is.

Appendix: data fitten met `curve_fit`

Wanneer je meetpunten hebt (x_i, y_i) met meetfouten σ_i, wil je vaak een model $y(x;\theta)$ vinden dat de data zo goed mogelijk beschrijft. Hierbij zijn θ de onbekende parameters (bijvoorbeeld een brandpuntsafstand f, of de richtingscoëfficiënt a en het intercept b van een lijn).

Least squares-principe.

Het idee van een least squares-fit is dat je de afwijkingen tussen model en meting (de residuen) minimaliseert:

$$r_i = y_i^{(\text{gemeten})} - y(x_i;\theta).$$

Zonder meetfouten.

Als de individuele meetonzekerheden σ_i niet bekend zijn, minimaliseer je de som van de kwadraten van de residuen:

$$S(\theta) = \sum_i r_i^2.$$

Dit heet een ordinary least squares-fit (OLS): alle meetpunten wegen even zwaar mee.

Met meetfouten.

Als de onzekerheden σ_i wel bekend zijn, deel je de residuen door de fout en minimaliseer je

$$\chi^2(\theta) = \sum_i \left(\frac{r_i}{\sigma_i}\right)^2.$$

Dit heet een weighted least squares-fit (WLS) of χ^2-fit. Bij onafhankelijke, normaal verdeelde meetfouten met bekende standaardafwijkingen komt dit overeen met maximum-likelihoodschatting, als het model correct is en de onzekerheid in x verwaarloosbaar is.

Uitkomst.

De functie `curve_fit` levert:

- de optimale parameters $\hat{\theta} =$ `popt`;
- de covariantiematrix $C =$ `pcov`, die een schatting geeft van de onzekerheden en correlaties van de parameters.

De standaardfout van parameter θ_j volgt uit

$$\sigma_{\theta_j} = \sqrt{C_{jj}}.$$

In Python komt dit overeen met:

- geen `sigma` meegeven $\rightarrow$ ordinary least squares (som van gekwadrateerde residuen);
- wel `sigma` meegeven $\rightarrow$ weighted least squares (χ^2).

Interpretatie.

De kwaliteit van de fit hangt af van:

- het gekozen model $y(x;\theta)$ (is het fysisch correct en past het bij de data?);
- de spreiding van de meetpunten en hun onzekerheden σ_i;
- de initiële schatting p_0 voor de parameters: bij een niet-lineaire fit kan een slechte start leiden tot lokale minima.

Een directe fit aan de oorspronkelijke meetwaarden voorkomt dat je de data en hun onzekerheidsverdeling hoeft te transformeren. Ook een directe fit vereist een geschikt model, realistische onzekerheden en een bruikbare startschatting; `curve_fit` garandeert geen globaal optimum. Bij de linearisatie geldt voor kleine relatieve onzekerheden: `sigma_inverse_b = b_err / b**2`. De fit hieronder behandelt de x-waarden als exact.

Voorbeeld: fitten met `curve_fit`

```python
import numpy as np
from scipy.optimize import curve_fit

# Modeldefinitie (hier: rechte lijn)
def linear(x, a, b):
    return a * x + b

# Meetdata (x,y) met fouten
xdata = np.array([1,2,3,4,5])
ydata = np.array([1.1,1.9,3.2,3.9,5.1])
yerr  = np.array([0.1]*5)

# Eerste schatting van parameters a,b
p0 = [1, 0]

# Fit aanroepen
popt, pcov = curve_fit(linear, xdata, ydata, sigma=yerr, p0=p0,
                       absolute_sigma=True)

# Resultaten
perr = np.sqrt(np.diag(pcov))
print("a =", popt[0], "+-", perr[0])
print("b =", popt[1], "+-", perr[1])
```

Appendix: Materialen Lijst Practica

- Onbekend Lens (1x)
- Optische rail
- Dia-houders + dia (letter F)
- Projectiescherm
- Rolmaat
- Ruiters 3 hoog, 1 laag.
- LED bron, 2 kabels en voedingsbron.

Practicum: Spectrometer bouwen

Leerdoelen

- ☐ Je kunt zelfstandig een onderzoeksvraag en (onderbouwde) verwachting opstellen over jouw experiment.
- ☐ Je bent in staat om een meetplan en experimentele opstelling te ontwerpen voor een relatief eenvoudig optica experiment (spectrometer).
- ☐ Je kunt zelfstandig een optische opstelling bouwen vanuit een gegeven ontwerp.
- ☐ Basisvaardigheid in datamanipulatie, visualisatie én model toetsen in Python.

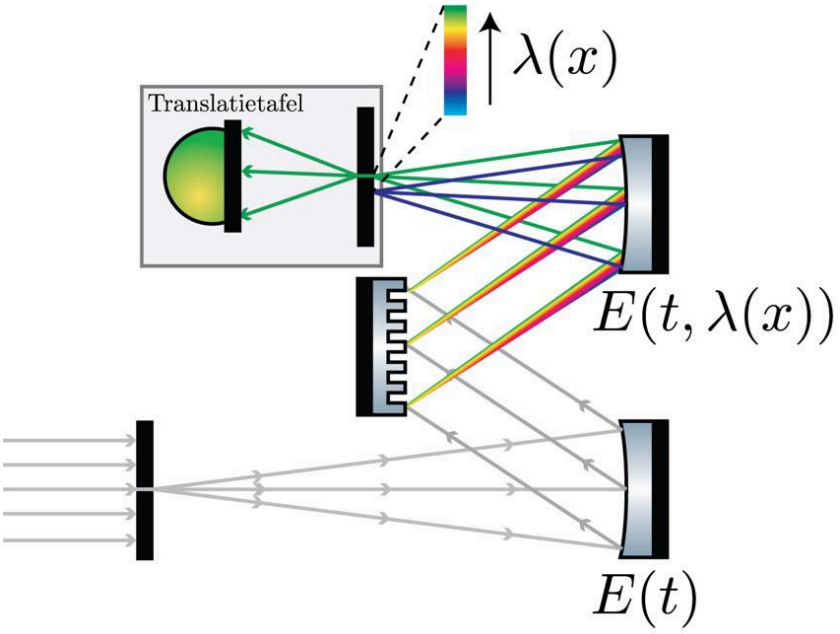
Veiligheid

1. We werken dit practicum met spectraallampen, deze worden heet en kunnen niet warm opgeborgen worden.
2. De lensen, spiegeltjes en overige optische onderdelen zijn erg duur & kwetsbaar! Ga hier voorzichtig mee om.
3. Dit practicum duurt twee weken, je hoeft de opstelling dus niet tussentijds af te breken.

Practicum week 1

Introductie

De oorsprong van de spectroscopie ligt in het begin van de 19e eeuw. Isaac Newton had in de 17e eeuw al laten zien dat wit licht met behulp van een prisma in een spectrum uiteenvalt, maar pas veel later werd dit verschijnsel systematisch onderzocht. In 1802 beschreef William Hyde Wollaston donkere lijnen in het zonnespectrum, die hij met een eenvoudig prisma waarnam. Enkele jaren later, in 1814, maakte Joseph von Fraunhofer nauwkeurige waarnemingen van deze lijnen. Hij ontwikkelde een verbeterde spectroscoop waarin een smalle spleet, een collimator en een prisma waren gecombineerd, waardoor een veel scherper spectrum zichtbaar werd. Fraunhofer kon hiermee honderden absorptielijnen in het zonlicht vastleggen, de later zogenoemde Fraunhoferlijnen. Deze uitvinding markeerde het begin van de spectroscopie als onderzoeksinstrument.
In bredere zin is een spectrometer een instrument waarmee het spectrum van een natuurkundige grootheid kan worden bepaald. Naast optische spectrometers bestaan er bijvoorbeeld massaspectrometers, NMR-spectrometers en Auger-elektronspectrometers. In dit experiment richten we ons op een optische spectrometer (spectrofotometer), waarmee zichtbaar licht kan worden geanalyseerd. Het principe berust erop dat licht in componenten van verschillende golflengtes wordt gescheiden, zie Fig. 6. Op die manier kan informatie worden verkregen over de samenstelling en eigenschappen van materialen. Toepassingen zijn er in tal van vakgebieden: in de chemie om stoffen te identificeren, in de astronomie om de atmosfeer van verre planeten te onderzoeken, in de natuurkunde om energieniveaus van atomen en moleculen te bepalen, en in de materiaalkunde om de zuiverheid van materialen te testen.



Figuur 6: Schematische weergave van een reflectiespectrometer: licht gaat door een spleet, wordt via een holle spiegel parallel gemaakt, gescheiden door een tralie en opgevangen door een detector.

In deze vierdelige practicumreeks gaan jullie aan de slag met het opbouwen van een **transmissie** spectrometer, het kalibreren en doormeten van twee onbekende (spectraal)lampen (week 1). In week 2 gaan jullie de spectrometer gebruiken voor het doen van een experiment: het bepalen van de concentratie van een onbekende Cu(II)sulfaat oplossing.

Voorbereidingsopdracht week 1

1. Maak een meetplan voor het bouwen van een **transmissie** spectrometer van maximaal 1 pagina[4]. Formuleer een hoofddoel voor deze week en geef een korte theoretische onderbouwing over hoe je controleert of deze spectrometer dient te werken.
2. Maak een schets van de **transmissie** spectrometer in Powerpoint of Inkscape[5] en zet deze in jouw labjournaal.
3. Maak een Jupyter notebook waarin je het Python script van de video-analyse klaar staat. Je kunt de code van deze Github-pagina gebruiken link.

Theoretische achtergrond

Licht kan worden beschreven als een golf met een bepaalde golflengte λ. Voor zichtbaar licht liggen deze golflengtes tussen ongeveer 380 en $780\,\mathrm{nm}$. Een detector, zoals een camera, meet de intensiteit van dit licht: hoeveel licht er van een bepaalde golflengte aanwezig is.
Om wit licht te ontleden in zijn afzonderlijke golflengtes maken we gebruik van een tralie (grating). Dit is een plaatje met zeer veel fijne, evenwijdige lijntjes. Door diffractie en interferentie wordt licht daarbij onder een hoek afgebogen die afhangt van de golflengte. De relatie tussen de golflengte λ, de spleetafstand d van de tralie en de afbuighoek θ is gegeven door:

$$\sin(\theta) = \frac{n\lambda}{d}, \tag{3}$$

Hier is n de orde van diffractie en d de afstand tussen twee tralielijnen. Deze vorm geldt bij loodrechte inval; θ wordt gemeten ten opzichte van de normaal op het tralie. Ditzelfde principe zorgt er ook voor dat je op een cd of dvd regenboogkleuren ziet verschijnen.
In de spectrometer wordt het uitgespreide licht door de tralie op een camera geprojecteerd. Door de posities op de detector te koppelen aan bekende golflengtes (ijking), kan de gemeten intensiteit worden weergegeven als functie van de golflengte: $I(\lambda)$. Het resultaat noemen we een spectrum: een overzicht van welke golflengtes in de lichtbron aanwezig zijn en met welke relatieve sterkte.

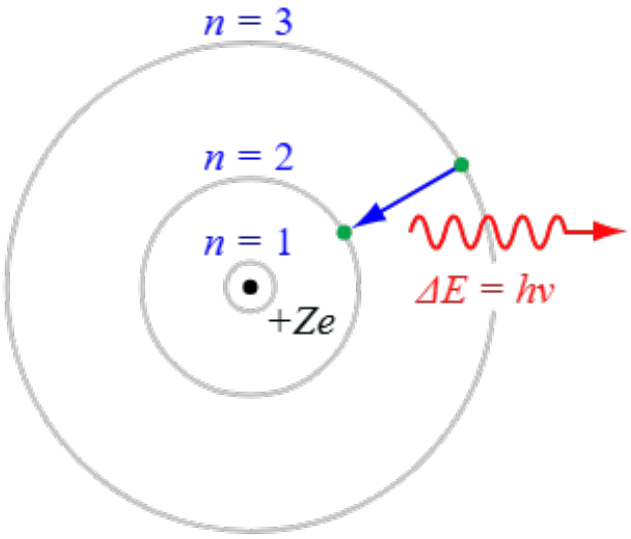
Opdracht 1(a)

1. Ga aan de slag met het bouwen van de spectrometer.
2. Gebruik als lichtbron een LED licht.
3. Houd je voortgang bij in het labjournaal.

Het ijken van de spectrometer

Bij het analyseren van lichtbronnen wordt vaak gebruikgemaakt van spectraallampen. Dit zijn lampen die bij opwarming of excitatie licht uitzenden met karakteristieke spectraallijnen, bijvoorbeeld van kwik, natrium of waterstof. Elke lijn correspondeert met een specifieke elektronische overgang in het atoom of molecuul van het gebruikte gas, zie Fig. 7. Het spectrum van zo'n lamp bestaat daarom uit een aantal scherpe pieken op vaste golflengtes, in plaats van een doorlopend kleurverloop.

[4] Zie Brightspace voor een korte uitleg over een meetplan.
[5] Inkscape is bijzonder geschikt voor het tekenen van optische opstellingen. Via Inkscape kun je een Optical-Table Library openen, waarna je de onderdelen op de juiste plek kan zetten. Zie hiervoor deze link.



Figuur 7: Het Borh model. Bij transititie van een electron naar een andere baan komt er een specifieke kleur licht vrij: de spectraallijn.

Om deze lijnen kwantitatief te analyseren, kan men de gemeten intensiteit van een spectraallijn bepalen aan de hand van een Lorentz-functie[6]:

$$I(\lambda) = I_0 \frac{\gamma^2}{(\lambda - \lambda_0)^2 + \gamma^2} \tag{4}$$

waarbij λ_0 de centrale golflengte van de lijn is, γ de halve breedte op halve hoogte van de piek, en I_0 de maximale intensiteit. Door een Lorentz-fit uit te voeren, kan men nauwkeurig de centrale golflengte en breedte van een spectraallijn bepalen. Deze *formele* methode is echter vrij lastig en het succes hiervan hangt af van de kwaliteit van de data. Een eenvoudigere methode is gebruik te maken van een peak-detection algorithme van Python.

Eenvoudige piekdetectie

```python
import numpy as np
from scipy.signal import find_peaks

# x en y zijn numpy arrays
peaks, _ = find_peaks(y, height=0)
peak_x = x[peaks]
peak_y = y[peaks]

print("Gevonden pieken:")
for xi, yi in zip(peak_x, peak_y):
    print(xi, yi)
```

Spectraallampen worden ook gebruikt om spectrometers te ijken. Wanneer we een lamp met bekende golflengtes meten, verschijnen de karakteristieke intensiteitspieken op specifieke posities op de detector. Bijvoorbeeld, een blauwe lijn van $450\,\mathrm{nm}$ kan helemaal links op de detector verschijnen, terwijl een gele lijn van $600\,\mathrm{nm}$ in het midden zichtbaar is. Met deze referentielijnen kunnen we alle posities op de detector koppelen aan hun bijbehorende golflengte. Door de bekende pieken als uitgangspunt te nemen, kunnen we de golflengtes van de tussenliggende punten nauwkeurig bepalen. Dit proces, het ijken van de spectrometer, is essentieel om betrouwbare en reproduceerbare metingen van onbekende spectra te kunnen uitvoeren. In appendix 2, tabel 3, staat de lijst van

[6]Een Lorentz-profiel verschijnt bij resonanties doordat een aangeslagen toestand een beperkte levensduur heeft, wat wiskundig de karakteristieke Lorentz-vorm oplevert, zie vergelijking 4. Meer informatie is te vinden hier.

spectraallijnen van helium, natrium, neon en kwik. Gebruik deze lijst om een tabel te maken voor de kalibratie, zie tabel 2.

Tabel 2: Calibratiepunten: pixelwaarde vs. golflengte

x-pixel	Golflengte (nm)
100	430.3
500	589.0
900	706.5
⋮	⋮
⋮	⋮

Deze tabel is een fictief rekenvoorbeeld; gebruik voor de ijking je eigen gemeten pixelposities en bekende lijnen van de referentielamp. Een lineaire kalibratie is een benadering: controleer met meerdere lijnen of deze in het gebruikte bereik voldoende nauwkeurig is. Het model is:

$$\text{Golflengte (nm)} = a\,\text{x-pixel} + b, \tag{5}$$

hierbij zijn a en b fitparameters die we moeten bepalen. Mocht ik nu een spectraallijn vinden die niet in tabel 3 staat, dan kan ik met een gegeven a en b de golflengte toch netjes bepalen.

Opdracht 1(b)

1. Ga aan de slag met het ijken van de spectrometer.
2. Meet het spectrum van een bekende lamp. Zet deze in je labjournaal.
3. Bepaal van iedere emissielijn de pixelpositie van de piek met behulp van het peak-detection-algoritme van SciPy. Koppel deze posities aan bekende referentiegolflengten om de ijklijn te bepalen. Mocht dit te lastig zijn en kom je hier niet uit, probeer de locaties (x-pixels) te schatten op het oog.
4. Houd je voortgang bij in het labjournaal.

Als je de spectrometer hebt geijkt, dan kan je jouw spectrometer gebruiken om ook van een onbekende lamp het spectrum te analyseren. Misschien lukt het jullie ook wel om op basis van de gemeten golflengten én tabel 3 om de juiste lamp te vinden.

Opdracht 1(c)

1. Meet de golflengten van de nieuwe lamp.
2. Welke lamp(en) komen overeen met jouw gevonden golflengten?
3. Houd je voortgang bij in het labjournaal.

Afronding week 1

1. Zet de gemaakte figuren in je digitaal labjournaal en maak een duidelijk onderschrift waarin je uitlegt wat de figuren laten zien.
2. Vul de **Resultaat- en Discussie**-sectie aan in je labjournaal. Hierin moet de uiteindelijke waarde van jouw experiment (met onzekerheid) staan en een korte discussie of deze waarde **realistisch** en **betrouwbaar** is.

Practicum week 2

Absorptie en transmissie van licht

In deze week gaan jullie metingen doen aan oplossingen van koper(II)sulfaat ($CuSO_4$). Dit zout geeft in oplossing een heldere blauwe kleur. Wanneer we een oplossing van $CuSO_4$ voor een witte lichtbron (LED) plaatsen, wordt een deel van het spectrum van het licht geabsorbeerd. Omdat de oplossing blauw van kleur is, kan het absorptiespectrum van $CuSO_4$ voorspeld worden op basis van de complementaire kleur (zie tabel. 4).

Wanneer licht door een materiaal gaat, kan het worden gereflecteerd, geabsorbeerd of doorgelaten. De hoeveelheid doorgelaten licht drukken we uit met de *transmissie* T, gedefinieerd als de verhouding tussen de doorgelaten intensiteit I en de oorspronkelijke intensiteit I_0:

$$T = \frac{I}{I_0}. \tag{6}$$

De *absorbantie* A wordt gedefinieerd met de logaritme met grondtal 10:

$$A = -\log_{10}(T) = -\log_{10}\left(\frac{I}{I_0}\right). \tag{7}$$

Meet I_0 met een blanco (dezelfde cuvet met oplosmiddel) en I met de oplossing, bij dezelfde golflengte en instellingen. Corrigeer beide signalen voor de donkerachtergrond. Volgens de wet van Beer–Lambert is de absorbantie bij een vaste golflengte en weglengte evenredig met de concentratie, zolang de wet toepasbaar is.

In het practicum zullen jullie verschillende concentraties $CuSO_4$-oplossingen meten en het verband tussen concentratie en absorbantie onderzoeken. Met behulp van een door jullie opgestelde ijklijn kunnen jullie uiteindelijk de concentratie bepalen van een onbekende oplossing.

Voorbereidingsopdracht week 2

1. Maak een meetplan voor het meten van een Cu(II)sulfaat oplossing met behulp van door jullie gemaakte spectrometer. Het meetplan moet maximaal 1 pagina zijn & aanwezig in het labjournaal[7]. Formuleer het hoofddoel/onderzoeksvraag, geef een theoretische verwachting en maak een meetstrategie.
2. Hoe gaat je ijklijn eruit zien? Wat staat er op de x-as en wat staat er op de y-as?
3. Maak een Jupyter notebook waarin je een voorbeeld analyse hebt uitgewerkt.

[7] Zie Brightspace voor een korte uitleg over een meetplan.

Opdracht 2(a)

1. Bepaal het absorptiespectrum van ($CuSO_4$). Zet dit figuur in je labjournaal.
2. Wat is de concentratie van de onbekende oplossing. Denk aan eenheden en de onzekerheid! Zet de bijbehorende ijklijnen ook in je labjournaal.
3. Houd je voortgang bij in het labjournaal.

Afronding week 2

1. Zet de gemaakte figuren in je digitaal labjournaal en maak een duidelijk onderschrift waarin je uitlegt wat de figuren laten zien.
2. Vul de **Resultaat- en Discussie**-sectie aan in je labjournaal. Hierin moet de uiteindelijke waarde van jouw experiment (met onzekerheid) staan en een korte discussie of deze waarde **realistisch** en **betrouwbaar** is.

Appendix 1: Materialen en tips

Materialenlijst

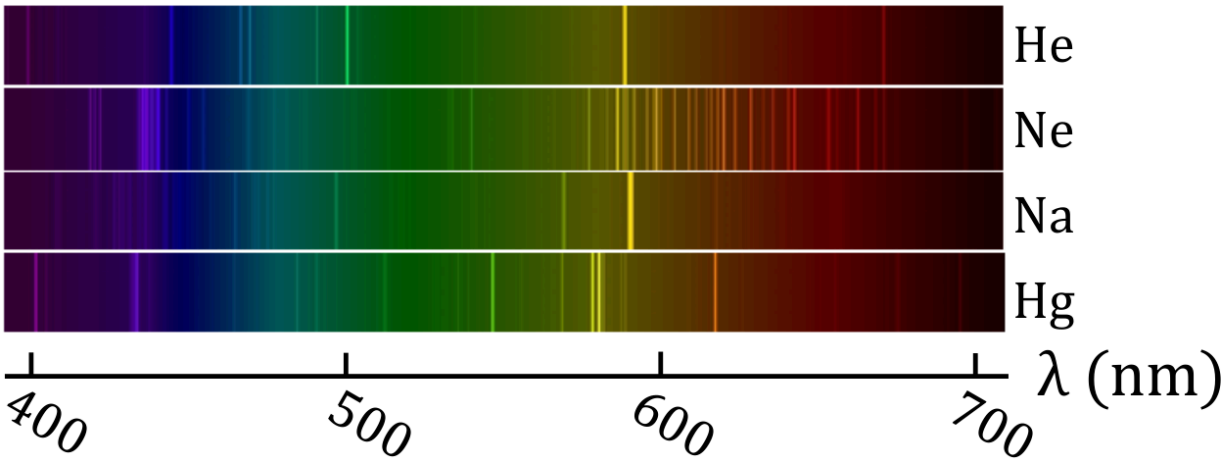
Voor dit practicum zijn de volgende componenten nodig:

- 1× verstelbare dunne spleet
- 1× concaaf spiegel, f = 15 cm
- 1× concaaf spiegel, f = 10 cm
- 1× transmissie tralie
- 1× camera (fotodetector)
- 1× lichtbron (LED of spectraallamp)

Tips

- Bepaal de onderlinge afstanden van alle componenten, beginnend bij de entree-spleet en de eerste concaaf spiegel. Let hierbij op de parallelle terugkaatsing van de stralen.
- Onderzoek op welke afstand de spiegels geplaatst moeten worden en hoe het licht optimaal op de tralie valt.
- Maak een schets van de stralengang door de opstelling.
- Zorg dat de opstelling op een breadboard van 45 cm bij 60 cm past.

Appendix 2: Spectraallijnen gasontladingslampen



Figuur 8: De (sterke) spectraallijnen van helium, neon, natrium en kwik.

Tabel 3: De golflengte van de belangrijkste spectraallijnen van natrium (Na), helium (He), neon (Ne) en kwik (Hg) in nanometer.

Na	He	Ne	Hg
497.7 nm	388.9 nm	585.2 nm	404.7 nm
568.8 nm	447.1 nm	588.2 nm	435.8 nm
589.0 nm	471.3 nm	603.0 nm	546.1 nm
589.6 nm	492.2 nm	614.3 nm	577.0 nm
615.4 nm	501.6 nm	640.2 nm	579.1 nm
819.5 nm	587.6 nm	650.7 nm	623.4 nm
	667.8 nm	659.9 nm	690.8 nm
	706.5 nm	703.2 nm	706.5 nm
		717.4 nm	
		724.5 nm	
		743.9 nm	
		748.9 nm	
		753.6 nm	
		837.8 nm	

Appendix 3: Data fitten

Voorbeeld: fitten met `curve_fit`

```python
import numpy as np
from scipy.optimize import curve_fit

# Lineair model
def linear(x, a, b):
    return a * x + b

# Voorbeelddata met fouten
xdata = np.array([1,2,3,4,5])
ydata = np.array([1.1,1.9,3.2,3.9,5.1])
yerr  = np.array([0.1]*5)

# Initiële schatting van parameters a,b
p0 = [1, 0]

# Fit uitvoeren
popt, pcov = curve_fit(linear, xdata, ydata, sigma=yerr, p0=p0,
                       absolute_sigma=True)

# Resultaten en onzekerheden
perr = np.sqrt(np.diag(pcov))
print("a =", popt[0], "+-", perr[0])
print("b =", popt[1], "+-", perr[1])
```

Appendix 4: Complementaire kleuren

Tabel 4: Relatie tussen geabsorbeerde golflengte, energie en waargenomen complementaire kleur.

λ geabsorbeerd	E (eV)	Geabsorbeerde kleur	Waargenomen kleur
$> 700\ \mathrm{n\ m}$	< 1.77	Infrarood	Niet waarneembaar
700 nm–635 nm	1.77–1.95	Rood	Groen
635 nm–590 nm	1.95–2.10	Oranje	Blauw
590 nm–560 nm	2.10–2.21	Geel	Violet
560 nm–520 nm	2.21–2.38	Groen	Rood
520 nm–490 nm	2.38–2.53	Cyaan	Rood–oranje
490 nm–450 nm	2.53–2.76	Blauw	Oranje
450 nm–400 nm	2.76–3.10	Violet	Geel
$< 400\ \mathrm{n\ m}$	> 3.10	Ultraviolet (UV)	Niet waarneembaar

Practica periode 2

Materie en Kwantumfenomenen

Practicum: Open-loop regeling van een aeropendulum

Leerdoelen

- ☐ Je kunt de dynamische respons van een simpel open-loop systeem experimenteel bepalen.
- ☐ Je kunt meetgegevens in Python inlezen, verwerken en visualiseren.
- ☐ Je kunt een eenvoudig fysisch model vergelijken met experimentele meetresultaten.

Veiligheid

1. Houd tijdens de metingen voldoende afstand van de draaiende propeller en raak deze nooit aan wanneer de voeding is ingeschakeld.
2. Controleer vóór iedere meting of het aeropendulum vrij kan bewegen en of zich geen objecten binnen het bewegingsbereik bevinden.
3. Twijfel je over een handeling of apparaat? Vraag dit altijd aan de practicumbegeleider.

Introductie

Regeltechniek speelt een belangrijke rol binnen de technische natuurkunde. Veel technische systemen, zoals drones, robots, satellieten, productieprocessen en autonome voertuigen, moeten automatisch bestuurd worden om een gewenste positie of toestand te bereiken. Voordat een regelsysteem ontworpen kan worden, is het belangrijk om eerst het gedrag van het systeem zelf goed te begrijpen. Dit gebeurt vaak door het systeem in open-loop te bestuderen.

Bij een open-loop systeem wordt een ingangssignaal aangeboden zonder dat de uitgang wordt teruggekoppeld naar de regeling. De besturing corrigeert dus niet voor verstoringen of afwijkingen. Hoewel deze manier van regelen relatief eenvoudig is, vormt zij de basis voor vrijwel alle moderne regelsystemen. Door de respons van een systeem op verschillende ingangssignalen te meten, kunnen eigenschappen zoals de gevoeligheid, tijdconstante, demping en niet-lineariteiten worden bepaald.

Een eenvoudig voorbeeld van open-loop regeling is een magnetron. Je stelt bijvoorbeeld een vermogen van 800 W en een tijd van 2 minuten in. De magnetron voert deze instelling uit, maar meet niet of het eten daadwerkelijk de gewenste temperatuur heeft bereikt.

Is het eten na 2 minuten nog te koud? Dan corrigeert de magnetron dit niet automatisch. De uitgang (temperatuur van het eten) wordt namelijk niet teruggekoppeld naar de ingang (vermogen en verwarmingstijd).

In dit practicum onderzoek je het dynamische gedrag van een aeropendulum. Dit systeem bestaat uit een arm die om een draaipunt kan roteren en wordt aangedreven door een elektrische propeller. De propeller levert een stuwkracht die de arm omhoog beweegt, terwijl de zwaartekracht de arm terug naar beneden probeert te brengen. Door de motorspanning stapsgewijs te variëren ontstaat een evenwicht tussen deze twee krachten, waardoor verschillende stationaire hoeken en dynamische responsen kunnen worden bestudeerd.

Tijdens het practicum meet je de hoek van het aeropendulum als functie van de tijd voor verschillende motorspanningen. De meetgegevens worden vervolgens in geanalyseerd om de dynamische eigenschappen van het systeem te bepalen en te vergelijken met een eenvoudig fysisch model. De kennis die in dit practicum wordt opgedaan vormt de basis voor een vervolgpracticum, waarin hetzelfde systeem met behulp van terugkoppeling (*closed-loop control*) automatisch zal worden geregeld.

Theorie

Een aeropendulum bestaat uit een draaibare arm met aan het uiteinde een elektromotor en propeller. De propeller levert een stuwkracht waardoor de arm omhoog beweegt. De zwaartekracht werkt deze beweging tegen. Door het vermogen van de motor te variëren kan de positie van het aeropendulum worden beïnvloed.

In dit practicum wordt het systeem in **open-loop** onderzocht. Dit betekent dat het vermogen van de motor vooraf wordt ingesteld en niet automatisch wordt aangepast op basis van de gemeten hoek. Voor iedere instelling wordt vervolgens onderzocht hoe het aeropendulum reageert.

De propeller levert een stuwkracht F_T op een afstand l_T van het draaipunt. Het bijbehorende moment is

$$M_T = l_T F_T. \tag{8}$$

Tegelijkertijd oefent de zwaartekracht een moment uit op het systeem. Wanneer het zwaartepunt zich op een afstand l van het draaipunt bevindt, is dit moment

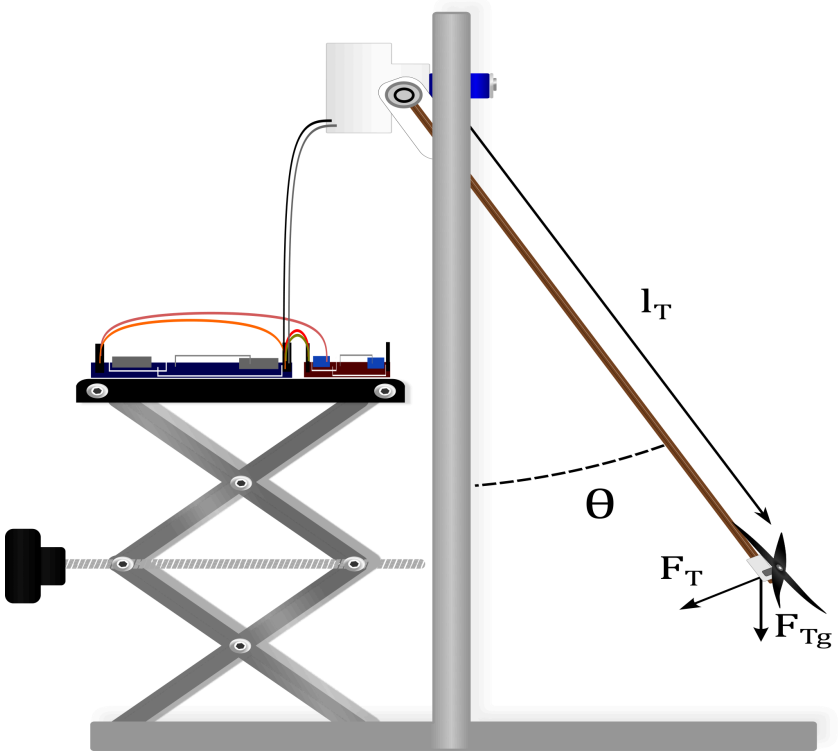
$$M_g = mgl \sin\theta, \tag{9}$$

waarbij m de massa van het bewegende systeem is, g de zwaartekrachtsversnelling en θ de hoek van de arm.

Volgens de tweede wet van Newton voor rotatie wordt de beweging bepaald door de som van deze momenten:

$$J\ddot{\theta} = l_T F_T - mgl \sin\theta, \tag{10}$$

waarbij J het massatraagheidsmoment van het aeropendulum rond het draaipunt is.



Figuur 9: Schematische weergave van het aeropendulum en de relevante krachten.

De stuwkracht van de propeller wordt bepaald door het vermogen dat aan de motor wordt geleverd. Het elektrisch ingangsvermogen is

$$P_{\text{el}} = UI, \tag{11}$$

met U de spanning over de motor en I de stroom door de motor. Slechts een deel van dit vermogen wordt door de motor en propeller omgezet in mechanisch vermogen. Met een rendement η kan dit worden geschreven als

$$P_{\text{motor}} = \eta U I. \tag{12}$$

De stuwkracht kan daarom worden beschouwd als een functie van het geleverde vermogen,

$$F_T = f(P_{\text{motor}}). \tag{13}$$

De precieze vorm van deze relatie hangt af van onder andere de motor en de propeller en hoeft niet lineair te zijn.

Wanneer het aeropendulum uiteindelijk tot stilstand komt, is de hoekversnelling nul. Het moment van de propeller is dan gelijk aan het moment van de zwaartekracht:

$$l_T F_T = mgl \sin\theta. \tag{14}$$

Hieruit volgt

$$\theta = \arcsin\left(\frac{l_T F_T}{mgl}\right). \tag{15}$$

Een groter motorvermogen leidt in het algemeen tot een grotere stuwkracht en daarmee tot een grotere evenwichtshoek. Tijdens het practicum wordt deze relatie experimenteel onderzocht door het motorvermogen te variëren en de resulterende hoek van het aeropendulum te meten.

Componenten

Het aeropendulum bestaat uit verschillende elektronische en mechanische componenten. De Arduino stuurt de motor aan via een H-brug en leest tegelijkertijd de hoek van het pendulum uit met een magnetische hoeksensor. Figuur 11 geeft een overzicht van de gebruikte aansluitingen.

Arduino Uno R4 Minima

Een Arduino is een programmeerbaar microcontrollerbord waarmee elektronische componenten kunnen worden aangestuurd en sensoren kunnen worden uitgelezen. In dit practicum wordt een **Arduino Uno R4 Minima** gebruikt.

De Arduino voert continu een programma uit waarin twee taken centraal staan: het aansturen van de elektromotor en het meten van de hoek van het aeropendulum. Hiervoor beschikt de Arduino onder andere over digitale aansluitingen, analoge ingangen en verschillende communicatieprotocollen.

In deze opstelling worden de digitale pinnen D9, D10 en D11 gebruikt voor de aansturing van de motor. De AS5600-hoeksensor communiceert via de pinnen A4 en A5 met de Arduino.

H-brug

Een elektromotor kan niet rechtstreeks vanuit een digitale uitgang van de Arduino worden gevoed. De motor vraagt aanzienlijk meer stroom dan een uitgang van de microcontroller kan leveren. De Arduino levert daarom alleen het *stuursignaal*; het elektrische vermogen voor de motor wordt geleverd door de externe voeding.

Tussen de Arduino en de motor bevindt zich daarom een **L298N H-brug**. Deze schakeling kan de externe voedingsspanning naar de motor schakelen. De naam H-brug komt van de configuratie van de interne schakelaars, die schematisch de vorm van de letter H heeft.

Door verschillende schakelaars te openen en sluiten kan de stroom door de motor in beide richtingen lopen. Daarmee kan ook de draairichting van de motor worden veranderd. Hiervoor gebruikt de L298N de ingangen IN1 en IN2. In onze opstelling worden deze aangestuurd door D10 en D11 van de Arduino.

Pulse Width Modulation

Naast de draairichting willen we ook kunnen regelen hoeveel vermogen de motor ontvangt. Hiervoor wordt **Pulse Width Modulation (PWM)** gebruikt.

Bij PWM wordt de voedingsspanning niet continu veranderd. In plaats daarvan wordt de spanning snel aan- en uitgeschakeld. De verhouding tussen de tijd dat het signaal aan staat en de totale periodetijd noemen we de *duty cycle* D:

$$D = \frac{t_{\text{aan}}}{T}. \tag{16}$$

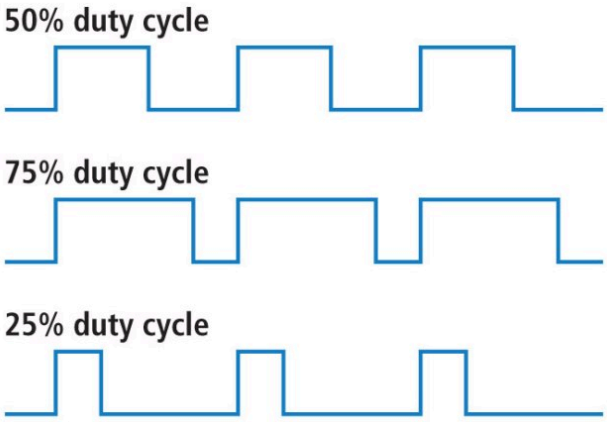
Een duty cycle van 100% betekent dat de motor continu wordt aangestuurd. Bij 50% staat de spanning gedurende de helft van iedere periode aan en bij 0% staat de motor volledig uit.

Voor een ideale PWM-schakeling is de gemiddelde spanning over één periode

$$U_{\text{gem}} = D\,U_{\text{voeding}}. \tag{17}$$

De motor reageert door zijn mechanische en elektrische traagheid niet afzonderlijk op iedere snelle puls. Daardoor kan met de duty cycle het gedrag van de motor worden geregeld.

In onze opstelling genereert pin D9 van de Arduino het PWM-signaal. Dit signaal wordt aangesloten op de **ENA**-ingang van de L298N. De Arduino bepaalt dus wanneer de H-brug de motor inschakelt, terwijl het daadwerkelijke motorvermogen uit de externe voeding komt.



Figuur 10: Voorbeeld van PWM-signalen met verschillende duty cycles. De amplitude blijft gelijk, maar de verhouding tussen de aan- en uittijd verandert.

AS5600-hoeksensor

De hoek van het aeropendulum wordt gemeten met een **AS5600 magnetische hoeksensor**. Op de draaiende as van het aeropendulum wordt een magneet geplaatst. De sensor meet de richting van het magnetische veld en kan daaruit de hoek van de as bepalen. Omdat er geen mechanisch contact nodig is tussen de sensor en de draaiende as, treedt hierbij vrijwel geen extra mechanische weerstand op.

De AS5600 heeft een resolutie van 12 bit. Eén volledige omwenteling wordt daardoor verdeeld in

$$2^{12} = 4096 \tag{18}$$

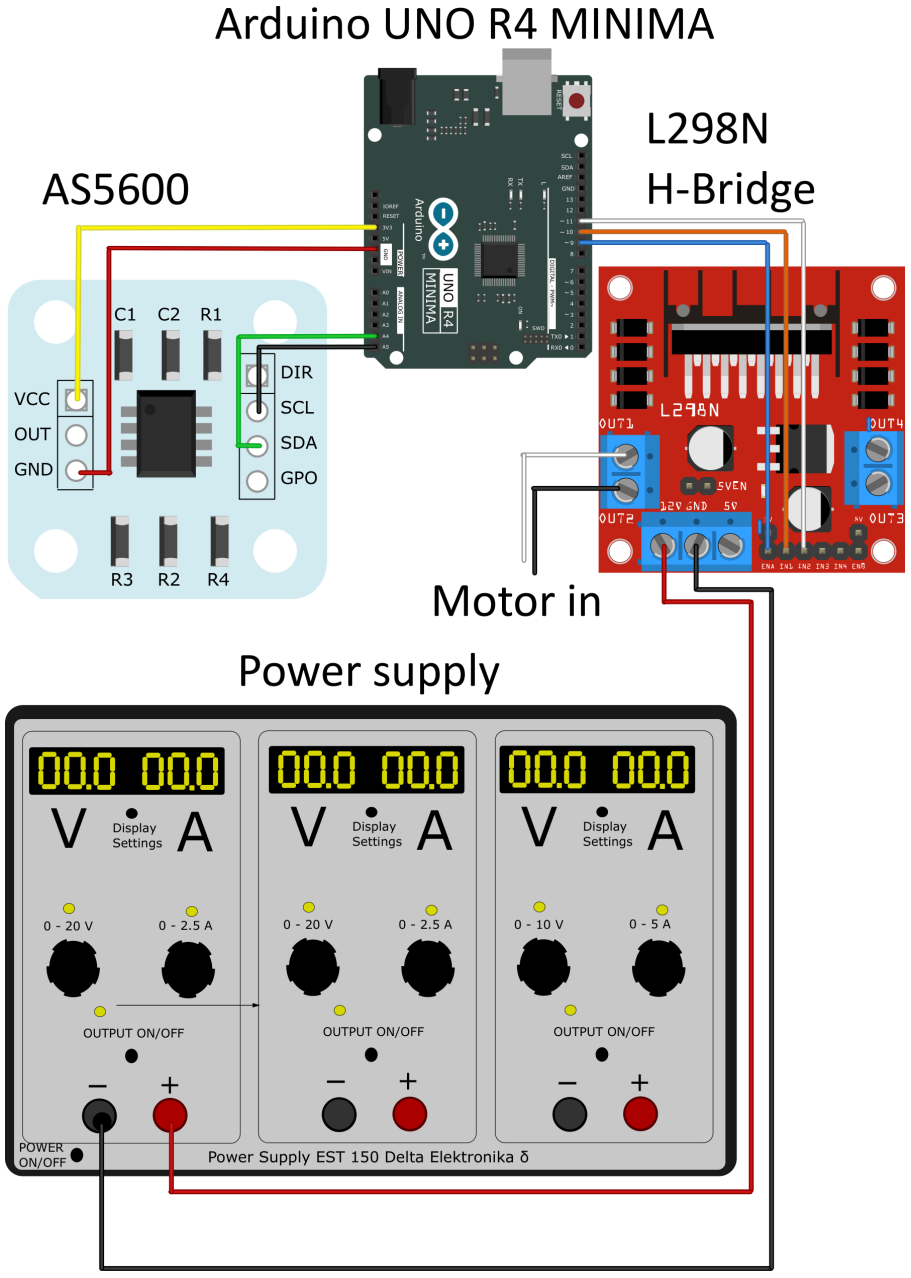
mogelijke waarden. Een ruwe sensorwaarde kan worden omgerekend naar een hoek volgens

$$\theta = \frac{\text{sensorwaarde}}{4096}\,360^\circ. \tag{19}$$

De sensor wordt digitaal uitgelezen via het **I²C-protocol**. Hiervoor zijn twee verbindingen nodig: SDA (*Serial Data*) voor het versturen van gegevens en SCL (*Serial Clock*) voor het kloksignaal. In deze opstelling is SDA aangesloten op A4 en SCL op A5 van de Arduino.

De positie van de magneet ten opzichte van de sensor is belangrijk. Wanneer de magneet te ver van de sensor staat of niet goed boven het midden van de sensor is geplaatst, kan de hoekmeting

onbetrouwbaar worden. Controleer daarom voor iedere meting of de sensor correct functioneert. Wanneer de arm langzaam met de hand wordt bewogen, moet de gemeten hoek vloeiend veranderen. Een constante waarde, grote sprongen of een foutmelding kan betekenen dat de magneet niet correct ten opzichte van de sensor staat.



Figuur 11: Aansluitschema van de Arduino Uno R4 Minima, L298N H-brug, AS5600-hoeksensor, motor en externe voeding.

Voorbereiding

1 Bekijk de geometrie van het aeropendulum. Onder welke voorwaarde geldt $l = l_T$? Leg in woorden uit wat deze aanname fysisch betekent.

2 In de stationaire toestand staat de arm stil en zijn de momenten rond het draaipunt in evenwicht. Laat zien dat dan geldt:

$$l_T F_T = mgl \sin\theta. \tag{20}$$

3 De motor zet niet al het opgenomen elektrische vermogen om in mechanisch vermogen. Neem een rendement van $\eta = 0.70$ aan:

$$P_{\text{mech}} = \eta U I. \tag{21}$$

Bereken bij een voedingsspanning van $U = 4$ V het mechanisch vermogen voor een aantal verschillende motorstromen.

4 Neem aan dat de stuwkracht van de propeller evenredig is met het mechanisch vermogen:

$$F_T = k_P P_{\text{mech}}, \tag{22}$$

waarbij k_P een constante is die de eigenschappen van de motor en propeller beschrijft. Combineer de bovenstaande vergelijkingen en leid een uitdrukking af voor de stationaire hoek θ als functie van de motorstroom I.

5 Kies realistische waarden voor m, l, l_T en k_P. Licht kort toe waarom je deze waarden hebt gekozen. Maak vervolgens in Desmos een grafiek van θ als functie van I.

6 Voorspel op basis van je grafiek wat er gebeurt wanneer de motorstroom wordt verhoogd. Verwacht je dat het eenvoudige model voor alle motorstromen even goed werkt? Leg kort uit.

Meten vanuit de Arduino IDE

Gebruik de CSV-sketch voor dit practicum in `Arduino/Openloop` of `Arduino/PID`. Open voor open-loopmetingen `Aero_pendulum_OL_CSV.ino` en voor PID-metingen `Aero_pendulum_PID_CSV.ino`, ieder in zijn eigen gelijknamige map. Het programma draait volledig op de Arduino. Python wordt pas achteraf gebruikt voor de analyse.

1. Selecteer in de Arduino IDE de Arduino Uno R4 Minima en de juiste poort. Installeer `AS5600` van Rob Tillaart. Alleen de PID-sketch heeft daarnaast de bibliotheek `PID` van Brett Beauregard nodig.
2. Controleer met de motorvoeding uit de bedrading en hoekconversie. Stel met de begeleider `MAX_PWM`, `MIN_ANGLE_DEG` en `MAX_ANGLE_DEG` vast. De meegeleverde waarden zijn voorlopig.
3. Wijzig voor open-loop `OPEN_LOOP_PWM`; wijzig voor PID `setpoint`, `kp`, `ki` en `kd`. De proefduur staat in `MEASUREMENT_MS` en is standaard 30000 ms. Upload met uitgeschakelde motorvoeding.
4. Open Serial Monitor en kies 115200 baud. Schakel de weergave van extra tijdstempels uit: de sketch levert zelf de tijd. Sluit Serial Plotter en andere programma's op dezelfde poort.
5. Verstuur de hoofdletter `C` voor tien seconden sensorcontrole zonder motoraansturing. Controleer de hoek met alleen USB aangesloten. De uitvoer stopt daarna vanzelf. Een ongeldige sensorstatus of werkhoek levert een foutregel en beëindigt de opname.
6. Maak Serial Monitor leeg vóór de echte meting. Schakel de gecontroleerde motorvoeding in terwijl de sketch wacht. Houd afstand van arm en propeller en verstuur `S`. De Arduino start de motor en verstuurt een CSV-kop gevolgd door meetregels. Een eventuele regeleinde-instelling mag blijven staan; CR en LF worden genegeerd.
7. Na de ingestelde proefduur stopt de Arduino de motor en de uitvoer automatisch. Verstuur `X` om eerder te stoppen. Bij een sensorfout of een hoek buiten het ingestelde bereik stopt de motor eveneens. Schakel na afloop de motorvoeding uit.

Het sluiten van Serial Monitor is geen stopcommando. Gebruik `X` of schakel de motorvoeding uit. Houd de fysieke uitschakeling bereikbaar: de softwarematige stop werkt alleen zolang de microcontroller zijn programma blijft uitvoeren.

CSV-bestand bewaren

De Arduino verstuurt CSV-tekst via USB; zonder opslagmodule schrijft hij geen bestand op de computer. Bewaar de meetgegevens als volgt:

1. Wacht totdat de uitvoer is gestopt. Selecteer en kopieer de volledige opname uit Serial Monitor, vanaf de regel die begint met `t_ms,setpoint_deg` tot en met de laatste volledige meetregel.
2. Plak deze tekst in een leeg tekstbestand, bijvoorbeeld in Kladblok. Sla op als `pid_run1.csv` of `ol_pwm60_run1.csv`. Kies bij opslaan zo nodig "Alle bestanden", zodat de naam niet eindigt op `.csv.txt`.
3. Gebruik een nieuwe bestandsnaam per proef. Neem geen meldingen van de IDE, geweigerde startmeldingen of een tweede CSV-kop op in hetzelfde bestand. Controleer bij langere proeven of het begin van de uitvoer nog aanwezig is; verkort de proef als de monitor oude regels niet bewaart.

De belangrijkste kolommen zijn `t_ms` (tijd sinds het startcommando), `angle_deg` (hoek), `pwm` (werkelijke PWM-telwaarde), `setpoint_deg` (leeg bij OL), `running` en `sensor_ok`. Noteer de gekozen PWM- of PID-instellingen en de maximale PWM zelf in het labjournaal. Een ongeldige hoek blijft leeg. Het scheidingsteken is een komma en het decimaalteken een punt. Noteer voedingsinstellingen en kalibratie apart in het labjournaal.

Meten, regelen en CSV-uitvoer gebeuren in dezelfde ronde, nominaal iedere 20 ms (50 Hz). Gebruik de tijdstempels voor de tijdas, met $t = 0$ bij het startcommando. De eerste meetregel volgt enkele milliseconden later. Controleer de werkelijke intervallen door opeenvolgende tijdstempels van elkaar af te trekken. Er is geen nulmeting vóór de motoraansturing; bepaal de beginhoek vooraf met `C`. Verdere instructies staan in `Arduino/README.md`.

Methode

- Gebruik de nieuwe sketch `Aero_pendulum_OL_CSV.ino` in de gelijknamige map onder `Arduino/Openloop`.
- Houd de voedingsspanning en stroomlimiet gelijk tijdens de metingen. Varieer `OPEN_LOOP_PWM` in vooraf afgesproken stappen binnen `MAX_PWM`.
- Schakel de motorvoeding uit voordat je een gewijzigde instelling uploadt. Voer daarna de start- en opslagprocedure hierboven uit.
- Wacht per instelling tot de arm een stationaire positie bereikt, of noteer dat dit binnen de meetduur niet gebeurt.
- Herhaal iedere instelling minimaal drie keer. Registreer per proef een apart CSV-bestand en noteer voedingsspanning en motorstroom voor de vergelijking met het fysische model.

Uitvoering

1. Maak een meettabel met de ingestelde PWM-telwaarde en de gemeten stationaire hoek.
2. Meet de stationaire hoek voor alle ingestelde PWM-waarden.
3. Herhaal iedere meting minimaal drie keer.
4. Noteer eventuele bijzonderheden, zoals een dode zone, oscillaties of verzadiging van de motor.

Verwerking

1. Laad de meetgegevens in met Python.
2. Controleer de CSV-kop: het scheidingsteken is een komma en het decimaalteken een punt. Analyseer alleen regels met `running=1` en `sensor_ok=1`.
3. Print de kolomnamen met `df.keys()`.
4. Bereken voor iedere PWM-instelling de gemiddelde stationaire hoek en de standaarddeviatie.
5. Maak een grafiek van de stationaire hoek als functie van de ingestelde PWM-telwaarde, inclusief foutbalken.
6. Vergelijk de gemeten relatie met de theoretische voorspelling uit de voorbereiding en bespreek eventuele afwijkingen.

Importeer data

```
import pandas as pd
df = pd.read_csv('ol_pwm60_run1.csv')
df = df[(df['running'] == 1) & (df['sensor_ok'] == 1)].copy()
if df.empty:
    raise ValueError('Geen geldige actieve meetpunten')
df['t_s'] = df['t_ms'] / 1000

print(df.keys())
```

Verwerking

1. Plot de stationaire hoek als functie van de ingestelde PWM-telwaarde. Voeg foutbalken toe indien meerdere metingen zijn uitgevoerd.
2. Vergelijk de theoretische hoek als functie van stroom met de apart gemeten motorstroom en stationaire hoek. Een PWM-telwaarde is geen gemeten spanning of stroom; zet deze niet rechtstreeks op de stroomas van het model.
3. Bepaal voor welke uitgang de propeller de arm voor het eerst in beweging brengt (de dode zone).

4 Onderzoek of de relatie tussen de Arduino-uitgang en de stationaire hoek lineair is. Licht je antwoord toe.
5 Bespreek mogelijke oorzaken van afwijkingen tussen het theoretische model en de metingen, zoals wrijving, niet-lineair motorgedrag, luchtweerstand en meetonzekerheid.

Afronding

1 Laat de meetresultaten controleren door de practicumbegeleider.
2 Voeg alle grafieken toe aan je digitale labjournaal en geef iedere figuur een duidelijk onderschrift.
3 Vul de sectie **Resultaten en Discussie** aan. Beschrijf hierin de relatie tussen de Arduino-uitgang en de stationaire hoek en vergelijk de experimentele resultaten met het theoretische model.
4 Reflecteer op de beperkingen van een open-loop systeem en beschrijf waarom terugkoppeling in veel technische toepassingen gewenst is.

Practicum: PID-regeling van een aeropendulum

Leerdoelen

- ☐ Je kunt uitleggen hoe terugkoppeling de hoek van een aeropendulum regelt.
- ☐ Je kunt de invloed van de P-, I- en D-term experimenteel onderzoeken.
- ☐ Je kunt een regelaar stapsgewijs afstellen en beoordelen op insteltijd, overshoot en stationaire fout.
- ☐ Je kunt meetgegevens uit de Arduino verwerken in Python en de beperkingen van de regeling herkennen.

Veiligheid

1. Gebruik dezelfde gecontroleerde opstelling en voedingsinstellingen als bij het open-loop-practicum. Laat het toegestane hoekbereik en de maximale motoruitgang vaststellen door de practicumbegeleider.
2. Schakel de motorvoeding uit voordat je bedrading, sensor of programma aanpast. De nieuwe CSV-sketch wacht na uploaden of resetten op een expliciet startcommando via Serial Monitor.
3. Houd handen, haar en losse voorwerpen buiten het bewegingsbereik van arm en propeller. Pak de arm niet vast terwijl de motorvoeding is ingeschakeld.
4. Houd de uitschakeling van de motorvoeding bereikbaar. Schakel direct uit bij toenemende oscillaties, een onjuiste hoekmeting of beweging richting een mechanische aanslag. Wacht tot de propeller stilstaat voordat je de opstelling benadert.

Introductie

In het open-looppracticum heb je onderzocht welke hoek ontstaat bij een ingestelde motoruitgang. Die hoek kan veranderen door bijvoorbeeld wrijving of een verandering van de belasting. De besturing merkt zo'n afwijking niet op.

In dit practicum koppelen we de gemeten hoek terug naar de Arduino. De Arduino vergelijkt de hoek met de gewenste waarde en past het PWM-signaal aan. Zo ontstaat een gesloten regelkring. Je onderzoekt eerst een proportionele regelaar, voegt vervolgens een integrerende term toe en onderzoekt ten slotte of een differentiërende term de respons verbetert.

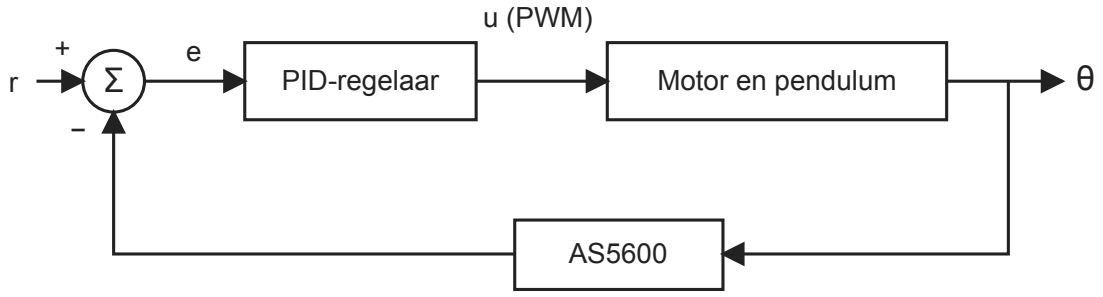
Theorie

Terugkoppeling en regelfout

De gewenste hoek heet het *setpoint*, $r(t)$. De gemeten hoek is $\theta(t)$. De regelfout is

$$e(t) = r(t) - \theta(t). \tag{23}$$

Bij een positieve fout staat de arm onder de gewenste hoek. Voor de gebruikte instelling `DIRECT` moet een grotere motoruitgang leiden tot een grotere gemeten hoek. Controle van dit teken is noodzakelijk voordat je de regeling inschakelt.



Figuur 12: Gesloten regelkring van het aeropendulum. De sensor levert de gemeten hoek voor de terugkoppeling.

De drie bijdragen

Een ideale PID-regelaar kan worden geschreven als

$$u^*(t) = K_p e(t) + K_i \int_0^t e(\tau)\,\mathrm{d}\tau + K_d \frac{\mathrm{d}e}{\mathrm{d}t}. \tag{24}$$

Hier is u^* de berekende uitgang vóór begrenzing. In deze opstelling drukken we de uitgang uit in PWM-telwaarden en de hoek in graden. De eenheden van K_p, K_i en K_d zijn daarom respectievelijk PWM-telwaarden per graad, per graad per seconde en maal seconde per graad.

- **P: reactie op de huidige fout.** Een grotere K_p geeft bij dezelfde fout meer motoruitgang. Dit kan de respons versnellen, maar ook overshoot of oscillaties veroorzaken. Omdat een niet-nul motoruitgang nodig is om de arm omhoog te houden, kan een P-regelaar zonder extra basisuitgang een blijvende fout hebben.
- **I: reactie op de opgebouwde fout.** Een aanhoudende fout bouwt een extra bijdrage op. Hiermee kan de stationaire fout afnemen. Te veel integrerende werking kan leiden tot overshoot en langzaam herstel.
- **D: reactie op snelle verandering.** Deze bijdrage kan de beweging afremmen en overshoot verminderen, maar is gevoelig voor meetruis.

De gebruikte bibliotheek `PID_v1` berekent de D-bijdrage uit de verandering van de *gemeten hoek*: $-K_d\,\mathrm{d}\theta/\mathrm{d}t$. Bij een constant setpoint komt dit overeen met de ideale formule. Bij een setpointsprong wordt zo een directe D-piek door die sprong vermeden. De bibliotheek begrenst ook de opgebouwde integratorbijdrage; langdurige verzadiging kan desondanks de respons beïnvloeden. Zie de broncode van PID_v1 voor de implementatie.

Begrenzing en bemonstering

Voor een maximale PWM-uitgang $u_{\max}$ geldt

$$u(t) = \min\big(\ u_{\max}, \max(0, u^{*}(t))\ \big), \qquad D \approx \frac{u}{255}. \tag{25}$$

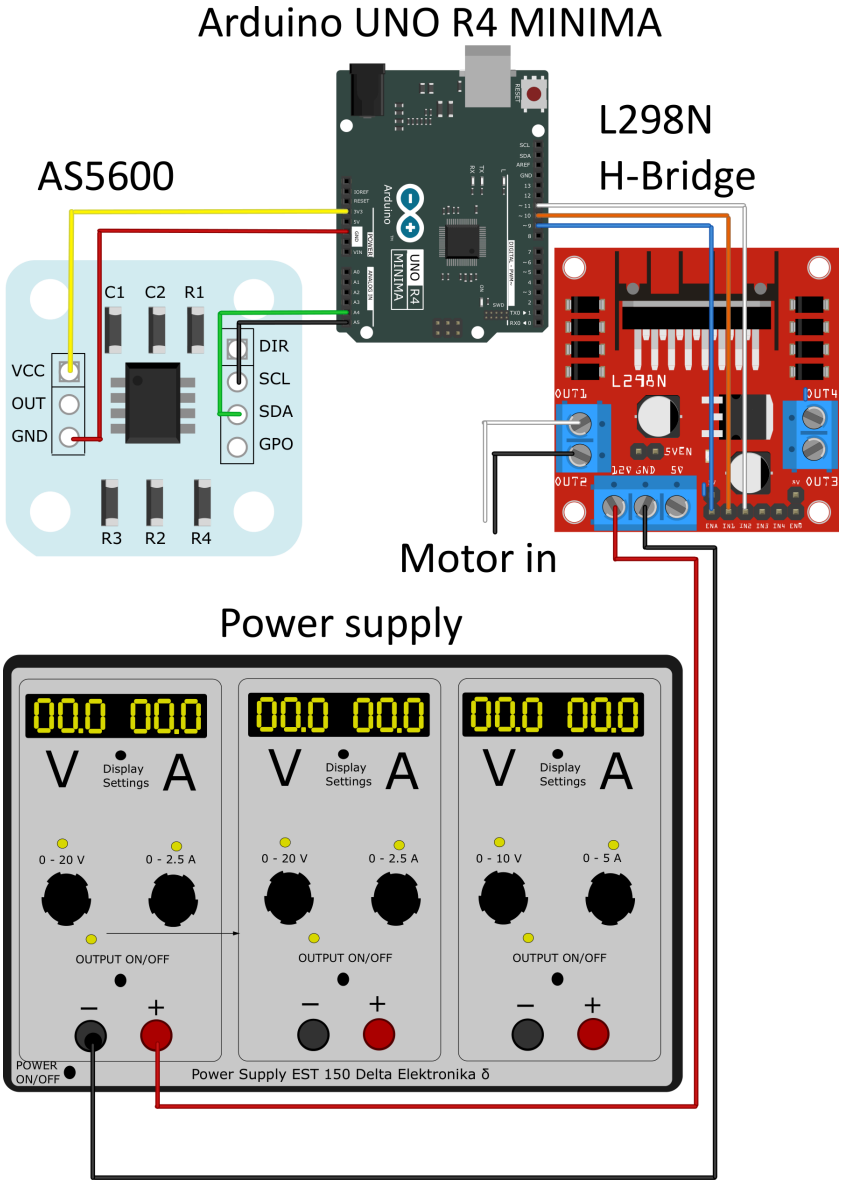
De motoruitgang is een PWM-instelling, geen gemeten spanning of stuwkracht. De regelaar kan in deze sketch geen negatieve stuwkracht vragen. Om de arm te laten zakken, vermindert hij de motoruitgang en werkt de zwaartekracht mee. Daarom hoeven opgaande en neergaande responsen niet gelijk te zijn.

De ingestelde regelperiode is 20 ms, overeenkomend met een nominale frequentie van 50 Hz. Serieel versturen kost echter ook tijd. De nieuwe CSV-sketch verstuurt iedere 20 ms tekst op 115200 baud, in dezelfde ronde als de hoekmeting en regeling. Neem daarom niet zonder controle aan dat opeenvolgende meetpunten precies 20 ms uit elkaar liggen.

Opstelling en Arduino-programma

Gebruik dezelfde Arduino Uno R4 Minima, L298N, AS5600, motor, voeding en mechanische opstelling als bij het open-looppracticum. Houd de voedingsinstellingen tijdens de vergelijking van regelaars gelijk.

Arduino-aansluiting	Component
D9	ENA van de L298N (PWM)
D10 en D11	IN1 en IN2 van de L298N
A4 en A5	SDA en SCL van de AS5600
GND	Gemeenschappelijke massa volgens het aansluitschema



Figuur 13: De aansluitingen blijven gelijk aan die van het open-looppracticum.

Open `Aero_pendulum_PID_CSV.ino` in de gelijknamige submap van `Arduino/PID`. Gebruik de bibliotheken die `Wire.h`, `PID_v1.h` en `AS5600.h` leveren. Bewaar een eigen kopie in een sketchmap met dezelfde naam als het hoofd-`.ino`-bestand. Zet daarin alleen de sketch die je wilt uploaden; meerdere complete sketches met elk een `setup()` en `loop()` horen niet in dezelfde sketchmap.

Instelling	Beginwaarde in de code	Betekenis
`setpoint`	40	Gewenste hoek in graden
`kp`	2	Proportionele versterking
`ki`	0.5	Integrerende versterking
`kd`	0	Differentiërende versterking
`SetSampleTime(20)`	20	Ingestelde regelperiode in ms
`MAX_PWM`	120 (voorlopig)	Grenzen van de motoruitgang

De begininstelling is dus een **PI-regelaar**. Deze waarden zijn geen garantie voor een veilige of goede afstelling van iedere opstelling. Je geeft in Serial Monitor de commando's `S` (start) en `X` (stop). Wijzig het setpoint en de versterkingen in de code en upload opnieuw met uitgeschakelde motorvoeding. Bij iedere start wordt de integrator gewist; tijdens wachten staat de regelaar uit.

De hoek wordt berekend als de ingestelde `ANGLE_OFFSET_DEG` minus de sensorhoek in graden. Dit veronderstelt een bepaalde montage en nulstand van de sensor. De ruwe sensorhoek loopt rond bij een volledige omwenteling. Controleer of de werkhoek correct is en binnen het gekozen bereik geen sprong vertoont.

Voorbereiding

1. Teken de regelkring en benoem setpoint, meetwaarde, regelfout, regelaar, actuator en sensor.
2. Bereken voor $r = 40^\circ$, $\theta = 30^\circ$ en $K_p = 2$ de P-bijdrage. Welke PWM-duty cycle hoort daarbij als de andere bijdragen nul zijn?
3. Gebruik je open-loopmetingen om een haalbare doelhoek en de benodigde stationaire motoruitgang te kiezen. Waarom kan een P-regelaar zonder basisuitgang daarvoor een niet-nul fout nodig hebben?
4. Voorspel wat er verandert wanneer je achtereenvolgens K_p, K_i en K_d verhoogt. Beschrijf ook een mogelijk nadeel per term.
5. Zoek in de sketch de hoekconversie, de uitgangsbegrenzing en het moment waarop de motor wordt aangestuurd. Wat gebeurt er na een reset?
6. Maak een meetplan voor een eerlijke vergelijking van P, PI en PID. Leg beginhoek, doelhoek, meetduur, herhalingen en criteria voor een goede regeling vooraf vast.

Uitvoering

Opdracht 1 — Controle en eerste instelling

1. Controleer met uitgeschakelde motorvoeding de bedrading en bewegingsvrijheid. Lees via USB de sensor uit en beweeg de stilstaande arm voorzichtig. Controleer nulstand, teken en continuïteit van de hoekmeting.
2. Laat een eventuele correctie van de hoekconversie controleren. Een grotere motoruitgang moet binnen het werkgebied een grotere gemeten hoek geven, passend bij `DIRECT`.
3. Kies met de begeleider een haalbaar setpoint uit de open-loopmetingen en een toegestane bovengrens voor PWM. Pas hiervoor `MAX_PWM` aan.
4. Begin met een P-regelaar: zet `ki = 0` en `kd = 0`. Kies een kleine positieve `kp` in overleg met de begeleider. Gebruik niet automatisch de volledige oorspronkelijke PI-instelling.
5. Leg de instellingen en voedingswaarden vast. Upload met uitgeschakelde motorvoeding. Houd bij het inschakelen afstand en controleer of de regeling de afwijking in de juiste richting corrigeert.

Meten vanuit de Arduino IDE

Gebruik de CSV-sketch voor dit practicum in `Arduino/Openloop` of `Arduino/PID`. Open voor open-loopmetingen `Aero_pendulum_OL_CSV.ino` en voor PID-metingen `Aero_pendulum_PID_CSV.ino`, ieder in zijn eigen gelijknamige map. Het programma draait volledig op de Arduino. Python wordt pas achteraf gebruikt voor de analyse.

1. Selecteer in de Arduino IDE de Arduino Uno R4 Minima en de juiste poort. Installeer `AS5600` van Rob Tillaart. Alleen de PID-sketch heeft daarnaast de bibliotheek `PID` van Brett Beauregard nodig.
2. Controleer met de motorvoeding uit de bedrading en hoekconversie. Stel met de begeleider `MAX_PWM`, `MIN_ANGLE_DEG` en `MAX_ANGLE_DEG` vast. De meegeleverde waarden zijn voorlopig.
3. Wijzig voor open-loop `OPEN_LOOP_PWM`; wijzig voor PID `setpoint`, `kp`, `ki` en `kd`. De proefduur staat in `MEASUREMENT_MS` en is standaard 30000 ms. Upload met uitgeschakelde motorvoeding.
4. Open Serial Monitor en kies 115200 baud. Schakel de weergave van extra tijdstempels uit: de sketch levert zelf de tijd. Sluit Serial Plotter en andere programma's op dezelfde poort.
5. Verstuur de hoofdletter `C` voor tien seconden sensorcontrole zonder motoraansturing. Controleer de hoek met alleen USB aangesloten. De uitvoer stopt daarna vanzelf. Een ongeldige sensorstatus of werkhoek levert een foutregel en beëindigt de opname.
6. Maak Serial Monitor leeg vóór de echte meting. Schakel de gecontroleerde motorvoeding in terwijl de sketch wacht. Houd afstand van arm en propeller en verstuur `S`. De Arduino start de motor en verstuurt een CSV-kop gevolgd door meetregels. Een eventuele regeleinde-instelling mag blijven staan; CR en LF worden genegeerd.
7. Na de ingestelde proefduur stopt de Arduino de motor en de uitvoer automatisch. Verstuur `X` om eerder te stoppen. Bij een sensorfout of een hoek buiten het ingestelde bereik stopt de motor eveneens. Schakel na afloop de motorvoeding uit.

Het sluiten van Serial Monitor is geen stopcommando. Gebruik `X` of schakel de motorvoeding uit. Houd de fysieke uitschakeling bereikbaar: de softwarematige stop werkt alleen zolang de microcontroller zijn programma blijft uitvoeren.

CSV-bestand bewaren

De Arduino verstuurt CSV-tekst via USB; zonder opslagmodule schrijft hij geen bestand op de computer. Bewaar de meetgegevens als volgt:

1. Wacht totdat de uitvoer is gestopt. Selecteer en kopieer de volledige opname uit Serial Monitor, vanaf de regel die begint met `t_ms,setpoint_deg` tot en met de laatste volledige meetregel.

2. Plak deze tekst in een leeg tekstbestand, bijvoorbeeld in Kladblok. Sla op als `pid_run1.csv` of `ol_pwm60_run1.csv`. Kies bij opslaan zo nodig “Alle bestanden”, zodat de naam niet eindigt op `.csv.txt`.
3. Gebruik een nieuwe bestandsnaam per proef. Neem geen meldingen van de IDE, geweigerde startmeldingen of een tweede CSV-kop op in hetzelfde bestand. Controleer bij langere proeven of het begin van de uitvoer nog aanwezig is; verkort de proef als de monitor oude regels niet bewaart.

De belangrijkste kolommen zijn `t_ms` (tijd sinds het startcommando), `angle_deg` (hoek), `pwm` (werkelijke PWM-telwaarde), `setpoint_deg` (leeg bij OL), `running` en `sensor_ok`. Noteer de gekozen PWM- of PID-instellingen en de maximale PWM zelf in het labjournaal. Een ongeldige hoek blijft leeg. Het scheidingsteken is een komma en het decimaalteken een punt. Noteer voedingsinstellingen en kalibratie apart in het labjournaal.

Meten, regelen en CSV-uitvoer gebeuren in dezelfde ronde, nominaal iedere 20 ms (50 Hz). Gebruik de tijdstempels voor de tijdas, met $t = 0$ bij het startcommando. De eerste meetregel volgt enkele milliseconden later. Controleer de werkelijke intervallen door opeenvolgende tijdstempels van elkaar af te trekken. Er is geen nulmeting vóór de motoraansturing; bepaal de beginhoek vooraf met `C`. Verdere instructies staan in `Arduino/README.md`.

Opdracht 2 — P-regeling

1. Onderzoek minimaal drie stapsgewijs gekozen waarden van K_p, met $K_i = K_d = 0$. Verhoog niet verder bij toenemende oscillaties of overschrijding van het afgesproken bereik.
2. Gebruik per meting dezelfde ruststand en hetzelfde setpoint. Schakel de motorvoeding uit, wacht tot de arm stilstaat en upload of reset. Schakel de gecontroleerde motorvoeding in terwijl de sketch wacht. Verstuur `S` in Serial Monitor. De Arduino-tijd in de CSV begint bij dat startcommando.
3. Meet tot een stationaire toestand zichtbaar is, of tot de vooraf afgesproken maximale meetduur. Registreer ook wanneer de arm het setpoint niet bereikt.
4. Herhaal iedere instelling minimaal drie keer. Bewaar per proef de CSV en noteer K_p, K_i, K_d, setpoint, PWM-grens en voedingsinstellingen in je labjournaal.
5. Vergelijk snelheid, oscillaties en blijvende fout. Kies een bruikbare P-instelling als uitgangspunt voor de volgende opdracht.

Opdracht 3 — PI-regeling

1. Houd de gekozen K_p gelijk en laat $K_d = 0$. Voeg een kleine positieve K_i toe. Onderzoek minimaal drie waarden en herhaal het meetprotocol van opdracht 2.
2. Start voor iedere proef een nieuwe registratie. Het startcommando wist de integrator en activeert de regeling direct. Daardoor neemt de integrator de geschiedenis van de vorige proef niet mee.
3. Onderzoek of de stationaire fout kleiner wordt en welke invloed K_i heeft op overshoot en insteltijd. Bekijk daarbij ook of de PWM-uitgang tegen een grens blijft staan.
4. Selecteer een PI-instelling en onderbouw de keuze met meetresultaten. Gebruik geen onhaalbaar setpoint om de integrator bewust te laten oplopen.

Opdracht 4 — PID-regeling

1. Houd de gekozen K_p en K_i gelijk. Vergelijk $K_d = 0$ met minimaal twee kleine positieve waarden van K_d en herhaal iedere meting drie keer.
2. Bekijk zowel de hoek als de PWM-uitgang. Minder overshoot kan samengaan met een onrustiger stuursignaal. Verlaag K_d wanneer meetruis de motoruitgang sterk laat fluctueren.

3 Kies op basis van de resultaten een eindinstelling. Als PI beter presteert dan PID, is $K_d = 0$ een geldige uitkomst; motiveer dit met je metingen.
4 Test deze instelling bij een tweede haalbare doelhoek met dezelfde begincondities. Onderzoek of de afstelling ook daar bruikbaar is.

Verdieping — Setpointsprong tijdens de regeling

Pas in een eigen kopie het setpoint na een vast tijdstip eenmalig aan naar een tweede goedgekeurde hoek. Dit vraagt een codewijziging; het kan niet via een commando in de oorspronkelijke sketch. Registreer het veranderende setpoint mee. Laat de wijziging vooraf controleren en test een opgaande en neergaande sprong. Raak de bewegende arm niet aan om een verstoring te maken. Vergelijk de responsen en verklaar de invloed van de eenzijdige aandrijving.

Verwerking in Python

Het onderstaande voorbeeld leest de uit Serial Monitor opgeslagen CSV in. Gebruik een apart bestand per proef en neem de door de Arduino verstuurde kolomkop mee.

Meetbestand inlezen en controleren

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('pid_meting.csv')
df['t_s'] = df['t_ms'] / 1000
dt = df['t_ms'].diff().dropna()
print('Meetinterval [ms]:', dt.describe())
if (dt <= 0).any():
    raise ValueError('Controleer op reset of onjuiste tijdstempels.')

fig, ax = plt.subplots(2, 1, sharex=True, figsize=(8, 6))
ax[0].plot(df['t_s'], df['angle_deg'], label='Gemeten hoek')
ax[0].plot(df['t_s'], df['setpoint_deg'], '--', label='Setpoint')
ax[0].set_ylabel('Hoek [graden]')
ax[0].legend()
ax[1].plot(df['t_s'], df['pwm'])
ax[1].set_ylabel('PWM [telwaarden]')
ax[1].set_xlabel('Tijd sinds startcommando [s]')
for a in ax:
    a.grid()
fig.tight_layout()
plt.show()
```

Gebruik de geregistreerde tijdstempels voor de tijdas. Het meet- en regelinterval hoort ongeveer 20 ms te zijn. De bibliotheek schaalt de I- en D-term met die ingestelde periode; afwijkingen daarvan zijn relevant voor de regeling. Vergelijk daarom ook de spreiding van de tijdsintervallen.

Beoordelingscriteria

Kies voor alle proeven dezelfde analysemethode. Neem $t_0 = 0$ bij het startcommando; bij de verdiepingsopdracht gebruik je het tijdstip van de setpointsprong. Een opstartrespons vanaf rust is niet hetzelfde experiment als een sprong vanuit een geregeld evenwicht.

- **Stationaire fout:** $e_{\mathrm{ss}} = r - \bar{\theta}$, met de gemiddelde hoek over een aantoonbaar stabiel eindinterval, bijvoorbeeld de laatste vijf seconden. Geef ook de standaarddeviatie en de lengte van dit interval. Bij blijvende drift of oscillatie rapporteer je dat geen stationaire toestand is bereikt.
- **Overshoot bij een opgaande respons:** $O = \max(0, \theta_{\max} - r)$ in graden. Eventueel normaliseer je als $100\%\, O / \mid r - \theta_0 \mid$, waarbij θ_0 de beginhoek is en $r \neq \theta_0$.
- **Insteltijd:** tijd vanaf t_0 totdat de hoek binnen een vooraf gekozen band rond het setpoint blijft, bijvoorbeeld $r \pm 2^\circ$, tot het einde van de registratie. Vereis daarna nog minimaal vijf seconden meettijd. Als dit niet gebeurt, rapporteer je “niet bereikt binnen de meetduur”.
- **Verzadiging:** geef aan of en hoelang de uitgang nabij nul of de ingestelde bovengrens blijft. Gebruik voor de tijdsduur de tijdstempels en noteer de gebruikte tolerantie.

Opdracht 5 — Vergelijken en verklaren

1. Controleer meetinterval, ontbrekende waarden en eventuele resets. Leg uit welke gegevens bruikbaar zijn en waarom.
2. Maak voor de gekozen P-, PI- en PID-instelling grafieken van hoek, setpoint en PWM. Gebruik dezelfde asschalen en lijn de proeven uit op t_0.

3 Maak een tabel met per instelling K_p, K_i, K_d, stationaire fout, overshoot en insteltijd. Rapporteer de spreiding over de drie herhalingen.
4 Verklaar de verschillen tussen P, PI en PID aan de hand van de drie regelbijdragen. Betrek de PWM-grafieken bij je verklaring.
5 Vergelijk de doelhoek en benodigde stationaire PWM met je open-loopmetingen. Bespreek waarom terugkoppeling kan corrigeren voor afwijkingen, maar geen onhaalbare hoek bereikbaar maakt.
6 Formuleer een conclusie: welke instelling voldoet het best aan jouw vooraf gekozen criteria? Bespreek de invloed van meetruis, timing, wrijving en uitgangsbegrenzing.

Afronding

1 Schakel de motorvoeding uit en wacht tot de propeller stilstaat.
2 Laat de gekozen afstelling en meetresultaten controleren door de practicumbegeleider.
3 Bewaar de gebruikte sketch, ruwe meetbestanden, instellingen en Python-analyse bij elkaar in je practicum-map.
4 Werk je labjournaal bij met de meetmethode, grafieken, vergelijkingstabel en onderbouwde conclusie.

Practicum: efficiëntie en vermogen van zonnecellen

Leerdoelen

- ☐ De student kan, aan de hand van een zelfstandig opgesteld meetplan, een experiment uitvoeren met een fotovoltaïsche (PV) cel, waarbij de student de opstelling aansluit en de metingen uitvoert.
- ☐ De student kan de stroom-voltage karakteristiek (I-V curve) van zonnecellen uitleggen en bepalen.
- ☐ De student kan het vermogen en de efficiëntie (FF) van een zonnecel bepalen.
- ☐ De student kan zelfstandig een experimentele verdieping opstellen en uitvoeren.

Veiligheid

1 In dit practicum werkt je elektriciteit, ga zorgvuldig om met de apparatuur en schakel bij twijfel de practicumbegeleider in.

Introductie

Zonnecellen vormen een belangrijk toepassingsgebied binnen de technische natuurkunde, waarin halfgeleiderfysica, optica en elektrische netwerkanalyse samenkomen. Een zonnecel, of fotovoltaïsche (PV) cel, zet invallend licht direct om in elektrische energie door middel van het fotovoltaïsche effect. Hierbij komen elektronen vrij wanneer fotonen met voldoende energie het halfgeleidermateriaal raken, meestal silicium. De vrijgekomen ladingsdragers worden vervolgens gescheiden door een intern elektrisch veld, waardoor een meetbare stroom ontstaat.



Figuur 14: Een zonnecel module.

Zonnecellen spelen een steeds belangrijkere rol in de energietransitie, doordat zij zonlicht direct omzetten in elektrische energie zonder CO_2-uitstoot. Binnen de technische natuurkunde vormt het begrijpen van de werking, het rendement en het systeemgedrag van zonnecellen een essentieel onderdeel van duurzame technologieontwikkeling. De prestatie van een zonnecel hangt af van factoren zoals de bandkloof, de intensiteit van het licht en interne verliezen, en kan experimenteel worden onderzocht door spanning en stroom te meten bij verschillende belastingen.

Voorbereidingsopdracht 1

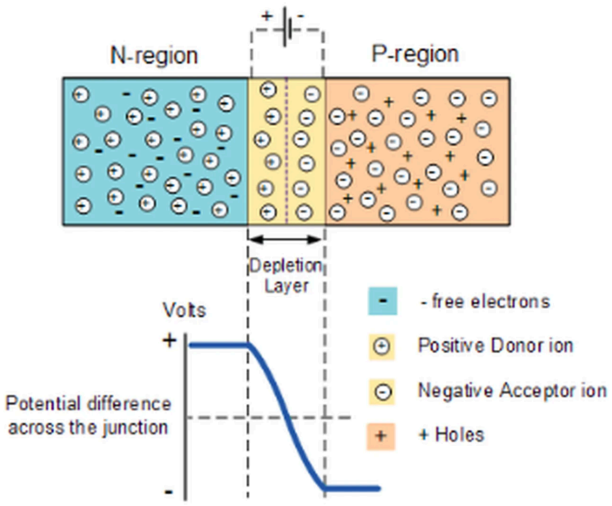
1. Bekijk de volgende voorbereidingsfilmpjes (, en .)
2. Maak een meetplan voor bepalen van het vermogen van een zonnecel.[8]. Formuleer het hoofddoel/onderzoeksvraag, geef een theoretische verwachting en maak een meetstrategie.
3. Maak een duidelijke opstelling voor het doormeten van een zonnecel.
4. Bedenk voor jezelf al wat voor verdiepend onderzoek je wilt doen bij de zonnecel.

[8] Zie Brightspace voor een korte uitleg over een meetplan.

Theorie

Een fotovoltaïsche (PV) zonnecel is in essentie een belichte pn-junction. Een pn-junction wordt gemaakt door één stuk halfgeleidermateriaal aan de ene kant p-type te doteren (met acceptoren) en aan de andere kant n-type (met donoren). In het p-gebied is de ladingsdragerdichtheid van gaten hoog, terwijl in het n-gebied de elektronen domineren. Zodra deze twee regio's met elkaar in contact staan, diffunderen elektronen vanuit het n-gebied naar het p-gebied en diffunderen gaten de andere kant op. Hierdoor blijft in het grensvlak een gedepleteerde (*depletion zone*) laag achter: een gebied waar vrijwel geen vrije ladingsdragers meer aanwezig zijn.

Deze depletion zone bevat wel vaste, geïoniseerde donoren (positief) en acceptoren (negatief), waardoor er een intern elektrisch veld ontstaat dat de diffusie stopt. Dit veld vormt de basis van de pn-juntion. In een zonnecel zorgt juist dit veld ervoor dat nieuw gegenereerde elektronen en gaten, door de absorptie van fotonen, uit elkaar worden getrokken en richting de n- en p-zijde bewegen. Wanneer er nu een extern draad tussen de n- en p-zijde wordt gelegd ontstaat er een stroom. Deze fotonen gegeneerde stroom is dus lichtafhankelijke stroom I_{ph}.



Figuur 15: Schematische weergave van een zonnecel.

Wanneer een externe belasting R wordt aangesloten, ontstaan er een spanning V en stroom I waarvan de waarde afhankelijk is van het gekozen belasting R. Door R te variëren kun je de stroomsterkte en voltage variëren en krijg je een volledige I-V-curve. Het maximale elektrische vermogen volgt uit

$$P = IV,$$

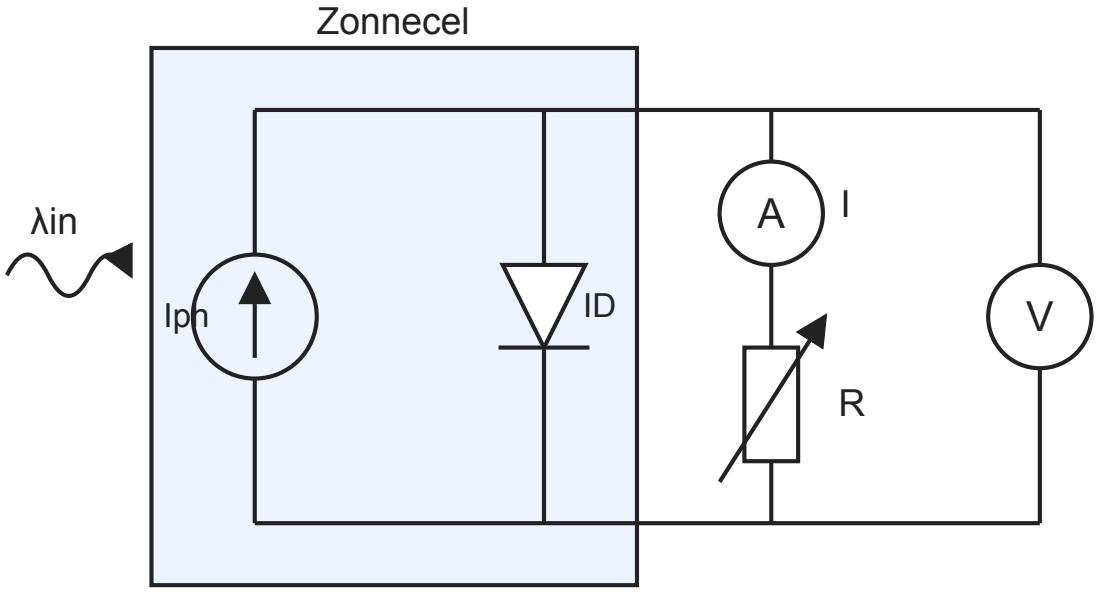
en het punt waarop dit vermogen maximaal is heet het *maximum power point* (P_{mpp}). In commerciële PV-systemen wordt dit punt automatisch gevolgd door de omvormer, die tevens de DC-uitgang omzet naar net-geschikte AC.

De interne kwaliteit van een zonnecel wordt gewoonlijk gekarakteriseerd met de *fill factor* (FF):

$$FF = \frac{P_{\mathrm{mpp}}}{V_{\mathrm{oc}} I_{\mathrm{sc}}} \times 100\%. \tag{26}$$

Hierbij zijn V_{oc} de openklemspanning en I_{sc} de kortsluitstroom. De FF geeft aan hoe rechthoekig de I-V-curve is. De totale efficiëntie vereist daarnaast kennis van het aangeboden lichtvermogen P_{lamp}:

$$\eta = \frac{P_{\mathrm{mpp}}}{P_{\mathrm{lamp}}} \times 100\%. \tag{27}$$



Figuur 16: Equivalent elektrisch model van een zonnecel met fotostroombron I_{ph}, diode I_D, uitgangsstroom I en spanning V over de variabele belasting R.

Het elektrisch gedrag van een zonnecel wordt doorgaans beschreven met het één-diode-model (Figuur 16). De fotostroom I_{ph} werkt als een spanningsonafhankelijke lichtbron, terwijl de diode het spanningsafhankelijke recombinatiepad vormt. Uit de knooppuntvergelijking volgt:

$$I = I_{\mathrm{ph}} - I_D. \tag{28}$$

De diodestroom wordt beschreven door de wet van Shockley[9]:

$$I_D = I_0\left(e^{\frac{qV}{nkT}} - 1\right), \tag{29}$$

zodat de volledige I-V-relatie van de zonnecel wordt:

$$I = I_{\mathrm{ph}} - I_0\left(e^{\frac{qV}{nkT}} - 1\right). \tag{30}$$

De twee uiterste punten van de I-V-curve volgen direct uit dit model. Bij $V = 0$ is de diodestroom verwaarloosbaar en geldt:

$$I_{\mathrm{sc}} = I_{\mathrm{ph}}.$$

Bij $I = 0$ verkrijgt men de openklemspanning:

$$V_{\mathrm{oc}} = \frac{nkT}{q}\left(\ln\frac{I_{\mathrm{ph}}}{I_0} + 1\right). \tag{31}$$

Door de belasting R systematisch te variëren kan de volledige I-V-curve experimenteel worden bepaald en daarmee het maximum power point, de fill factor en de efficiëntie van de zonnecel.

[9] De benaming “de wet van Shockley“ verdient enige nuance. Hoewel William Shockley een belangrijke bijdrage leverde aan de halfgeleiderfysica, is hij later bekend geworden om zijn controversiële en onwetenschappelijke uitspraken over intelligentie en ras. Deze opvattingen staan haaks op de waarden van de academische gemeenschap. Om die reden kiezen sommige instellingen ervoor om de betreffende diodevergelijking neutraal te benoemen, bijvoorbeeld als de ideale diodevergelijking of de pn-overgangsvergelijking.

Practicum

Opdracht 1(a)

1. Plaats de zonnecel in de opstelling en richt de lamp zo dat het oppervlak zo homogeen mogelijk wordt belicht. Noteer hierbij de afstand van de zonnecel tot de lamp.
2. Controleer de lichtverdeling door de intensiteit in het midden, de vier hoeken en langs de randen te meten. Noteer deze in het digitale labjournaal.

Opdracht 1(b)

1. Voer een volledige I–V-meting uit door de belasting systematisch te variëren.
2. Zorg voor een voldoende dichtheid aan meetpunten (minimaal 15–20) zodat zowel het kortsluitgebied als het openklemgebied goed wordt bemonsterd.
3. Registreer voor elk punt de stroom I, weerstand R en spanning V.

Opdracht 1(c)

1. Plot de I-V curve en bepaal aan de hand van een fit de kortsluitstroom en de open klem spanning plus de onzekerheid hierop.
2. Maak vervolgens de P-V-curve en bepaal het maximum power point (P_{mpp}, V_{mpp}, I_{mpp}).
3. Bepaal de interne kwaliteit van de zonnecel.
4. Bepaal de efficiëntie van de zonnecel.
5. Maak een P-R-curve, kan je hier een fit op toepassen? Waarom wel en waarom niet?

Als je bovenstaande opdrachten hebt afgerond kun je in de basis een zonnecel doormeten. De volgende stap is om hier een klein onderzoek mee te doen. Misschien heb je tijdens het practicum wel iets bedacht wat je graag wilt onderzoek, dat kan!

Opdracht 1(c)

1. Schrijf een kort projectvoorstel (Plan van Aanpak), van maximaal 10 regels, waarin je een voorstel doet voor een klein onderzoekje.
2. Voer het onderzoek uit en noteer jouw resultaten in het labjournaal.

Afronding

1. Zet de gevonden figuren in je digitaal labjournaal en maak een duidelijk onderschrift waarin je uitlegt wat de figuren laten zien.
2. Vul de **Resultaat- en Discussie**-sectie aan in je labjournaal. Hierin moet de uiteindelijke waarde van jouw experiment (met onzekerheid) staan en een korte discussie of deze waarde **realistisch** en **betrouwbaar** is.

Practica periode 3

Straling en (medische) Detectietechnieken

Practicum: SEM-EDX

Leerdoelen

- ☐ Je kan een beeld vormen met een SEM.
- ☐ Je kan elementen in een monster bepalen met EDX.

Veiligheid

1. Dit practicum heeft een verplichte interactieve training, geen voorbereiding = geen deelname! Kom daarnaast ook 15 minuten voor de start van jouw tijdslot!
2. Een SEM-EDX is een zeer kostbaar apparaat. Twijfel je over stappen die je moet zetten? Vraag dit dan aan Pieter-Paul Hooijschuur.

Introductie

In dit practicum maak je kennis met de Scanning Elektronen Microscopie (SEM) en Energy Dispersive X-ray Spectroscopy (EDX), twee krachtige technieken voor materiaalonderzoek. SEM biedt de mogelijkheid om oppervlakken met een hoge resolutie te bestuderen, terwijl EDX de chemische samenstelling van een monster kan analyseren. Tijdens het practicum leer je de basisprincipes van beide methoden en voer je metingen uit om hun praktische toepassingen te verkennen. Tijdens dit practicum zal door elke student met de SEM-EDX gewerkt worden, te vinden in het Nicolaas Tulphuis D2.11.

Ter voorbereiding is er een **verplichte interactieve training met een Virtual SEM**. De interactieve site is modulair opgebouwd en geeft uitleg over de werking en instellingen die nodig zijn om een juist beeld te krijgen.



Figuur 17: Moderne SEM-EDX apparatuur.

In dit practicum ga je een analyse van morfologie en elementsamenstelling van een aantal stubs doen met objecten waarmee de onderzoeksvraag per stub moet worden beantwoord.

Theorie

In de kennisclips op Brightspace worden verschillende imaging technieken uitgelegd met de elektronenmicroscoop. Gebruik die informatie samen met de simulatie om de voorbereidingsvragen uit de box hierboven te beantwoorden in je labjournaal.

Een scanning elektronenmicroscoop (SEM) vormt beelden door een dunne, gefocusseerde elektronenbundel lijn voor lijn over het oppervlak van een monster te scannen. In tegenstelling tot een lichtmicroscoop worden hier geen fotonen gebruikt, maar hoogenergetische elektronen die typisch tussen 1 en 30 kV versneld worden. Wanneer deze elektronen het monster binnendringen ontstaat een interactievolume op micrometerschaal, waarin de elektronen elastisch en inelastisch verstrooien. Deze interacties leveren verschillende signaaltypen op die informatie geven over topografie, materiaalcontrast en chemische samenstelling.

De meest gebruikte signalen in SEM zijn secundaire elektronen (SE) en backscattered elektronen (BSE). SE hebben een lage energie en ontsnappen alleen vanuit de bovenste nanometers van het oppervlak, waardoor zij uiterst geschikt zijn voor het in kaart brengen van fijne topografische details. BSE hebben een hogere energie en ontstaan diepere in het monster; zware elementen (hoger atoomnummer) verstrooien elektronen sterker terug en verschijnen daarom helderder, wat compositiecontrast mogelijk maakt. Daarnaast ontstaan karakteristieke röntgenfotonen wanneer een invallend elektron een binnenste schil ioniseert en een elektron van een hogere schil terugvalt. De

energie van dit röntgensignaal (EDX/EDS) is element-specifiek, waardoor lokale chemische analyse uitgevoerd kan worden.
Een SEM bestaat uit een elektronenkanon, elektromagnetische lenzen die de bundel focussen, afbuigspoelen die het beeld scannen, een vacuümkamer met monsterhouder (stub), en verschillende detectoren. Omdat elektronen interactie hebben met lucht is hoog vacuüm essentieel. Veel materialen moeten geleidbaar zijn om ladingopbouw te vermijden; niet-geleidende monsters worden vaak voorzien van een ultradunne geleidende coating (bijvoorbeeld goud, platina of koolstof).
Tijdens het experiment zijn meerdere instelparameters van belang. De versnellingsspanning bepaalt de penetratiediepte en daarmee het contrast en de ruimtelijke resolutie. De werkafstand beïnvloedt de scherpte, vooral bij hoge vergrotingen. Verder spelen de bundelspotsize (probe current), de monstervoorbereiding (droog, gefixeerd, geleidbaar) en de vacuümcondities een cruciale rol. Door een geschikte keuze van instellingen kan men variëren tussen oppervlaktedetail, compositiecontrast en maximale EDX-signaalopbrengst. Hierdoor vormt SEM–EDX een veelzijdige techniek voor materiaalonderzoek op micro tot nanometerschaal.

Belangrijke variabelen tijdens SEM–EDX

Parameter	Effect
Versnellingsspanning (kV)	Bepaalt penetratiediepte en interactievolume; hogere kV verhoogt signaal (BSE/EDX) maar kan resolutie verminderen en lading op isolatoren versterken.
Spotsize / probe current	Grotere spotsize geeft meer signaal (voor EDX) maar lagere resolutie.
Working distance	Kortere afstand geeft hogere resolutie; langere afstand kan nuttig zijn voor EDX-geometrie.
Vergroting	Wordt bepaald door het scanveld, niet door optiek zoals in lichtmicroscopie.
Monstervoorbereiding	Monster moet droog zijn; vaak geleidende coating voor isolatoren om ladingopbouw te voorkomen.
Vacuum	Nodig om elektronenverstrooiing door lucht te minimaliseren en detectorgevoeligheid te waarborgen.

Voorbereidingsopdracht week 1

1. Bestudeer voorafgaand aan de interactieve training de kennisclips op Brightspace.
2. Doorloop zelfstandig de interactieve training:
   http://www.ammrf.org.au/myscope/sem/introduction/
3. Voer de SEM- en EDX-simulaties uit via het tabblad *practical*.
4. Noteer de antwoorden op de vragen in je digitale labjournaal:
   - Welke instellingen zijn er bij de SEM simulatie en wat betekenen ze?
   - Welke elementen zitten er in messing, 14K goud en witgoud?
   - Waar heb je deze elementen voor nodig binnen het practicum?
5. Beantwoord de voorbereidende theorievragen en noteer deze in je labjournaal:
   - SEM: Leg uit wat SE en BSE detectie is. Wanneer gebruik je SE, en wanneer BSE?
   - EDX: Waarom is de gedetecteerde röntgenstraling in de EDX-machine afhankelijk van het acceleratievoltage?

Uitvoering

Vanwege de complexiteit van de SEM krijg je een korte instructie, waarna je zelfstandig een analyse uitvoert met de SEM-EDX.
Voer onderstaande oefeningen uit om kennis te maken met de SEM-EDX in het lab. Wissel daarbij de bediener bij de verschillende oefeningen af zodat iedereen de SEM heeft bediend.

Let op!

Je mag diverse instellingen wijzigen, de enige die je absoluut niet mag variëren is de “FIL I Target” (stroom door het filament)!

Opdrachten SEM

Opdracht 1: SE- en BSE-beeldvorming

Doel: leren scherpstellen, contrast optimaliseren en verschillen tussen SE- en BSE-detectie waarnemen.

1. Selecteer de referentiestub (positie 9) en kies de **SE-detector**.
2. Stel het **acceleratie-voltage** in op **15 kV**.
3. Druk op *reduced*, focus en vergroot tot een duidelijk beeld ontstaat.
4. Houd **brightness** op ∼50% en optimaliseer het beeld met **contrast**.
5. Stel de **working distance** in op ∼7,5 mm (verplaats Z-as en herfocus).
6. Wissel naar de **BSE-detector** en herhaal bovenstaande stappen.
7. Noteer voor beide detectoren de optimale instellingen (EHT en WD) en licht toe wat je als “optimaal” beoordeelt (bijv. spreiding grijswaarden, scherpte, contrast).

Opdracht 2: Effect van acceleratiespanning

Doel: inzicht krijgen in penetratiediepte, contrast en resolutie bij verschillende voltages.

1. Selecteer de referentiestub (positie 9) en kies de **BSE-detector**.
2. Stel WD in op ∼7,5 mm en begin bij **1–3 kV** (eventueel 5 kV).
3. Zoek een gebied met lichte bolletjes, focus en zoom in.
4. Verhoog het acceleratie-voltage naar **10 kV**; optimaliseer focus en contrast.
5. Verhoog naar **25 kV**; optimaliseer opnieuw.
6. Noteer voor elk voltage je observaties (helderheid, contrast, ruis, resolutie, doordringen van bundel).
7. Herhaal bovenstaande stappen bij de **SE-detector**.
8. Bespreek de waargenomen verschillen tussen lage, middelhoge en hoge kV-instellingen (voordelen/nadelen voor beeldkwaliteit).

Opdracht 3: Effect van spot size

Doel: onderzoeken hoe bundelstroom en resolutie afhangen van spot size.

1. Selecteer de referentiestub (positie 9).
2. Stel acceleratiespanning in op **5 kV** en WD op ∼7,5 mm.
3. Kies de **SE-detector**, focus en zoom.
4. Stel **spot size** ∼**400** in en optimaliseer scherpte en contrast.
5. Verhoog naar **spot size** ∼**600** en optimaliseer opnieuw.
6. Noteer verschillen in ruis, helderheid, resolutie en detailweergave.
7. Herhaal dezelfde procedure met de **BSE-detector**.

Opdrachten EDX

Zet de EDX computer aan. Start Esprit1.9. Start programma RComm32 op (bureublad SEM computer!). De SEM maakt middels het communicatieprogramma RComm32 connectie met Esprit1.9.

Opdracht 4: Elementanalyse (EDX)

Doel: spectrumverwerving en interpretatie van karakteristieke röntgenpieken (Sn/C monster).

1. Selecteer referentiestub (positie 9; Sn/C).
2. Kies de **BSE-detector**.
3. Stel zelf een geschikt **acceleratievoltage** in en houd de **working distance** op $\sim$7,5 mm (focus na elke Z-aanpassing).
4. Stel **spot size** $\sim$**300–400** in.
5. Open de EDX-software, kies *New* en start *Preview*.
6. Zoom in op een deeltje en selecteer **Point analysis**; plaats het analyse-kruis op het deeltje.
7. Controleer de signaalopbrengst (kcps); pas spot size aan totdat $\gtrsim 1$ kcps wordt bereikt.
8. Start de acquisitie (*Acquire*) en bekijk het spectrum.
9. Schakel relevante lijnen (bijv. Sn en C) zichtbaar en noteer observaties.
10. Controleer:
    - herkenbare karakteristieke lijnen van Sn en C;
    - eventuele extra elementen in het spectrum;
    - aanwezigheid van som-/pile-up peaks.
11. Herhaal de meting bij aangepast acceleratievoltage (bijv. 5 of 10 kV) en interpreteer verschillen in piekintensiteit, achtergrond en detecteerbare lijnen.

Opdracht 5: Elementmapping (EDX-mapping)

Doel: ruimtelijke spreiding van elementen visualiseren via mapping.

1. Selecteer de **BSE-detector** en plaats de referentiestub (positie 9).
2. Kies een passend **acceleratievoltage** en stel de **WD** in op $\sim$7,5 mm (herfocus tijdens Z-correcties).
3. Zoek een gebied met bolvormige deeltjes en stel scherp.
4. Start de EDX-software, kies *New* en *Preview*.
5. Zoom in binnen de EDX-interface indien gewenst.
6. Pas spot size aan zodat de input ≈ 1 kcps bedraagt.
7. Kies **Mapping** en selecteer maximaal vijf te analyseren elementen.
8. Observeer de elementverdeling: welke elementen tonen duidelijke aanwezigheid, welke ontbreken waarschijnlijk?
9. Rapporteer je waarnemingen en interpreteer het materiaalcontrast en de ruimtelijke relatie tussen de gedetecteerde elementen.

Afronding

1. Noteer je observaties, resultaten en conclusie/discussie in je digitale labjournaal.
2. Laat je resultaten controleren door een practicumbegeleider.
3. Werk openstaande vragen uit en vraag goedkeuring aan de practicumbegeleiders.

Appendix 1: bindingsenergie

Electron Binding Energies for X-ray Production (in keV)

Atomic Number	Element	K	L (average)
1	H	0.01	
2	He	0.02	
3	Li	0.05	
4	Be	0.11	
5	B	0.19	
6	C	0.28	
7	N	0.41	0.04
8	O	0.54	0.04
9	F	0.70	
10	Ne	0.87	0.03
11	Na	1.07	0.04
12	Mg	1.30	0.06
13	Al	1.56	0.09
14	Si	1.84	0.12
15	P	2.15	0.15
16	S	2.47	0.19
17	Cl	2.82	0.22
18	Ar	3.21	0.28
19	K	3.61	0.32
20	Ca	4.04	0.38
21	Sc	4.49	0.43
22	Ti	4.97	0.49
23	V	5.47	0.55
24	Cr	5.99	0.62
25	Mn	6.54	0.69
26	Fe	7.11	0.76
27	Co	7.71	0.83
28	Ni	8.33	0.91
29	Cu	8.98	0.99
30	Zn	9.66	1.09
31	Ga	10.37	1.19
32	Ge	11.10	1.29
33	As	11.87	1.40
34	Se	12.66	1.52
35	Br	13.47	1.64
36	Kr	14.33	1.78
37	Rb	15.20	1.91
38	Sr	16.11	2.05
39	Y	17.04	2.20
40	Zr	18.00	2.35
41	Nb	18.99	2.51
42	Mo	20.00	2.67
43	Tc	21.04	2.84
44	Ru	22.12	3.01
45	Rh	23.22	3.19
46	Pd	24.35	3.37
47	Ag	25.51	3.56
48	Cd	26.71	3.76
49	In	27.94	3.97
50	Sn	29.20	4.18
51	Sb	30.49	4.40
52	Te	31.81	4.63
53	I	33.17	4.87
54	Xe	34.56	5.12
55	Cs	35.99	5.36
56	Ba	37.44	5.62
57	La	38.93	5.88
58	Ce	40.44	6.15
59	Pr	41.99	6.41
60	Nd	43.57	6.69
61	Pm	45.18	6.97
62	Sm	46.83	7.26
63	Eu	48.52	7.55
64	Gd	50.24	7.85
65	Tb	52.00	8.16
66	Dy	53.79	8.47
67	Ho	55.62	8.79
68	Er	57.49	9.12
69	Tm	59.39	9.46
70	Yb	61.33	9.80
71	Lu	63.31	10.15
72	Hf	65.35	10.52
73	Ta	67.42	10.90
74	W	69.53	11.28
75	Re	71.68	11.67
76	Os	73.87	12.07
77	Ir	76.11	12.49
78	Pt	78.40	12.91
79	Au	80.73	13.34
80	Hg	83.10	13.78
81	Tl	85.53	14.23
82	Pb	88.01	14.70
83	Bi	90.53	15.17
84	Po	93.11	15.67
85	At	95.73	16.16
86	Rn	98.40	16.67
87	Fr	101.14	17.19
88	Ra	103.92	17.72
89	Ac	106.76	18.26
90	Th	109.65	18.82
91	Pa	112.60	19.38
92	U	115.61	19.96

Figuur 18: De bindingsenergie van elektronen in verschillende atomen.

Appendix 2: bindingsenergie

X-ray Emission Line Energies in keV

Atomic Number	Element	X-ray Line Energy in keV				
		Kα	Kβ	Lα	Lβ	Lγ
3	Li	0.054				
4	Be	0.109				
5	B	0.183				
6	C	0.277				
7	N	0.392				
8	O	0.525				
9	F	0.677				
10	Ne	0.849				
11	Na	1.041	1.071			
12	Mg	1.254	1.302			
13	Al	1.487	1.557			
14	Si	1.740	1.836			
15	P	2.014	2.139			
16	S	2.308	2.464			
17	Cl	2.622	2.816			
18	Ar	2.958	3.191			
19	K	3.314	3.590			
20	Ca	3.692	4.013	0.341	0.345	
21	Sc	4.091	4.461	0.395	0.400	
22	Ti	4.511	4.932	0.452	0.458	
23	V	4.952	5.427	0.511	0.519	
24	Cr	5.415	5.947	0.573	0.583	
25	Mn	5.899	6.490	0.637	0.649	
26	Fe	6.404	7.058	0.705	0.719	
27	Co	6.930	7.649	0.776	0.791	
28	Ni	7.478	8.265	0.852	0.869	
29	Cu	8.048	8.905	0.930	0.950	
30	Zn	8.639	9.572	1.012	1.035	
31	Ga	9.252	10.264	1.098	1.125	
32	Ge	9.886	10.982	1.188	1.219	
33	As	10.544	11.726	1.282	1.317	
34	Se	11.222	12.496	1.379	1.419	
35	Br	11.924	13.291	1.480	1.526	
36	Kr	12.649	14.112	1.586	1.637	
37	Rb	13.395	14.961	1.694	1.752	
38	Sr	14.165	15.836	1.807	1.872	
39	Y	14.958	16.738	1.923	1.996	
40	Zr	15.775	17.668	2.042	2.124	2.303
41	Nb	16.615	18.623	2.166	2.257	2.462
42	Mo	17.479	19.608	2.293	2.395	2.624
43	Tc	18.367	20.619	2.424	2.537	
44	Ru	19.279	21.657	2.559	2.683	2.965
45	Rh	20.216	22.724	2.697	2.834	3.144
46	Pd	21.177	23.819	2.839	2.990	3.329
47	Ag	22.163	24.942	2.984	3.151	3.520
48	Cd	23.174	26.096	3.134	3.317	3.717
49	In	24.210	27.276	3.287	3.487	3.921

Atomic Number	Element	X-ray Line Energy in keV				
		Kα	Kβ	Lα	Lβ	Lγ
50	Sn	25.271	28.486	3.444	3.663	4.131
51	Sb	26.359	29.726	3.605	3.844	4.348
52	Te	27.472	30.996	3.769	4.030	4.571
53	I	28.612	32.295	3.938	4.221	4.801
54	Xe	29.779	33.624	4.110		
55	Cs	30.973	34.987	4.287	4.620	5.280
56	Ba	32.194	36.378	4.466	4.828	5.531
57	La	33.442	37.801	4.651	5.042	5.789
58	Ce	34.720	39.257	4.840	5.262	6.052
59	Pr	36.026	40.748	5.034	5.489	6.322
60	Nd	37.361	42.271	5.230	5.722	6.602
61	Pm	38.725	43.826	5.433	5.961	6.892
62	Sm	40.118	45.413	5.636	6.205	7.178
63	Eu	41.542	47.038	5.846	6.456	7.480
64	Gd	42.996	48.697	6.057	6.713	7.786
65	Tb	44.482	50.382	6.273	6.978	8.102
66	Dy	45.998	52.119	6.495	7.248	8.419
67	Ho	47.547	53.877	6.720	7.525	8.747
68	Er	49.128	55.681	6.949	7.811	9.089
69	Tm	50.742	57.517	7.180	8.101	9.426
70	Yb	52.389	59.370	7.416	8.402	9.780
71	Lu	54.070	61.283	7.656	8.709	10.143
72	Hf	55.790	63.234	7.899	9.023	10.516
73	Ta	57.532	65.223	8.146	9.343	10.895
74	W	59.318	67.244	8.398	9.672	11.286
75	Re	61.140	69.310	8.653	10.010	11.685
76	Os	63.001	71.413	8.912	10.355	12.095
77	Ir	64.896	73.561	9.175	10.708	12.513
78	Pt	66.832	75.748	9.442	11.071	12.942
79	Au	68.804	77.984	9.713	11.442	13.382
80	Hg	70.819	80.253	9.989	11.823	13.830
81	Tl	72.872	82.576	10.269	12.213	14.292
82	Pb	74.969	84.936	10.552	12.614	14.764
83	Bi	77.108	87.343	10.839	13.024	15.248
84	Po	79.290	89.800	11.131	13.447	15.744
85	At	81.520	92.300	11.427	13.876	16.251
86	Rn	83.780	94.870	11.727	14.316	16.770
87	Fr	86.100	97.470	12.031	14.770	17.303
88	Ra	88.470	100.130	12.340	15.236	17.849
89	Ac	90.884	102.850	12.652	15.713	18.408
90	Th	93.350	105.609	12.969	16.202	18.983
91	Pa	95.868	108.427	13.291	16.702	19.568
92	U	98.439	111.300	13.615	17.220	20.167
93	Np			13.944	17.750	20.785
94	Pu			14.279	18.294	21.417
95	Am			14.617	18.852	22.065

Figuur 19: De X-ray energieën van verschillende overgangen bij verschillende atomen.

Practicum: bepaling van de verhouding e/m_e met een elektronenbundel

Leerdoelen

- ☐ De student kan de verhouding e/m_e bepalen door de data te lineariseren en hierop een passende fit toe te passen.
- ☐ De student kan veilig werken met hoogspanningsbronnen.

Veiligheid

1. De elektronenbuis is kwetsbaar en duur; ga er altijd zorgvuldig mee om.
2. Sluit voedingen pas aan wanneer alle draaiknoppen op $0\,\mathrm{V}$ en $0\,\mathrm{A}$ staan.
3. Laat de opstelling controleren door een practicumbegeleider voordat je spanning toepast.

Introductie

De verhouding van de elementaire lading e over de elektron massa m_e is in 1897 voor het eerst gemeten door Joseph John Thomson. Hiermee toonde hij aan dat het elektron een deeltje is met zowel een massa als een lading.

Een elektronenkanon is een van de meest eenvoudige deeltjesversnellers. Door een stroom door een gloeidraad te laten lopen wordt op macroscopisch niveau de gloeidraad warm. Op microscopisch niveau gebeuren er verschillende dingen. Één van deze dingen is dat de elektronen meer energie krijgen. Hierdoor hebben ze een grotere kans om te ontsnappen uit de vaste stof van de gloeidraad, een proces dat bestaat uit thermische emissie. Brengen we dit geheel in een uniform elektrisch veld door een spanningsverschil aan te leggen tussen de gloeidraad en een positieve anode, dan zullen de (bijna) vrije elektronen gaan versnellen. Zo krijg je elektronen die een redelijk uniforme snelheid krijgen in de richting van de anode.

In de opstelling zoals weergegeven in Fig. 21 kun je deze elektronenbundel zichtbaar maken doordat er een verdund fluorescerend gas is aangebracht binnen het hermetisch afgesloten glaswerk. Dit maakt de bundel zichtbaar als een groene streep. Vervolgens kun je met zogenaamde Helmholtz spoelen een magnetisch veld aanleggen waardoor je de bundel kan afbuigen, dit volgt uit de wet van Lorentz. Het uiteindelijke doel van dit experiment is om de verhouding e/m_e te bepalen.

Voorbereidingsopdracht

1. Bereken vooraf de theoretische verhouding e/m_e.
2. Stel een hypothese op over de waarde die jij experimenteel zult meten.
3. Schrijf vergelijking (2) om zodat r als functie van B wordt geschreven.
4. Hoe kun je uit de richtingscoëfficiënt van jouw fit e/m_e bepalen?
5. Bepaal vooraf hoeveel verschillende waarden van I_H je wilt gebruiken.

Theorie

Wanneer een elektron zich beweegt in een homogeen magnetisch veld, werkt er een Lorentzkracht op het deeltje. Voor een elektron dat loodrecht door een magnetisch veld beweegt geldt:

$$F_L = evB. \tag{32}$$

De kracht staat altijd loodrecht op zowel de bewegingsrichting als het magnetisch veld. Hierdoor verricht het magnetisch veld geen arbeid op het elektron: het verandert niet de snelheid, maar uitsluitend de bewegingsrichting.

Een kracht loodrecht op de snelheid fungeert als centripetale kracht en veroorzaakt een kromlijnige, idealiter cirkelvormige, baan. Uit de krachtenbalans volgt de baanstraal:

$$r = \frac{m_e v}{eB}. \tag{33}$$

Kinetische energie en versnelspanning

Elektronen worden versneld door de anodespanning U. De kinetische energie van het elektron wordt dan:

$$eU = \frac{1}{2} m_e v^2, \tag{34}$$

zodat de snelheid volgt als:

$$v = \sqrt{\frac{2eU}{m_e}}. \tag{35}$$

Door deze snelheid in de straalvergelijking te substitueren en te elimineren ontstaat na herschrijven de klassieke uitdrukking:

$$\frac{e}{m_e} = \frac{2U}{B^2 r^2}. \tag{36}$$

Deze formule vormt de kern van het experiment: met bekende U, en gemeten B en r, kan de verhouding e/m_e worden bepaald zonder directe snelheidsmeting.

Magnetisch veld van Helmholtz-spoelen

Voor twee spoelen met straal R, elk met n windingen en spoelstroom I_H, is het magnetisch veld in het centrum:

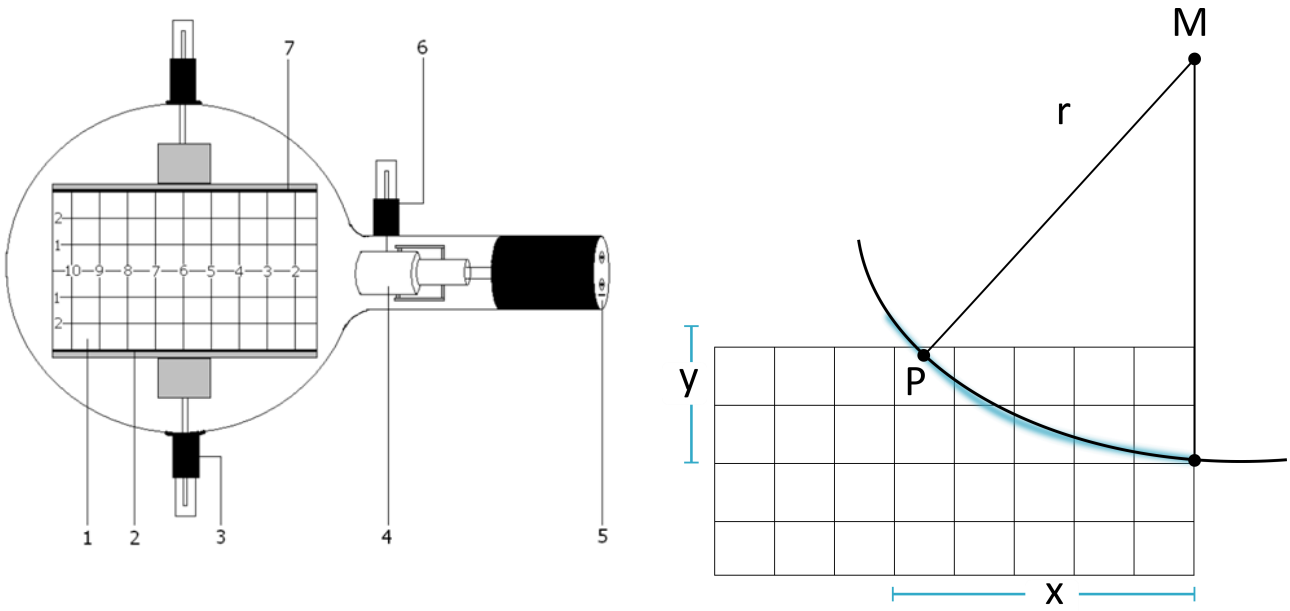
$$B = \mu_0 H = \left(\frac{4}{5}\right)^{3/2} \frac{\mu_0 n}{R} I_H. \tag{37}$$

Voor $n = 320$ windingen en $R = 68\,\mathrm{mm}$ volgt numeriek:

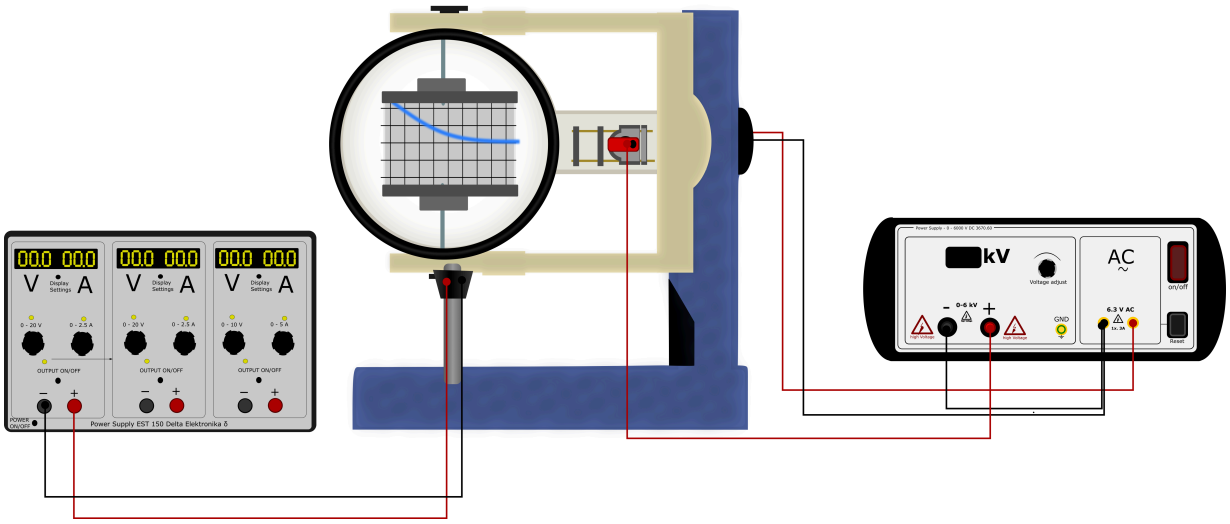
$$B \approx 4.17 \times 10^{-3} I_H. \tag{38}$$

Het veld is dus lineair evenredig met de spoelstroom I_H, wat experimentele controle en kalibratie eenvoudig maakt. De baanstraal kan niet rechtstreekt gemeten worden, maar volgt uit twee meetbare grootheden x en y binnen de buis, zie Fig. 20:

$$r = \frac{x^2 + y^2}{2y}. \tag{39}$$



Figuur 20: Geometrie van de elektronenbuis en de straalbepaling.

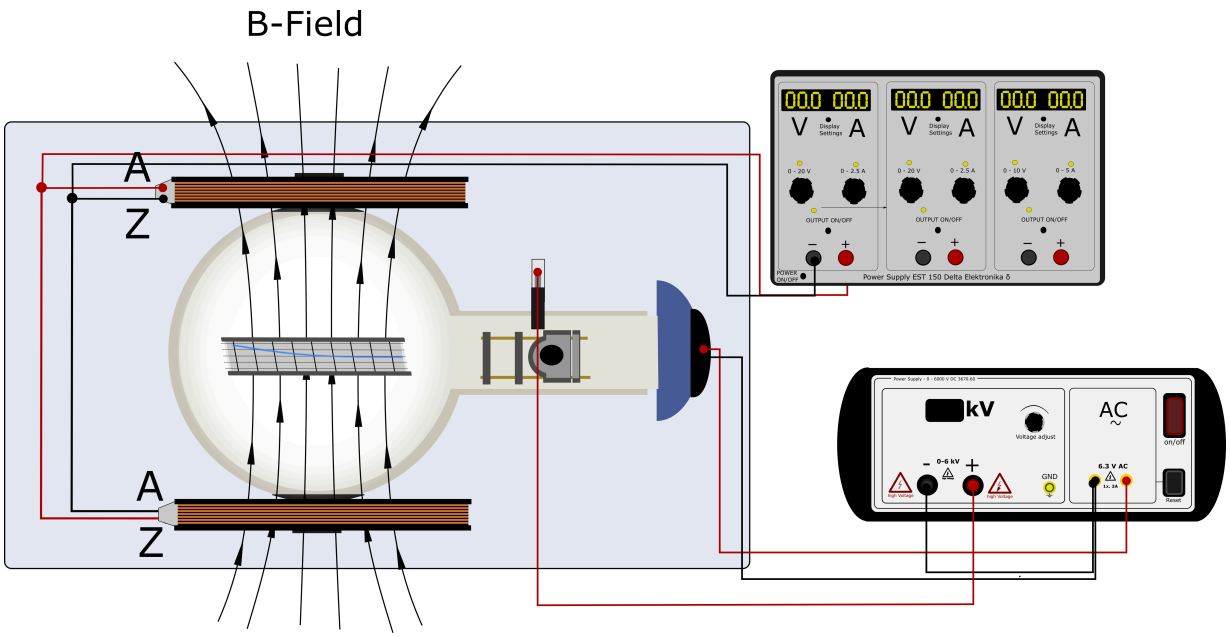


Figuur 21: Opstelling elektronenkanon

Practicum

Experiment

1. Stel de opstelling op zoals weergegeven in Figuur 22.
2. Sluit de minpool van de anodespanning aan op de 4 mm aansluiting met min-markering.
3. Plaats de Helmholtz-spoelen in de buishouder.
4. Sluit de verwarmingsspanning aan op de lamp (deze kan je niet varieren en is $U_F = 6.3 - 6.5\mathrm{V}$) en wacht enkele minuten tot de lamp brand.
5. Kies een anode spanning U_A tussen 1000 en 5000 V en wacht tot de anodestroom stabiliseert.
6. Breng spanning aan op de Helmholtz-spoelen en observeer de afbuiging van de bundel.
7. Meet x en y voor verschillende waarden van spoelstroom I_H.
8. Indien tijd: herhaal voor andere waarden van U_A.



Figuur 22: Opstelling elektronenkanon

Verwerking

1. Bepaal U, r en B voor alle metingen.
2. Plot r versus $1/B$ inclusief onzekerheden.
3. Pas een lineaire regressie toe en bepaal daaruit e/m_e.
4. Bepaal de standaardfout van e/m_e via de regressie (en foutenpropagatie).
5. Visualiseer jouw meetwaarden in een histogram en observeer wat er gebeurt als je meer data toevoegt.

Resultaten en Discussie

1. Laat de grafiek zien van r tegen $1/B$.
2. Bespreek of de trend overeenkomt met jouw theoretische verwachting.
3. Onderzoek systematische afwijkingen en hun invloed op de fit.
4. Verklaar eventuele verschillen tussen jouw gemeten waarde en de verwachting.

Afronding

1. Zet de gevonden figuren in je digitaal labjournaal en maak een duidelijk onderschrift waarin je uitlegt wat de figuren laten zien.
2. Vul de **Resultaat- en Discussie**-sectie aan in je labjournaal. Hierin moet de uiteindelijke waarde van jouw experiment (met onzekerheid) staan en een korte discussie of deze waarde **realistisch** en **betrouwbaar** is.
3. Ruim pas op na akkoord van de begeleider.

Practicum: Electronenbundel – Elektronendiffractie op grafiet

Leerdoelen

- ☐ De student kan diffractieringen meten en de bijbehorende golflengte van elektronen bepalen.
- ☐ De student kan de Bragg-vergelijking toepassen om uit diffractie op microkristallijne grafiet de roosterafstand te bepalen.
- ☐ De student kan de de Broglie-relatie experimenteel bevestigen.

Veiligheid

1. De elektronenbuis is dunwandig, sterk gevacueerd en kan imploderen. Ga uiterst voorzichtig om met de glasbuis. :contentReference[oaicite:1]index=1
2. Raak het glas niet aan met gereedschap; voorkom mechanische stress.
3. Laat de buis afkoelen voordat je demonteert; de buis kan warm worden tijdens gebruik. :contentReference[oaicite:2]index=2

Introductie

Elektronen werden in de klassieke fysica lange tijd beschouwd als puntdeeltjes zonder golfkarakter. Aan het einde van de 19e eeuw werd echter steeds duidelijker dat licht een dubbel karakter heeft: het gedraagt zich zowel als golf (Young, Fresnel) als deeltje (Einstein, foto-elektrisch effect). In 1924 stelde Louis de Broglie een revolutionair idee voor: niet alleen licht, maar alle materiedeeltjes (waaronder elektronen) hebben een golflengte. Deze golflengte hangt af van het impuls van het deeltje en wordt beschreven door de de Broglie-relatie:

$$\lambda = \frac{h}{p}, \tag{40}$$

waarbij h de constante van Planck is en p het impuls van het elektron. Het bijzondere aan elektronen is dat hun golflengte experimenteel zichtbaar kan worden gemaakt. Dit idee was zó revolutionair, dat het de basis vormde van het atoommodel van Bohr. Het werd kort daarna werd dit bevestigd in experimenten van Davisson en Germer (1927), die diffractiepatronen van elektronen op nikkel observeerden, vergelijkbaar met röntgendiffractie. Dit was het eerste directe bewijs dat elektronen zich als materiegolven gedragen en markeerde een fundamentele stap in de ontwikkeling van de kwantummechanica.

In dit experiment gaan jullie aan de slag met elektronendiffractie. Het is historisch gezien een van de meest elegante experimenten: het visualiseert in één oogopslag dat een fundamenteel deeltje een golflengte heeft die groot genoeg is om interferentie te vertonen op atomair schaalniveau. In de opstelling van dit practicum wordt een elektronenbundel gericht op een dun polycrystallijn grafietlaagje, en de resulterende diffractieringen worden zichtbaar gemaakt op een fluorescerend scherm.

Voorbereiding

1. Zoek op wat de verwachte roosterafstanden van grafiet zijn.
2. Leid de relatie af tussen λ en U_A via vergelijking (1).
3. Leid af hoe je uit D en L de hoek ϑ bepaalt.
4. Formuleer een hypothese: hoe verandert λ als functie van U_A?
5. Noteer hoe je onzekerheden in D gaat schatten.

Theorie

Wanneer elektronen versneld worden door een versnellingsspanning U_A, krijgen ze een kinetische energie die gelijk is aan het elektrische energiewerk:

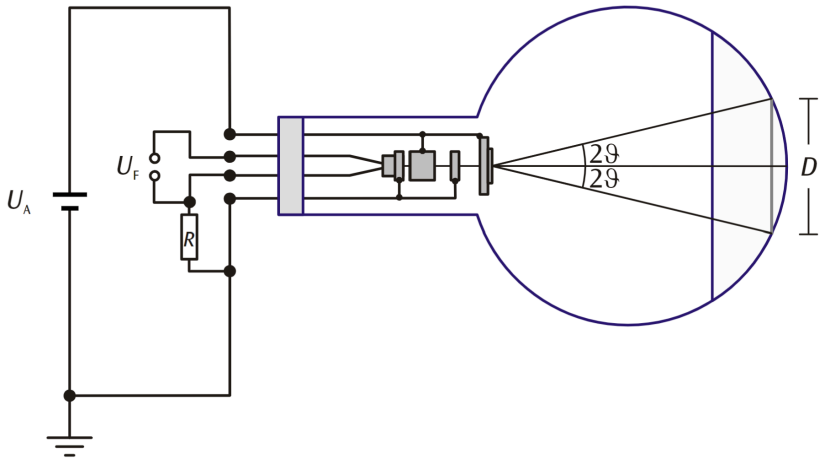
$$eU_A = \frac{1}{2}mv^2. \tag{41}$$

Daaruit volgt dat de snelheid, en dus het impuls, direct gekoppeld is aan de versnellingsspanning. Door dit te combineren met de de Broglie-relatie ontstaat een uitdrukking voor de golflengte als functie van de spanning:

$$\lambda = \frac{h}{\sqrt{2meU_A}}. \tag{42}$$

Bij versnellingen van enkele kilovolt ligt de bijbehorende golflengte in het picometerbereik. Dit is van dezelfde orde als interatomaire afstanden in vaste stoffen. Daardoor kunnen elektronen, net als röntgenstraling, diffractie vertonen wanneer ze op kristalroosters vallen. Het waarnemen van interferentie- en diffractieringen vormt dus een directe experimentele bevestiging van de golfnatuur van elektronen.

In een elektronen-diffractiebuis wordt een elektronenbundel gericht op een dunne polycrystalline grafietlaag. Het uitgewaaierde diffractiepatroon wordt zichtbaar gemaakt op een fluorescent scherm, zie Fig. 23. Hierbij verschijnen karakteristieke ringen: elke ring komt overeen met een groep kristalvlakken die precies voldoet aan de diffractievoorwaarde, afhankelijk van de golflengte van het elektron en de roosterafstand van het materiaal.

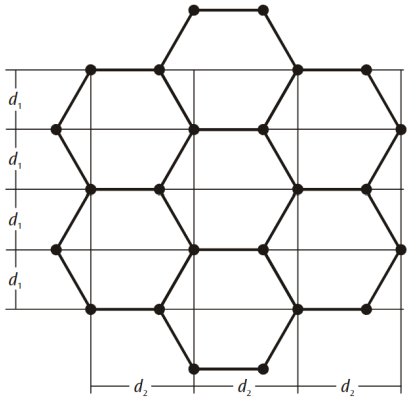


Figuur 23: Schematische weergave van het elektronen diffractie experiment.

Wanneer een bundel elektronen een polycrystallijn materiaal raakt, treden diffractie-effecten op vanuit de vele willekeurig georiënteerde microkristallen. De conditionele reflectie van golven aan kristalvlakken wordt beschreven door de Bragg-voorwaarde:

$$2d \sin \theta = n\lambda, \tag{43}$$

waarbij d de afstand is tussen de kristalvlakken, θ de Bragg-hoek, λ de golflengte van het elektron en n de diffractieorde.



Figuur 24: Kristalstructuur van grafiek, met hierbij de twee kristralrooster constanten $d_1 \approx 123\,\mathrm{pm}$ en $d_2 \approx 213\,\mathrm{pm}$.

Grafiet heeft twee dominante roosterafstanden in het hexagonale vlak $d_1 \approx 123\,\mathrm{pm}$ en $d_2 \approx 213\,\mathrm{pm}$, zie Fig. 24. Hierdoor ontstaan in het diffractiepatroon twee heldere ringen, één voor elk kristalvlak. De straal van de ring die zichtbaar is op het fluorescentiescherm hangt samen met de hoek θ. Voor een elektronen-diffractiebuis met afstand L tussen grafiet en scherm volgt:

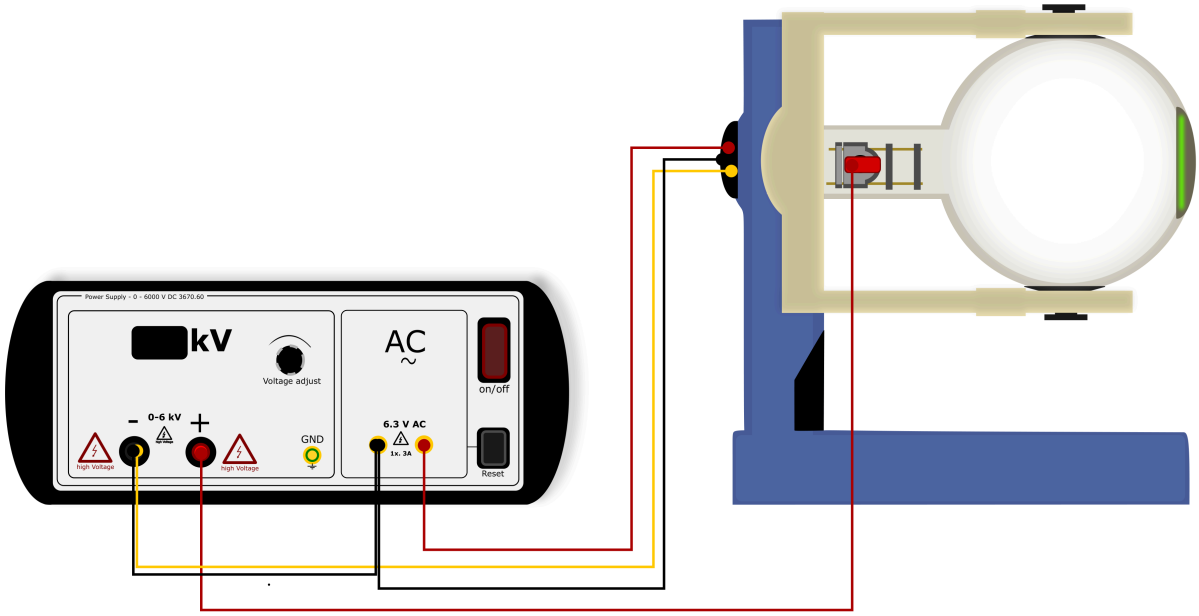
$$D = 2L \tan(2\theta), \tag{44}$$

waarbij D de diameter van een diffractiering is. Door D te meten bij verschillende versnellingsspanningen U_A, kan men via de Bragg-vergelijking de golflengte reconstrueren. Door λ zowel via de Bragg-vergelijking als via de de Broglie-relatie te bepalen, kun je experimenteel aantonen dat elektronen zich gedragen als materiegolven.

Practicum

Uitvoering

1. Monteer de diffractiebuis in de houder; zorg dat de pennen correct contact maken. Zie Fig. 25 voor de opstelling.
2. Sluit de gloeidraad aan op de heater-uitgangen van de hoogspanningsvoeding.
3. Sluit de negatieve hoogspanningsuitgang aan op het kathodecontact en de positieve kant op de anodecontacten volgens de buishouder.
4. Zet de hoogspanning op $U_A = 6000\,\mathrm{V}$ en observeer de diffractieringen.
5. Meet de diameter van beide ringen D_1 en D_2 op het fluorescentiescherm.
6. Verlaag U_A telkens met $500\,\mathrm{V}$, herhaal de meting van D_1 en D_2.
7. Noteer systematisch alle meetwaarden in je logboek.



Figuur 25: Elektronen diffractie opstelling.

Verwerking

1. Bepaal voor iedere spanning U_A de verwachte de Broglie-golflengte via (3).
2. Bepaal voor iedere gemeten diameter D de Bragg-hoek:
$$\theta = \arctan\left(\frac{D}{4L}\right).$$
3. Toets voor beide ringen de Bragg-vergelijking:
$$2d\sin\theta = \lambda.$$
4. Maak een grafiek van de Bragg-bepaalde golflengte vs. de de Broglie-golflengte.
5. Maak een fit. In het ideale geval heb je een 1:1 lijn wanneer beide metingen dezelfde overeenkomen.

Resultaten & Discussie

1. Toon grafieken voor $D_1(U_A)$ en $D_2(U_A)$.
2. Toon de vergelijking tussen experimentele λ_{Bragg} en theoretische $\lambda_{\mathrm{de\ Broglie}}$.
3. Bespreek onzekerheden in D, invloed van schermkromming en afleesfouten.

4 Bespreek of de fit jouw verwachting ondersteunt.
5 Formuleer een conclusie: bevestigd jouw meting het golfkarakter van elektronen.

Afronding

1 Zet de gevonden figuren in je digitaal labjournaal en maak een duidelijk onderschrift waarin je uitlegt wat de figuren laten zien.
2 Vul de **Resultaat- en Discussie**-sectie aan in je labjournaal. Hierin moet de uiteindelijke waarde van jouw experiment (met onzekerheid) staan en een korte discussie of deze waarde **realistisch** en **betrouwbaar** is.
3 Ruim de opstelling pas op als de stroom UIT staat en de buis voldoende is afgekoeld.

Practicum: apparaat communicatie met Python

Leerdoelen

- ☐ Kan communiceren met basis labapparatuur via Python
- ☐ Kan uitleggen welke stappen nodig zijn voor communicatie met labapparatuur.
- ☐ Is bekend en kan overweg met de PyVisa-package van Python.
- ☐ De student kan een overzichtelijke map- en modulestuctuur gebruiken en opzetten voor instrumentcommunicatie, passend bij een groter Python-project.

Veiligheid

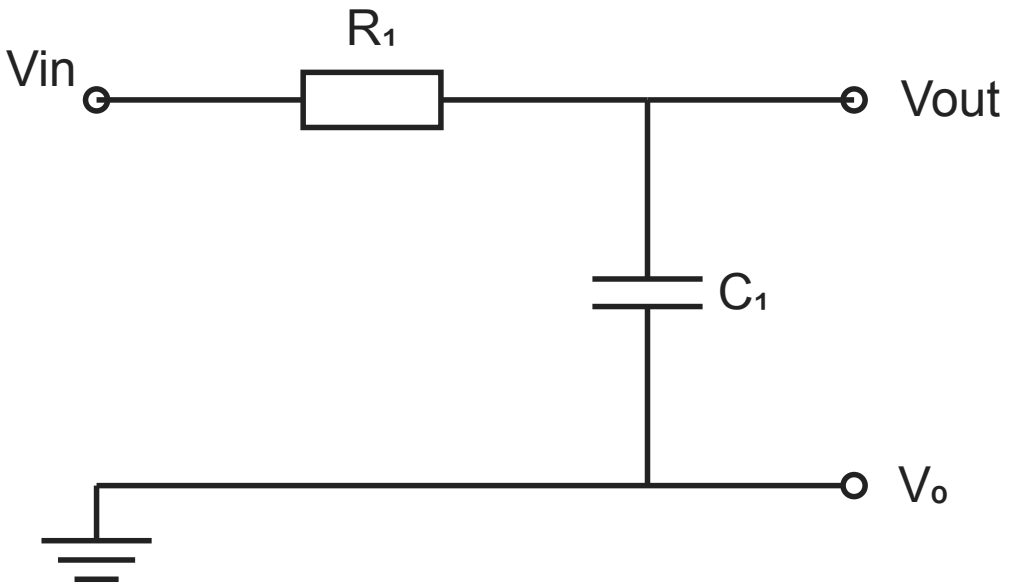
1 We werken deze practicumreeks met elektrische schakelingen. Zorg ervoor dat je deze goed aansluit voordat je verder gaat met experimenteren. Anders kan er kortsluiting ontstaan in de apparatuur. Laat je werk checken vooraf.

Historische ontwikkeling van apparaatuuraansturing

Het betrouwbaar uitvoeren en registreren van metingen vormt het hart van elk experiment. De manier waarop meetinstrumenten worden aangestuurd, heeft de afgelopen decennia een sterke ontwikkeling doorgemaakt. Aanvankelijk gebeurde dit volledig handmatig: instellingen werden aangepast met draaiknoppen en schakelaars, en resultaten werden afgelezen op analoge meters of oscilloscopen. De eerste automatiseringsstappen kwamen met seriële communicatieprotocollen zoals **RS-232** en later **GPIB** (General Purpose Interface Bus), waarmee instrumenten centraal vanaf een computer konden worden aangestuurd. Programmeertalen als *Fortran*, *C* en later *LabVIEW* werden gebruikt voor het verzenden van commando's en het uitlezen van data, meestal via relatief laag-niveau communicatie die gebruikers handmatig moesten programmeren.
Met de komst van microcontrollers veranderde de interne aansturing van apparaten ingrijpend. Deze compacte chips combineren processor, geheugen en I/O en voeren specifieke taken uit, zoals het regelen van spanningen of uitlezen van sensoren. Veel meetinstrumenten kregen interne microcontrollers, terwijl aan de buitenzijde seriële of GPIB-communicatie gehandhaafd bleef. Tegelijkertijd maakten goedkope ontwikkelplatforms zoals de **Arduino** en **Raspberry-Pi** het mogelijk om zelf eenvoudig meet- en regelsystemen te bouwen, wat vooral in onderwijs en kleinschalig onderzoek veel werd toegepast.
Tegenwoordig is apparaatuuraansturing grotendeels gestandaardiseerd via de **VISA**-standaard (Virtual Instrument Software Architecture), die zowel klassieke verbindingen (RS-232, GPIB) als moderne interfaces zoals **USB** en **Ethernet** ondersteunt. Hierdoor kan met één softwarelaag een breed scala aan instrumenten worden aangestuurd. Een belangrijke recente ontwikkeling is de opkomst van Python als universeel hulpmiddel. Met bibliotheken als `PyVISA` kunnen instrumenten direct via GPIB, USB of netwerk worden bestuurd. Waar vroeger dure propriëtaire software nodig was, volstaat nu vaak een korte Python-script en een standaardinterface. In deze practicumserie gaan we aan de slag met apparaat-communicatie via Python. De schakeling die wij gaan aansturen betreft een passief RC-filtersysteem.

Passieve RC-filtersystemen

Een **passief RC-filter** is een eenvoudig elektrisch netwerk van weerstanden en condensatoren waarmee de frequentierespons van een signaal kan worden aangepast, zonder actieve componenten zoals versterkers. Zulke filters worden veel toegepast om ongewenste frequenties te onderdrukken of juist door te laten, bijvoorbeeld om ruis te verwijderen of een meetsignaal te conditioneren voordat het naar een meetinstrument gaat. In figuur 26 staat een voorbeeld elektrische schakeling van een laagdoorlaatfilter.



Figuur 26: Laagdoorlaatfilter: R in serie, C naar massa; de uitgangsspanning wordt over C gemeten.

Het gedrag van een RC-filter kan worden beschreven met behulp van de impedantie van een condensator,

$$Z_C(\omega) = \frac{1}{i\omega C},$$

waarbij $\omega = 2\pi f$ de hoekfrequentie is en i de imaginaire eenheid. Voor een eenvoudig **laagdoorlaatfilter**, bestaande uit een weerstand R in serie en een condensator C naar massa, volgt voor de overdrachtsfunctie

$$H(\omega) = \frac{V_{\text{uit}}}{V_{\text{in}}} = \frac{1}{1 + i\omega RC}.$$

Bij lage frequenties ($\omega \ll 1/RC$) is $\mid H \mid \approx 1$ en wordt het signaal vrijwel ongehinderd doorgelaten. Bij hoge frequenties ($\omega \gg 1/RC$) neemt de amplitude af als $1/\omega$, zodat snelle variaties en ruis worden onderdrukt. De overgangsfrequentie, of **afsnijfrequentie**, wordt gegeven door

$$f_c = \frac{1}{2\pi RC}.$$

Om van de overdrachtsfunctie naar een **Bode-plot** te gaan, splits je $H(\omega)$ op in een amplitude- en fasegedeelte. De amplitude wordt uitgedrukt in decibel,

$$\mid H(\omega) \mid_{\text{dB}} = 20 \log_{10} \mid H(\omega) \mid,$$

en de fase als

$$\phi(\omega) = \arg\big(H(\omega)\big).$$

Voor het laagdoorlaatfilter hierboven volgt eenvoudig

$$\mid H(\omega) \mid = \frac{1}{\sqrt{1 + (\omega RC)^2}}, \qquad \phi(\omega) = -\arctan(\omega RC).$$

Door deze twee functies uit te zetten tegen de logaritme van de frequentie ontstaat de Bode-plot: een amplitude-grafiek die de verzwakking per frequentie laat zien en een fase-grafiek die aangeeft

hoe sterk het signaal wordt vertraagd. Helaas kunnen wij de fase-verschuiving niet meten met een **Digitale Multi Meter (DMM)**, hiervoor zouden wij een oscilloscoop moeten gebruiken. Deze grafische weergave maakt het filtergedrag bij lage, overgangs- en hoge frequenties direct zichtbaar. Naast **laagdoorlaatfilters** bestaan er ook **hoogdoorlaatfilters** door de positie van weerstand en condensator om te wisselen. In dat geval worden juist lage frequenties onderdrukt en hoge frequenties doorgelaten. Met combinaties van meerdere RC-netwerken kunnen **banddoorlaat- of bandstopfilters** worden gerealiseerd.

Laagdoorlaat en hoogdoorlaat onderscheiden

Meet de uitgangsspanning steeds tussen het knooppunt van R en C en massa.

- **Laagdoorlaat:** R in serie met de ingang, C naar massa. Meet de uitgang over C.
- **Hoogdoorlaat:** C in serie met de ingang, R naar massa. Meet de uitgang over R.

Voor het hoogdoorlaatfilter geldt

$$H_{\text{hoog}}(\omega) = \frac{i\omega RC}{1 + i\omega RC}.$$

Lage frequenties worden onderdrukt; hoge frequenties worden doorgelaten. Beide filters hebben een afsnijfrequentie $f_c = 1/(2\pi RC)$, waar de amplitudeverhouding $1/\sqrt{2}$ is (ongeveer -3 dB). Deze formules veronderstellen een ideale spanningsbron en een uitgang die niet merkbaar wordt belast door het meetinstrument.

Voorbereidingsopdracht 1

- Installeer **NI-Max** op jouw laptop Link[10].
- Maak een nieuwe virtuele omgeving in Anaconda en installeer hierop de volgende packages: "pyvisa", "pyvisa-py","numpy", "matplotlib", "ipykernel" & "Pyserial?", gebruik Python versie 3.11.
- Download (of clone) de github repository voor de aansturing van de apparaten via Link.
- Maak een schematische tekening van een zelfgekozen passief filtersysteem.
- Schrijf een meetplan voor doormeten van een jouw filter (bode-plot).

[10] Je zult hiervoor een account moeten aanmaken. Dit is helaas de harde realiteit dat niet alles écht open-source is.

Practicum

Opdracht 1(a)

In deze eerste opdracht gaan we de het filter bouwen en deze handmatig doormeten. Voordat we gaan automatiseren moeten we immers zeker weten dat onze apparatuur goed aangesloten staat.

1. Bouw op het breadboard jouw gekozen filter en sluit de functiegenerator en digitale multimeter aan. Controleer bij jouw practicumbegeleider of deze goed aangesloten staan.
2. Meet, met de hand, bij enkele verschillende frequenties het in-en uitgangssignaal en bereken de overdrachtsfunctie $\mid H(\omega) \mid$ op deze frequenties. Gebruik hiervoor een (RMS) voltage van $V_{rms} = 2$ volt.
3. Schrijf jouw resultaten op in je digitale labjournaal.

Opdracht 1(b)

Nu we zeker weten dat het filtersysteem werkt gaan we deze aansturen via Python.

1. Sluit de digitale multimeter en de functiegenerator aan via de USB-B kabel.
2. Open NI-MAX en zoek de bijbehorende apparaat ID'en van de apparatuur.
3. Maak een nieuwe Jupyter Notebook met de naam `rc_filter.ipynb` en plaats deze op de juiste locatie binnen jouw projectstructuur (zie Figuur 27).[11]
4. Controleer nu met onderstaande code of je kan communiceren met jouw apparatuur. **Let op!** De functiegenerator (fg) en de digitale multimeter (dmm) kunnen omgewisseld zijn.

Met deze code controleer je of je kan communiceren met je meetapparatuur. Daarnaast zorgt deze code ervoor dat de communicatiepoorten geopend zijn. Dit is een belangrijk details, want, na gebruik moeten wij ook de apparatuur weer sluiten.

Test communicatie meetapparatuur

```python
import pyvisa
rm = pyvisa.ResourceManager()

fg = rm.open_resource(rm.list_resources()[0])
dmm = rm.open_resource(rm.list_resources()[1])

#Controleer of de functiegenerator (fg) en de digitale multimeter (dmm) niet
omgewisseld zijn.
print("Instrument gevonden:", fg.query("*IDN?"))
print("Instrument gevonden:", dmm.query("*IDN?"))
```

Nu we zeker weten dat we kunnen communiceren met onze labapparatuur, is het belangrijk om de code duidelijk en onderhoudbaar te organiseren. Het doel is om niet telkens handmatig ieder instrument aan te sturen, maar gebruik te maken van herbruikbare functies en modules die dit voor ons afhandelen.
Een overzichtelijke manier om dit te structureren is met een *boomstructuur*, waarbij elke map een duidelijke rol binnen het project heeft. Bovenin bevinden zich de algemene modules en documentatie; dieper in de structuur staan de instrument-drivers, hulpfuncties en mappen voor data-opslag. Deze indeling maakt het eenvoudiger om de code te begrijpen, te testen en later uit te breiden. In figuur 27 is te zien hoe deze onderdelen logisch zijn gegroepeerd. De map `instruments` bevat bijvoorbeeld de drivers voor verschillende meetinstrumenten, terwijl `utils` de ondersteunende functies bundelt die in meerdere modules worden gebruikt. De map `data` is gereserveerd voor ruwe meetdata en

[11] In professionele software-engineering wordt doorgaans niet met notebooks gewerkt, maar uitsluitend met `.py`-modules. Voor dit practicum maken we een uitzondering, omdat we zowel de meetcode als de gegenereerde data en visualisaties in één bestand willen kunnen bekijken.

gegenereerde plots. Deze indeling maakt het eenvoudiger om onderdelen te vinden, te testen en uit te breiden, en zorgt ervoor dat de communicatiecode voor verschillende instrumenten consistent blijft opgebouwd.

```
Communicatie/
├── instruments/
│   ├── __init__.py
│   ├── hardware.py
│   ├── rigol_dg1022.py
│   └── rigol_dm3058e.py
├── utils/
│   ├── __init__.py
│   └── functions.py
├── data/
│   ├── raw/
│   └── plots/
├── rc_filter.py(nb)
├── README.md
└── .gitignore
```

Figuur 27: Boomdiagram voor apparaatcommunicatie.

Opdracht 1(c)

Nu we zeker weten dat we kunnen communiceren met onze labapparatuur via Python, kunnen we de eerste testmetingen gaan uitvoeren.

1. Stel **via Python** een voltage in van 2 Volt (Root Mean Squared) bij de functiegenerator met een frequentie van 100 Hz. Controleer op de display van de functiegenerator of deze correct staat ingesteld. Gebruik hiervoor de onderstaande code. Zie je niets gebeuren? Dan kan het zijn dat jouw de functiegenerator en digitale multimeter omgewisseld zijn!
2. Leg uit wat iedere regel doet en schrijf dit op in je labjournaal.

Functiegenerator aansturen met Python

```
from instruments.hardware import open_instruments
from instruments.rigol_dg1022 import set_fixed_sine

rm, fg, dmm = open_instruments(message = 1)

#Aansturing functigenerator
f = 100 #Hz
vrms_output = 2.0 #Volt
set_fixed_sine(fg, freq=f, vrms=vrms_output)
```

Opdracht 1(c)

De volgende stap is het doen van een meting via Python. Hiervoor gebruiken wij de digitale multimeter.

1. Meet het ingangsvoltage en het uitgangsvoltage 5 keer. Gebruik hiervoor als inspiratie onderstaande code.
2. Sla de data op in de dictionary `results`.

Digitale multimeter aansturen met Python

```
from instruments.rigol_dm3058e import measure_vrms, measure_vrms_n
from utils.functions import save_as_csv

#Start meting digitale multimeter
vals = measure_vrms_n(dmm,5)

#opslaan in een nette structuur
results = {}
results[f] = {
            "samples": vals,
            "mean": float(vals.mean()),
            "std": float(vals.std(ddof=1)),
            "n": len(vals)
        }
save_as_csv(results, "data/raw/input_voltage.csv")
```

Opdracht 1(c)

De laatste stap nu de data weer inladen.

1. Laad de data in. Gebruik hiervoor als inspiratie onderstaande code.
2. Schrijf de gemeten gemiddelde waarde en standaard fout in je labjournaal.

Uitlezen data

```
from utils.functions import load_from_csv
data = load_from_csv("data/raw/input_voltage.csv")

freqs_in  = list(data.keys())
V_mean    = [v["mean"]    for v in data.values()]
V_std     = [v["std"]     for v in data.values()]
samples   = [v["samples"] for v in data.values()]
```

Opdracht 1(c)

1. Sluit de apparatuur nu weer af met Python. Zie onderstaande code.

Afsluiten meetapparatuur

```
from instruments.hardware import close_instruments
close_instruments(rm, fg, dmm, message=1)
```

Opdracht 1(d)

1. Maak een bode-plot van jouw gekozen filtersysteem.
2. Bepaal de afsnijfrequentie aan de hand van jouw **bode-plot** en vergelijk deze met jouw gekozen waarde. Je hoeft hiervan alleen te kijken naar de amplitude en dus niet naar de fase[12].

[12] Dat kan ook niet in dit geval, daarvoor hebben we een oscilloscoop nodig.

Afronding

1. Zet de gevonden figuren in je digitaal labjournaal en maak een duidelijk onderschrift waarin je uitlegt wat de figuren laten zien.
2. Vul de **Resultaat- en Discussie**-sectie aan in je labjournaal. Hierin moet de uiteindelijke waarde van jouw experiment (met onzekerheid) staan en een korte discussie of deze waarde **realistisch** en **betrouwbaar** is.

Appendix 1: Drivers & virtuele omgeving

Voor communicatie tussen Python en laboratoriumapparatuur (zoals oscilloscopen en functiegeneratoren) wordt gebruikgemaakt van de VISA-standaard (*Virtual Instrument Software Architecture*). Deze standaard definieert een uniforme interface voor het aanspreken van meetinstrumenten via verschillende fysieke verbindingen, zoals USB, GPIB en LAN.

VISA, PyVISA en drivers

- **VISA-driver (bijv. NI-VISA, Keysight VISA, TekVISA, R&S VISA):** Dit is de onderliggende software die zorgt voor de fysieke communicatie met het instrument. Zie dit als de *postdienst* die weet hoe een bericht (via USB, LAN, GPIB) op de juiste plek komt.
- **PyVISA (Python package):** Dit is een Python-interface naar de VISA-driver. PyVISA stuurt geen data rechtstreeks naar de hardware, maar gebruikt de geïnstalleerde VISA-library (bijv. `visa64.dll`) als tussenlaag.
- **SCPI (Standard Commands for Programmable Instruments):** Dit is de *taal* waarin de instrumenten commando's ontvangen, zoals `MEAS:VOLT?` of `FREQ 1KHZ`. Vrijwel alle moderne meetinstrumenten ondersteunen SCPI.
- **NI-MAX:** NI Measurement & Automation Explorer is een hulpmiddel om apparaten te detecteren en resource strings (zoals `USB0::0x1AB1::::::INSTR`) te achterhalen. PyVISA kan ook zonder NI-MAX werken, mits het juiste adres van het instrument bekend is.

Naast NI-VISA kunnen ook andere VISA-implementaties worden gebruikt (Keysight, Tektronix, Rohde & Schwarz, of Rigol UltraSigma).

NI-MAX

NI Measurement & Automation Explorer (NI-MAX) is een hulpprogramma van National Instruments waarmee je:

- Apparaten kunt detecteren en configureren die via GPIB, USB, LAN of andere bussen verbonden zijn.
- Resource strings kunt achterhalen die nodig zijn voor communicatie via `PyVISA`.
- Instrumenten kunt testen door eenvoudige SCPI-commando's te sturen.

NI-MAX wordt meestal geïnstalleerd samen met NI-VISA, de driver die `PyVISA` gebruikt om daadwerkelijk met het instrument te communiceren.

Tip: Na installatie kun je in NI-MAX je Rigol oscilloscoop vinden en de juiste USB-resource string kopiëren voor gebruik in Python.

Installatie en virtuele omgeving

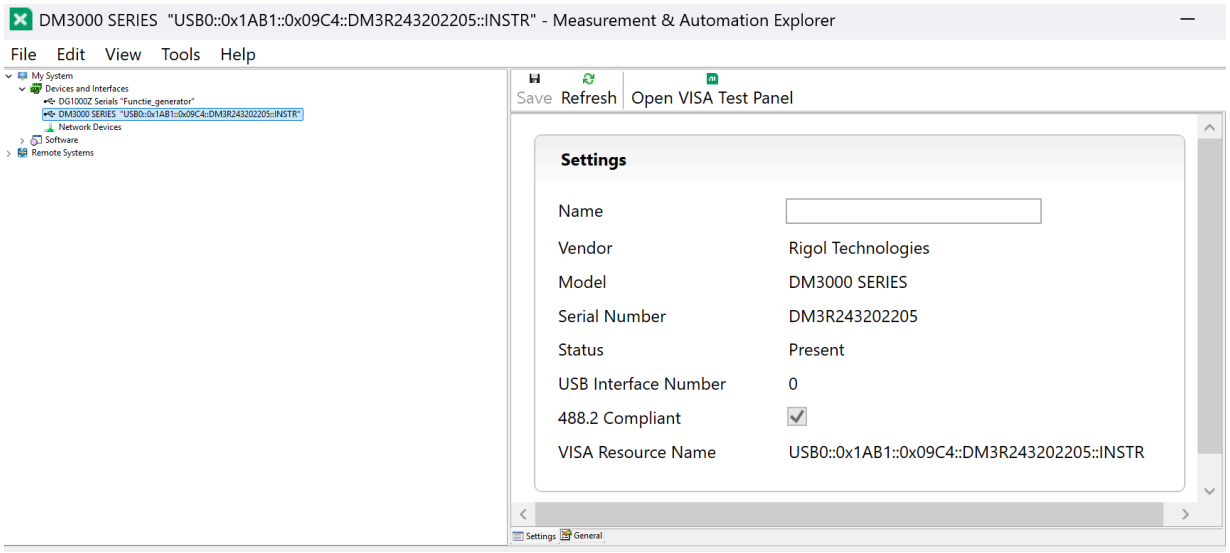
Het is aan te raden om te werken in een virtuele Python-omgeving en de benodigde packages via `pip` of `conda-forge` te installeren:

Anaconda Prompt: virtuele omgeving en installatie

```
# Maak een nieuwe virtuele omgeving via Anaconda Prompt of Navigator
(base) C:\Users\student> conda create -n Practicum2_3 python=3.11
(base) C:\Users\student> conda activate Practicum2_3

# Installeer PyVISA en optioneel de pure Python backend
(Practicum2_3) C:\Users\student> pip install pyvisa pyvisa-py numpy
               matplotlib scipy ipykernel
```

Appendix 2: Materialen en tips



Figuur 28: Startscherm van NI-Max software. Apparaat ID start met "USB".

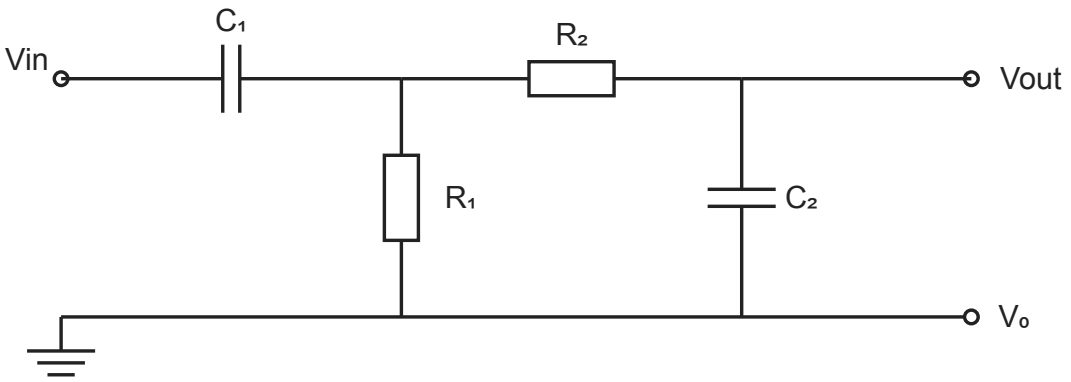
Tabel 6: Veelgebruikte SCPI-commando's voor de Rigol DG1022 / DG1022Z functiegenerator.

Doel	SCPI-commando
Selecteer kanaal	`SOUR1`, `SOUR2`
Golfvorm kiezen	`SOUR1:FUNC SIN`
Amplitude in Vpp	`SOUR1:VOLT <value>`
Amplitude in Vrms	`SOUR1:VOLT:UNIT VRMS`
Offset instellen	`SOUR1:VOLT:OFFS <value>`
Frequentie instellen	`SOUR1:FREQ <value>`
Fase instellen	`SOUR1:PHAS <deg>`
Output aan	`OUTP1 ON`
Output uit	`OUTP1 OFF`
Output laden instellen	`OUTP1:LOAD <ohm>`
Bestaande instellingen uitlezen	`SOUR1:VOLT?`, `SOUR1:FREQ?`
Burst-mode aan/uit	`SOUR1:BURS:STAT ON`
Modulatie AM/FM/PWM	`SOUR1:AM:STAT ON`, `SOUR1:FM:STAT ON`
Noise waveform	`SOUR1:FUNC NOIS`
Square-wave duty cycle	`SOUR1:PULS:DCYC <percent>`
Ramp symmetry	`SOUR1:FUNC:RAMP:SYMM <0–100>`
Systeem reset	`*RST`
Identificatie opvragen	`*IDN?`

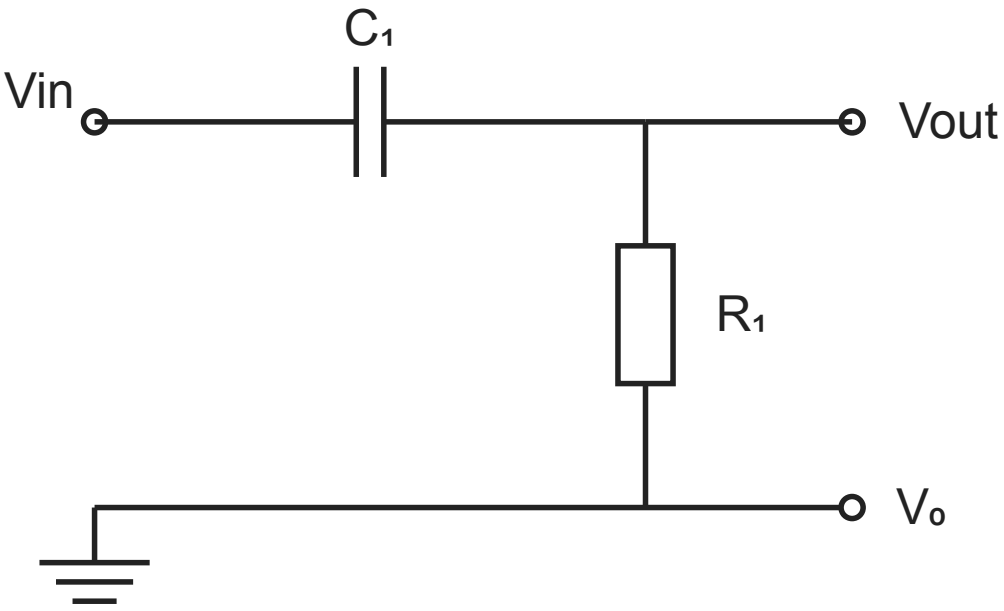
Tabel 7: Veelgebruikte SCPI-commando's voor de Rigol DM3058E digitale multimeter.

Doel	SCPI-commando
Identificatie opvragen	`*IDN?`
Reset instrument	`*RST`
Foutenbuffer leegmaken	`*CLS`
Stop lopende metingen	`ABORt`
AC-spanning meten (Vrms)	`CONF:VOLT:AC`
DC-spanning meten	`CONF:VOLT:DC`
Wisselspanning uitlezen (ad-hoc)	`MEAS:VOLT:AC?`
Gespacificeerde meting uitlezen	`READ?`
Frequentie meten	`CONF:FREQ`
Weerstand meten	`CONF:RES`
Stroom meten (AC/DC)	`CONF:CURR:AC`, `CONF:CURR:DC`
Triggerbron instellen	`TRIG:SOUR IMM`
Aantal metingen per trigger	`TRIG:COUN <n>`
Aantal samples instellen	`SAMP:COUN <n>`
Acquisitie starten	`INIT`
Buffer uitlezen	`FETCh?`
NPLC instellen (integratietijd)	`VOLT:AC:NPLC <x>`
Automatische bereikkeuze uit	`VOLT:AC:RANG:AUTO OFF`
Bereik instellen	`VOLT:AC:RANG <range>`
Auto-zero uit	`ZERO:AUTO OFF`
Detectieband AC	`VOLT:AC:DET:BAND 20 / 100`
Display aan/uit	`DISP:ENAB ON`, `DISP:ENAB OFF`
Status opvragen	`STAT:OPER:COND?`
Fouten opvragen	`SYST:ERR?`

Appendix 3: Schematische tekeningen schakelingen



Figuur 29: Schema van het banddoorlaatfilter (bandpass).



Figuur 30: Hoogdoorlaatfilter: C in serie, R naar massa; de uitgangsspanning wordt over R gemeten.

Appendix 4: apparaataansturing

Voorbeeld: functiegenerator aansturen met Python

```python
def set_fixed_sine(gen, freq=100, vrms=5.0, channel=1):
    """
    Stel de functiegenerator in op een vaste sinusgolf.
    Inclusief expliciet aan- en uitzetten van de uitgang.
    """

    chan = f"SOUR{channel}"

    gen.write(f"{chan}:FUNC SIN")           # sinusgolf kiezen
    gen.write(f"{chan}:VOLT:UNIT VRMS")     # amplitude in Vrms
    gen.write(f"{chan}:VOLT {vrms}")        # waarde instellen
    gen.write(f"{chan}:FREQ {freq}")        # frequentie instellen

    gen.write(f"OUTP{channel} ON")          # uitgang inschakelen

    # Optioneel: gegevens teruggeven, maar NIET verplicht:
    # return {"freq": freq, "vrms": vrms, "channel": channel}

# ------------------------------------------------------------
# Voorbeeldgebruik:
# ------------------------------------------------------------
import pyvisa

rm = pyvisa.ResourceManager()
gen = rm.open_resource("USB0::0x1AB1::0x0642::DG1ZAxxxxxxx::INSTR")

set_fixed_sine(gen, freq=100, vrms=5.0, channel=1)

gen.close()
```

Practica periode 4

Materiaalkunde en Stromingsleer

Practicum: Trekbankproef I

Leerdoelen

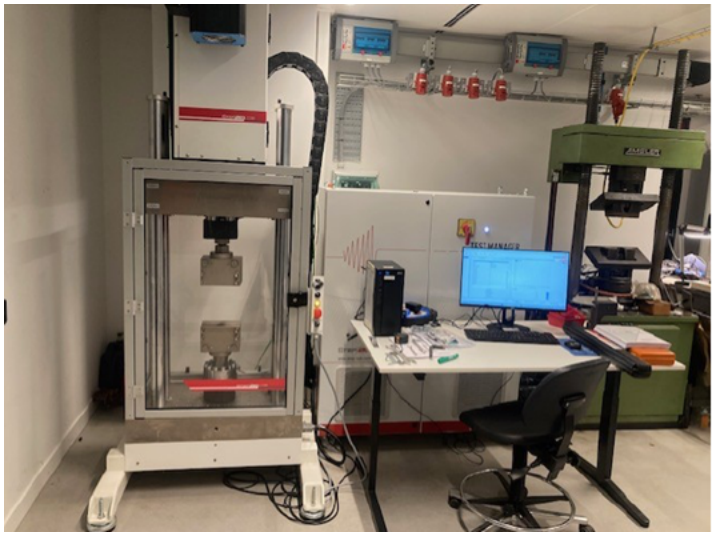
- ☐ Je kan een trekbankproef uitvoeren en nauwkeurig kracht en verlenging meten
- ☐ Je kan spannings-rekdiagrammen opstellen en interpreteren
- ☐ Je kan een foutanalyse uitvoeren en de onzekerheid in de elasticiteitsmodulus afleiden en bepalen

Veiligheid

1. Een trekbank werkt met hoge krachten en opgeslagen elastische energie. Bij breuk kan het monster plotseling losschieten of fragmenteren. Houd daarom altijd voldoende afstand en blijf buiten de directe lijn van het monster.
2. Raak het monster en de klemmen niet aan tijdens de meting. Wacht tot de machine volledig stilstaat voordat je iets aanpast of verwijdert.
3. We werken op een werktuigbouwkundige afdeling: volg altijd de instructies van Jan-Willem (en/of collega's) op met betrekking tot veilig werken.

Introductie

In dit practicum onderzoek je hoe een materiaal reageert op mechanische belasting door het gecontroleerd uit te rekken in een trekbankproef (zie Fig. 31). Terwijl de kracht wordt opgevoerd, doorloopt het materiaal verschillende regimes: eerst elastisch gedrag (omkeerbare vervorming), daarna plastische vervorming en uiteindelijk breuk.
Door de gemeten kracht en verlenging om te rekenen naar spanning en rek, stel je een spanning-rek-diagram op. Hieruit bepaal je fundamentele materiaaleigenschappen zoals de elasticiteitsmodulus, rekgrens en breukspanning. Deze grootheden beschrijven niet alleen hoe sterk een materiaal is, maar vooral hoe het zich gedraagt en faalt onder belasting.



Figuur 31: Een trekbank in het makerslab.

Voorbereidingsopdracht 1

1. Bekijk de video over de trekbankproef op Brightspace
2. Lees paragraaf 12.3 en 12.4 uit *OpenStax University Physics I*
3. Neem een USB-stick mee om de data uit de trekbank op te kunnen slaan.
4. Beantwoord in je labjournaal:
   - Leid de foutpropagatieformule voor E af
   - Welke meetfout heeft de grootste invloed op E?
   - Hoe bepaal je E met een fit, welke vergelijking heb je hiervoor nodig?

Theorie

Tijdens de trekproef worden de kracht F en de verlenging ΔL van het monster gemeten als functie van de aangelegde belasting. Deze grootheden worden omgerekend naar spanning en rek via:

$$\sigma = \frac{F}{A}, \qquad \varepsilon = \frac{\Delta L}{L_0} \tag{45}$$

Hierbij is A de oorspronkelijke doorsnede van het monster en L_0 de beginlengte. In het lineair-elastische regime geldt een lineair verband tussen spanning en rek (de wet van Hooke), waarbij de elasticiteitsmodulus E de helling van de spanning-rekcurve bepaalt:

$$E = \frac{\sigma}{\varepsilon} = \frac{F L_0}{A \Delta L} \tag{46}$$

In de praktijk wordt E bepaald uit een lineaire fit van het elastische gedeelte van het spanning-rekdiagram, omdat individuele metingen ruis en meetonzekerheden bevatten. Aangezien alle gemeten grootheden (F, ΔL, L_0 en A) onzekerheden bevatten, moet ook de onzekerheid in de elasticiteitsmodulus worden bepaald met behulp van foutenpropagatie. Hierbij is het belangrijk om te beseffen dat E relatief gevoelig is voor kleine fouten in ΔL, omdat deze grootheid in de noemer voorkomt.
De doorsnede A hangt af van de geometrie van het trekmonster. Twee veelgebruikte vormen zijn rechthoekige en cilindrische (ronde) monsters. Rechthoekige monsters zijn eenvoudig te vervaardigen en hebben een constante doorsnede, maar zijn gevoeliger voor spanningsconcentraties aan randen en hoeken. Dit kan leiden tot vroegtijdige lokale plastische vervorming en minder representatieve breukkarakteristieken.
Cilindrische monsters daarentegen hebben een axiaal-symmetrische geometrie en worden doorgaans nauwkeuriger machinaal bewerkt. Door deze symmetrie is de spanningsverdeling homogener, wat resulteert in betrouwbaardere metingen van materiaaleigenschappen zoals elasticiteitsmodulus, vloeigrens en breukspanning.

Practicum

Experiment: Trekproef

Tijdens het practicum wordt met pennen een trekproef uitgevoerd. Er wordt een kleine introductie over de pennen en de trekbank gegeven. Gegevens noteer je in je labjournaal en op een logformulier die je bij het practicum zult krijgen, zie Brightspace. Na de trekproef vullen jullie de resterende gevraagde gegevens in voor de pen die je zelf onderzoekt.
Let op: Het is belangrijk dat je een inschatting van de fout in de oppervlak via de diameter of rechthoek, de lengte, en de fout in de kracht bepaalt om de fout in de elasticiteitsmodulus te kunnen berekenen.

1. Voer een trekproef uit op het materiaal
2. Meet kracht en verlenging tijdens de proef
3. Noteer alle gegevens in je labjournaal of logformulier
4. Bepaal en noteer schattingen van de meetfouten in:
   - kracht F
   - lengte L_0
   - verlenging ΔL
   - oppervlak A

Verwerking

1. Zet de kracht en uitrekking om naar spanning en rek.
2. Plot de spanning-rekdiagram.
3. Bepaal de elasticiteitsmodulus in het elastische gebied, welke eenheid hoort hierbij?

4 Bepaal de fout in E via de lineaire fit in het elastische gebied.
5 Stel een tabel op met de waarden, zie logformulier:
   - maximale trekspanning
   - elasticiteitsgrens
   - breukspanning
   - breukrek
   - insnoering

Resultaten en Discussie

- Welk materiaal heb je gemeten? Motiveer je antwoord en houd rekening met de onzekerheid op E.
- Zijn er systematische afwijkingen aanwezig? Was de correctie via de lineaire fit nodig?

Afronding

1 Verwerk alle figuren en resultaten in je labjournaal.
2 Schrijf een duidelijke resultaat- en discussiesectie.
3 Laat je werk controleren door de begeleider.

Practicum: Trekbankproef II

Leerdoelen

- ☐ Je kan een trekbankproef uitvoeren en nauwkeurig kracht en verlenging meten
- ☐ Je kan spannings-rekdiagrammen opstellen en interpreteren
- ☐ Je kunt energie-dissipatie tijdens een compressie experiment.

Veiligheid

1. Een trekbank werkt met hoge krachten en opgeslagen elastische energie. Bij breuk kan het monster plotseling losschieten of fragmenteren. Houd daarom altijd voldoende afstand en blijf buiten de directe lijn van het monster.
2. Raak het monster en de klemmen niet aan tijdens de meting. Wacht tot de machine volledig stilstaat voordat je iets aanpast of verwijdert.
3. We werken op een werktuigbouwkundige afdeling: volg altijd de instructies van Jan-Willem (en/of collega's) op met betrekking tot veilig werken.

Introductie

In dit vervolgexperiment onderzoek je het gedrag van visco-elastische materialen, zoals rubber, onder compressie. In tegenstelling tot metalen, die in eerste benadering elastisch en energiebehoudend gedrag vertonen, combineren deze materialen elastische respons met dissipatieve processen. Dit betekent dat een deel van de mechanische energie tijdens vervorming wordt omgezet in warmte. Wanneer een rubberen monster herhaaldelijk wordt samengedrukt en ontlast, volgt het kracht-verplaatsings diagram niet hetzelfde pad bij belasten en ontlasten. Er ontstaat een hysterese-lus, waarvan de oppervlakte direct overeenkomt met het energieverlies per cyclus. Dit energieverlies is een macroscopische manifestatie van interne processen, zoals ketenbewegingen en relaxatie in het polymeernetwerk.

De mate van dissipatie hangt sterk af van factoren zoals vervormingssnelheid, temperatuur en de interne structuur van het materiaal. Bij hogere belastingssnelheden of grotere vervormingen wordt doorgaans meer energie gedissipeerd.

Een herkenbaar voorbeeld hiervan is het opwarmen van autobanden tijdens het rijden. Terwijl de band continu vervormt bij contact met het wegdek, doorloopt het rubber steeds cycli van belasting en ontlasting. Door de hysterese in het materiaal gaat hierbij energie verloren, die wordt omgezet in warmte. Dit verklaart waarom banden warm worden tijdens gebruik en waarom materiaalkeuze een belangrijke rol speelt in rolweerstand, slijtage en energie-efficiëntie.

De compressieproef in dit practicum maakt het mogelijk om deze dissipatieve eigenschappen kwantitatief te karakteriseren via de oppervlakte van de hysterese-lus en zo inzicht te krijgen in het visco-elastisch gedrag van het materiaal.

Voorbereidingsopdracht 1

1. Neem een USB-stick mee om de data uit de trekbank op te kunnen slaan.
2. Maak een schets de elasticiteit-modulus tegenover de tijd voor een viscoelastisch materiaal met twee tijdschalen (vergelijking 8, met i = 1,2). Je mag hiervoor zelf de waardes kiezen van E_i en τ_i. Zet deze schets in jouw labjournaal. Gebruik hiervoor pen en papier óf Desmos Link.
3. Hoe haal ik uit deze schets de waarde van E_∞? Geef dit aan in je schets.
4. Hoe bepaal je E_∞ met een fit, welke vergelijking heb je hiervoor nodig?

Theorie

Tijdens de compressie meet je de kracht F en de indrukking ΔL van het rubbermonster. Analoog aan de trekproef definieer je de spanning en rek als:

$$\sigma = \frac{F}{A}, \qquad \varepsilon = \frac{\Delta L}{L_0} \tag{47}$$

Hierbij is A het contactoppervlak van het monster en L_0 de oorspronkelijke hoogte. Voor een ideaal elastisch materiaal zou de spanning op elk moment alleen afhangen van de actuele rek. In dat geval is er een eenduidig verband tussen σ en ε, en lopen belasten en ontlasten over hetzelfde pad.

Bij rubbers en andere visco-elastische materialen is dat anders. Daar hangt de spanning niet alleen af van de huidige vervorming, maar ook van de eerdere belastinggeschiedenis en van de snelheid waarmee het materiaal wordt belast. Een deel van de mechanische energie wordt tijdelijk opgeslagen, terwijl een ander deel wordt gedissipeerd. Daardoor vallen het belastingspad en ontlastingspad niet samen en ontstaat een hysterese-lus in het spanning-rekdiagram.

De energie die per cyclus verloren gaat is gegeven door:

$$E_{\text{verlies}} = \oint \sigma \, d\varepsilon \tag{48}$$

Deze integraal is gelijk aan de oppervlakte binnen de hysterese-lus. Hoe groter deze oppervlakte, hoe meer energie per cyclus als warmte in het materiaal wordt omgezet.

Maxwell-model In het Maxwell-model is de totale rek de som van de rek in de veer en de demper:

$$\varepsilon = \varepsilon_{\text{veer}} + \varepsilon_{\text{demper}} \tag{49}$$

Voor de afzonderlijke elementen geldt:

$$\sigma = E\varepsilon_{\text{veer}}, \qquad \sigma = \eta \frac{d\varepsilon_{\text{demper}}}{dt} \tag{50}$$

Combineren van deze relaties leidt tot de constitutieve vergelijking van het Maxwell-model:

$$\frac{d\varepsilon}{dt} = \frac{1}{E}\frac{d\sigma}{dt} + \frac{\sigma}{\eta} \tag{51}$$

Hierin is E de elastische modulus van de veer en η de viscositeit van de demper.

Spanningsrelaxatie

Beschouw een experiment waarbij het materiaal plots op een constante rek ε_0 wordt gebracht en vervolgens wordt vastgehouden:

$$\varepsilon(t) = \varepsilon_0 \tag{52}$$

Bij een visco-elastisch materiaal zal de spanning in de tijd afnemen: het materiaal “ontspant”. In het eenvoudige Maxwell-model gebeurt dit exponentieel:

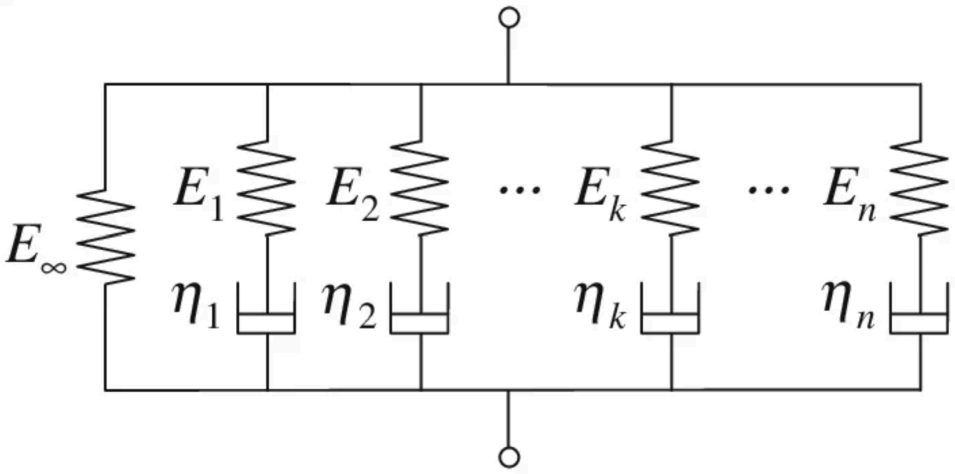
$$\sigma(t) = \sigma_0 e^{-t/\tau}, \qquad \tau = \frac{\eta}{E} \tag{53}$$

Dit model beschrijft dat de spanning uiteindelijk volledig verdwijnt. Het materiaal gedraagt zich op lange tijdschalen dus vloeistofachtig.

Voor veel echte materialen, zoals rubbers, is dit echter te simpel. Daar blijft vaak een deel van de spanning behouden door een elastisch netwerk in het materiaal. In dat geval wordt het gedrag beter beschreven door een gegeneraliseerd Maxwell-model, zie Fig. 32:

$$E(t) = \frac{\sigma(t)}{\varepsilon_0} = \left(E_\infty + \sum_i E_i e^{-t/\tau_i} \right) \tag{54}$$

Hierin is E_∞ de blijvende (lange-tijd) elasticiteitsmodulus, en beschrijven de exponentiële termen relaxatieprocessen op verschillende tijdschalen τ_i.



Figuur 32: Generalized Maxwell-model.

Hieruit volgt dat het materiaal op korte tijdschalen stijver is (hoge effectieve modulus), terwijl op langere tijdschalen relaxatie optreedt en de modulus afneemt naar een constante waarde E_∞.
Dit tijdsafhankelijke gedrag ligt ten grondslag aan hysterese en energieverlies bij cyclische belasting.

Interpretatie voor het experiment

Bij cyclische belasting, zoals in dit practicum, wordt het materiaal voortdurend belast en ontlast. Door de tijdsafhankelijke respons ontstaat een faseverschil tussen spanning en rek, wat resulteert in een hysterese-lus in het spanning-rekdiagram. De oppervlakte van deze lus komt overeen met het energieverlies per cyclus:

$$E_{\text{verlies}} = \oint \sigma \, d\varepsilon \tag{55}$$

Dit energieverlies is een directe maat voor de dissipatieve eigenschappen van het materiaal en hangt samen met de interne relaxatieprocessen zoals beschreven door het Maxwell-model. Het is dus niet een constant energieverlies, maar hangt af van de snelheid van o.a. (de)compressie.

Practicum

Experiment 1: relaxatie rubber

1. Bepaal de elasticiteitsmodulus voor Shore A 10 & 30 samples.
2. Analyseer de uniforme rek ε_0 data. Hoeveel tijdschalen zie je? Welke waarde van E_∞ heb je gevonden?

Experiment 2: energieverlies per cyclus

1. Plaats een rubber monster tussen de drukplaten.
2. Voer meerder laad-en ontlaadcycli uit.
3. Meet de kracht en verplaatsing continu.
4. Varieer eventueel de maximale compressie en snelheid van de belasting.

Verwerking experiment 1

1. Maak een spanning-tijd diagram.
2. Bepaal E_∞ en onderzoek hoeveel tijdschalen je nodig hebt om deze rubber goed te kunnen modelleren.
3. Noteer het aantal tijdschalen, de tijdschalen τ_i (in de juiste eenheid) en de visco-elasticiteit per tijdsschaal E_i.

Verwerking experiment 2

1. Zet kracht en verplaatsing om naar spanning en rek
2. Plot het spanning-rekdiagram voor meerdere cycli
3. Bepaal de oppervlakte van de hysterese-lus (numeriek)
4. Vergelijk energieverlies tussen verschillende cycli
5. Onderzoek het effect van belastingssnelheid (indien gemeten)

Resultaten en Discussie

- Hoe groot is het energieverlies per cyclus?
- Hoe verandert de hysterese bij herhaald belasten?
- Wat is het effect van de belastingssnelheid?
- Verklaar het energieverlies fysisch (denk aan interne wrijving / relaxatieprocessen)

Afronding

1. Verwerk alle figuren en resultaten in je labjournaal.
2. Schrijf een duidelijke resultaat- en discussiesectie.
3. Laat je werk controleren door de begeleider.

Practicum: Hardheid en Kerfslag

Leerdoelen

- ☐ Je kan een hardheidsmeting uitvoeren
- ☐ Je kan verschillende soorten hardheidsmetingen onderscheiden
- ☐ Je kan een kerfslagmeting uitvoeren
- ☐ Je kan de invloed van temperatuur op materiaalgedrag bepalen en beoordelen

Veiligheid

1 We werken op een werktuigbouwkundige afdeling vandaag, volg de instructies van Jan-Willem (en/of collega's) op met betrekking tot veilig werken.

Introductie

Tijdens dit practicum maak je kennis met twee belangrijke materiaal-karakterisatietechnieken (zie Fig. 33): de hardheidsproef en de kerfslagproef.
De hardheidsproef meet de weerstand van een materiaal tegen lokale plastische vervorming. Deze eigenschap hangt samen met fundamentele materiaaleigenschappen zoals de treksterkte, elasticiteitsgrens en elasticiteitsmodulus. In dit practicum bepaal je de hardheid van gelaste staalmonsters met behulp van micro- en macro-Vickers en Rockwell metingen. Hierbij wordt ook het verschil tussen basismateriaal, las en warmte-beïnvloede zone (HAZ) onderzocht.
De kerfslagproef richt zich op een andere mechanische eigenschap, namelijk de taaiheid: het vermogen van een materiaal om energie te absorberen vóór breuk. Deze proef geeft inzicht in hoe een materiaal reageert op dynamische belasting (schokken of stoten). Volgens de ISO 148-1 standaard bepaal je de opgenomen energie bij breuk voor verschillende temperaturen, waarmee de overgang van taai naar bros gedrag (ductile-to-brittle transition) zichtbaar wordt.



Setup voor de kerfslagproef



Setup voor de micro Vickers proef

Figuur 33: Overzicht van de gebruikte meetopstellingen

Voorbereidingsopdracht

1. Bekijk de kennisclips op Brightspace
2. Bekijk de volgende video: Link 1
3. Beantwoord in je labjournaal:
   - Geef de formule voor de micro Vickers hardheid
   - Leid een formule af voor de fout in de hardheid
   - Welke fout (kracht of diameter) domineert?
   - Hoe herken je de heat affected zone (HAZ) in een las?
   - Hoe herken je een geslepen oppervlak bij breuk?

Theorie

Hardheid

Van verschillende materialen (onder andere staalsoorten) zijn de fabricage-hardheden bekend. De hardheid is belangrijk indien eigenschappen als slijtvastheid en vervorming van het oppervlak belangrijk zijn voor de gebruikte toepassing van het materiaal. De hardheid van een staal kan veranderen door een bewerkingsproces zoals: lassen en/of een warmtebehandeling.

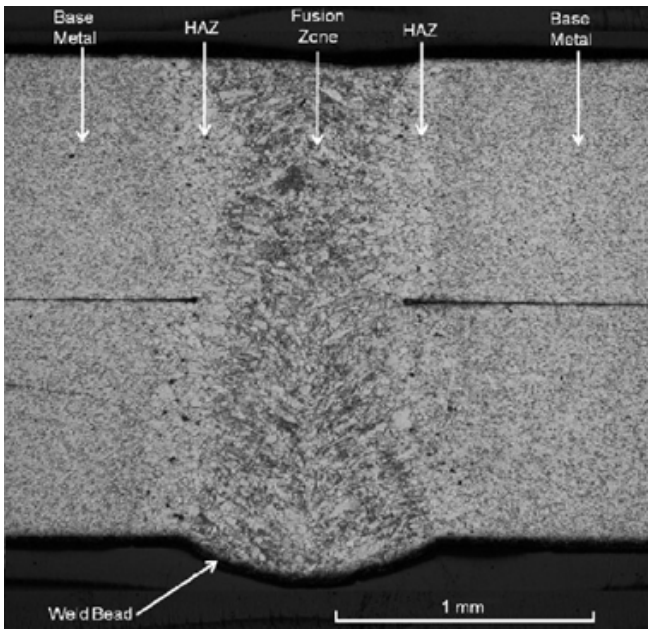
De macro Vickers hardheidsmeter heeft een grotere indrukking en geeft daardoor een gemiddelde hardheid van het materiaal weer. Met de micro Vickers wordt op specifieke plaatsen zeer kleine indrukkingen gemaakt waardoor het verloop van hardheid in het materiaal duidelijk zichtbaar kan worden gemaakt. Je hebt in de reader gelezen dat de macro Vickers' hardheid te berekenen is met:

$$H = 0,189\frac{F}{d^2} \tag{56}$$

waarbij F de kracht is en d de gemiddelde indrukdiameter. De fout in de hardheid volgt uit de foutenpropagatie.

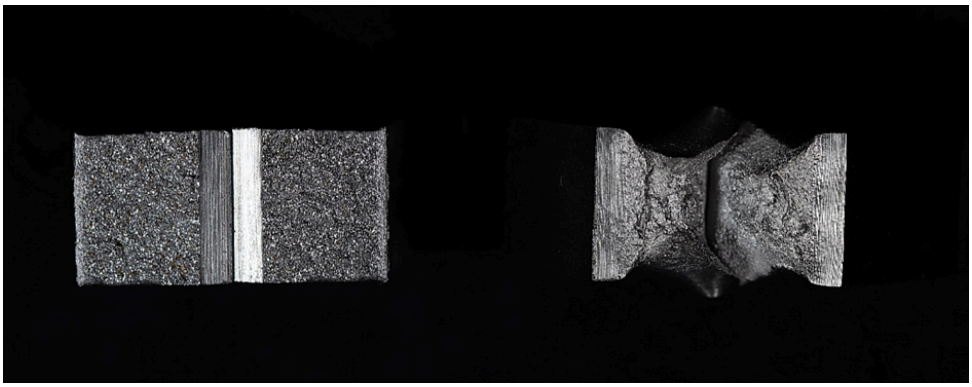
Lassen

Lassen is een verbindingstechniek waarbij twee basismaterialen met elkaar verbonden worden door middel van smelten en het toevoegen van materiaal. Dit betekent dat er plaatselijk energie moet worden toegevoegd om het staal vloeibaar te maken. Een veel gebruikte methode is elektrisch booglassen. Door een hoge stroom wordt het materiaal verhit tot het smelt en hierdoor smelt ook het toevoegmateriaal (elektrode). Dit toevoegmateriaal wordt zo geselecteerd dat het nagenoeg uit hetzelfde materiaal bestaat als de te lassen basismaterialen. Het basismateriaal heeft bepaalde eigenschappen die ontstaan vanuit de microstructuur van het staal. Deze microstructuur kun je zichtbaar maken met behulp van een microscoop (bij vergroting van 200x-1500x). Koolstof komt in het basismateriaal voor in de vorm van vrije koolstofatomen of gebonden als carbide (Fe_3C). Door het staal opnieuw te verhitten zal koolstof diffunderen (verplaatsen) naar de plaats van de plek van de hogere temperatuur. Dit betekent praktisch dat er in de overgang tussen het basismateriaal en de las een overgangszone ontstaat waar de koolstof aan het materiaal wordt onttrokken doordat deze diffundeert naar het gedeelte waar de samensmelting van de materialen plaatsvindt. Deze overgangszone noemt men de warmte beïnvloedde zone of Heat Affected Zone (HAZ) (34). Aangezien koolstof vrijwel de belangrijkste factor is voor het verkrijgen van mechanische eigenschappen van het staal, zal het gelaste materiaal dus plaatselijk verschillen in mechanische eigenschappen.



Figuur 34: Voorbeelde doorsnede las met HAZ.

Kerfslag



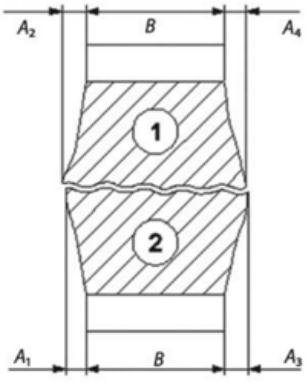
Figuur 35: Twee verschillende typen breuken van preparaten in de kerfslagproef.

Een van de eigenschappen om materialen te beschrijven is de taaiheid. De taaiheid komt duidelijk naar voren in de kerfslagwaarde. Hoe hoger de kerfslagwaarde, hoe taaier een materiaal zich zal gedragen, Fig. 35 illustreert twee verschillende typen breuken. Staal zal bij lagere temperaturen een meer brosse breuk vertonen. De overgang van taai naar bros gedrag wordt de ductile-to-brittle transitietemperatuur genoemd en ligt voor staal ergens tussen de –40 °C tot 20 °C. Als bij meerdere temperaturen proeven worden uitgevoerd kan een grafiek worden gemaakt die de overgang van bros naar taai gedrag weergeeft, afhankelijk van de temperatuur. Naast de kerfslagwaarde, kun je van elke proefstaaf de vervorming van de proefstaven en de breukvlakken beoordelen, zoals weergegeven in Figuur 36 in de norm ISO 148. Het gaat hierbij om de zijdelingse, laterale vervorming (taaiheid) van de vierkante staaf en een beoordeling van het breukvlak op taai en bros uiterlijk. Je kan dit vergelijken met hetzelfde materiaal maar dan bij een andere temperatuur, zo zie je de invloed van temperatuur op de taaiheid van het materiaal. De hoeveelheid vervorming zegt iets over de taaiheid van het materiaal op bijbehorende temperatuur.

Je meet de afstanden A_1, A_2, A_3 en A_4 van de proefstaaf met een meetklok of hoogteschuifmaat. De laterale vervorming wordt vervolgens bepaald als som van de tegenoverliggende meetpunten:

$$\delta_1 = A_1 + A_3, \qquad \delta_2 = A_2 + A_4 \tag{57}$$

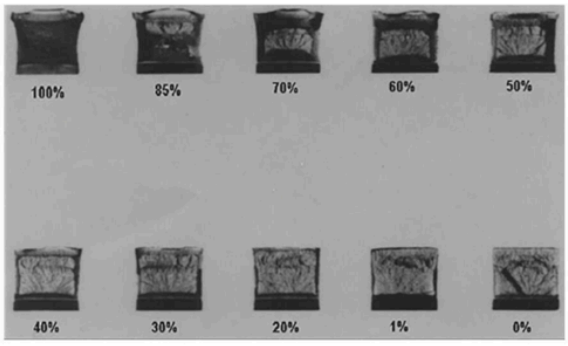
Hierbij representeren δ_1 en δ_2 de vervorming in twee loodrechte richtingen van de doorsnede van de proefstaaf.



Key

1	side one of fractured test piece
2	side two of fractured test piece
B	thickness of test piece, mm
A_1, A_2, A_3, A_4	distance measured, mm

Beoordeling vervorming



a) Fracture appearance charts and per cent shear fracture comparator



Beoordeling percentage breuk

Figuur 36: Beoordeling van vervorming en breukgedrag

De bepaling van het percentage taaie breuk oppervlak kan door het oppervlak te bekijken en te vergelijken met de afbeelding in Fig. 36b. Een brosse breuk geeft een helder/glimmend en kristallijn uiterlijk en een taaie breuk een dof en vezelachtig uiterlijk. Op basis hiervan doet men een uitspraak over de hoeveelheid percentage taaie breuk in de doorsnede van de gebroken kerfslagstaaf.

Practicum

Experiment 1: Hardheidsmetingen

Je gaat het verloop van hardheid vanuit het basismateriaal tot in de las bepalen. Hiervoor worden er minimaal 1 hardheidsmeting per gebied (basismateriaal, HAZ en las) uitgevoerd met de micro Vickers en 3-hardheidsmetingen per gebied met macro Vickers. Het aantal bij de micro Vickers is lager in verband met de complexiteit van de meting. Noteer de gemeten waarden en opmerkingen in een registratie tabel in het labjournaal.
Let op: Het is belangrijk dat je een inschatting van de fout in de diameter en de fout in de kracht bepaalt om de fout in de hardheid te kunnen berekenen.

1. Voer 2–3 hardheidsmetingen uit voor:
   - Basismateriaal (macro Vickers)
   - HAZ (macro Vickers)
   - Las (macro Vickers)
2. Voer 1–2 micro Vickers metingen uit in dezelfde gebieden
3. Meet een referentiesample met macro Vickers én Rockwell (zo kunnen we deze schalen met elkaar vergelijken)
4. Noteer alle meetwaarden plus meetfouten in jouw labjournaal.

Experiment 2: Kerfslagproef

Tijdens het practicum worden door elk team proefstaafjes bij verschillende temperaturen onderworpen aan de kerfslagproef. Er wordt een kleine instructie over het gebruik van de valhamer gegeven, waarna de staven zelfstandig worden getest. De proef wordt uitgevoerd met proefstaven volgens, in overeenstemming met ISO 148: L=55 mm, vierkant 10 mm, Kerf 3 mm. Deze norm is iets anders dan in de reader materiaalkunde staat.

1. Voer een nulmeting uit (zonder proefstaaf) om te bepalen hoeveel wrijving aanwezig is.
2. Bepaal de kerfslagwaarde voor het materiaal, compenseer voor de aanwezig wrijving.
3. Herhaal de kerfslagproef voor verschillende temperaturen.
4. Meet laterale vervorming en bepaal percentage taaie breuk.

Verwerking

1. Bepaal de fout in de macro Vickers' hardheid op twee verschillende manieren:
   - De algemene formule voor de foutenpropagatie.
   - De standaarddeviatie tussen de verschillen metingen.
2. Plot het verloop van de hardheid van de lasnaad voor de micro en macro Vickers' hardheid.
3. Maak voor de kerfslagproef een registratietabel, waarin je de kgm-waarde, hoeveelheid Joule, de laterale vervorming en taai/bros percentage noteert bij de verschillende temperaturen. Noteer ook de gegevens van de andere teams die de andere materialen hebben bepaald.

Resultaten & Discussie

Hardheid

- Welke van de twee fouten (doormiddel van foutenpropagatie of de standaardeviatie) is hoger? Leg uit waar dit verschil door ontstaat.
- Waar is de hardheid het hoogst in de lasnaad?
- Wat valt je op als je de micro Vickers en de macro Vickers verloop van de lasnaad bekijkt?
- Leg uit of er bij je metingen sprake is van systematische afwijking(en) en hoe dit je metingen beïnvloed.
- Verklaar eventuele verschillen tussen de gemeten resultaten.

Kerfslag

- Hoe verandert de taaiheid bij verschillende temperaturen?
- Maak een schatting van de ductile-to-brittle transition temperatuur van het preparaat.
- Leg uit of er bij je metingen sprake is van systematische afwijking(en) en hoe dit je metingen beïnvloed.
- Verklaar eventuele verschillen tussen de gemeten resultaten.

Afronding

1. Zet alle figuren in je labjournaal met duidelijke onderschriften
2. Schrijf een resultaat en discussie sectie met onzekerheden
3. Laat je werk controleren door de begeleider

Practicum: Viscositeit met de Ubbelohde viscometer

Leerdoelen

- ☐ Je kan de kinematische en dynamische viscositeit meten met een Ubbelohde viscosimeter.
- ☐ Je kan een foutanalyse uitvoeren met behulp van foutenpropagatie.
- ☐ Je kan de dynamische viscositeit van mengsels experimenteel bepalen.

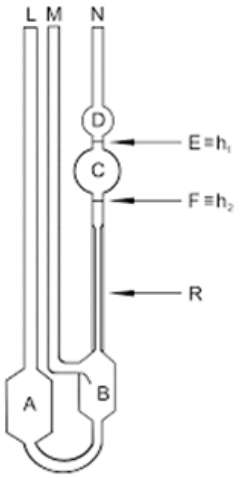
Veiligheid

1. De Ubbelohde viscometer is gemaakt van dun glas en zeer breekbaar. Ga hier voorzichtig mee om.
2. Werk zorgvuldig met vloeistoffen en voorkom morsen. Ruim gemorste vloeistoffen direct op.
3. Gebruik de viscometer alleen volgens de instructies van de begeleider.

Introductie

In dit practicum onderzoek je de viscositeit van vloeistoffen met behulp van een Ubbelohde viscometer. Dit is een klassieke meetmethode waarbij de stroming van een vloeistof door een dunne capillair wordt gebruikt om de viscositeit te bepalen.

Hoewel moderne apparatuur vaak gebruik maakt van roterende systemen, biedt de Ubbelohde viscometer een eenvoudige en nauwkeurige manier om viscositeit te meten op basis van stromingstijd, Link. In dit practicum bepaal je de dynamische viscositeit van water en vergelijk je verschillende methoden om de meetnauwkeurigheid te evalueren.



Figuur 37: Een Ubbelohode viscometer.

In het tweede deel van het practicum onderzoek je mengsels van vloeistoffen en bestudeer je hoe moleculaire interacties de viscositeit beïnvloeden.

Voorbereidingsopdracht 1

1. Bekijk de video over het gebruik van de Ubbelohde viscometer.
2. Lees de handleiding van de UBC-2 viscometer op Brightspace.
3. Noteer:
   - de dynamische en kinematische viscositeit van water.
   - de bijbehorende eenheden en dimensies.
4. Stel een foutpropagatieformule op voor μ.
5. Zoek viscositeiten van water en glycerol op.
6. Bereken x_1 en x_2 voor verschillende mengverhoudingen.
7. Bedenk een strategie om G_{12} te bepalen.

Theorie

Kinematische viscositeit van een vloeistof

De tijd t die het vloeistofniveau nodig heeft om te dalen van positie E naar positie F heeft een direct verband met de zogenaamde kinematische viscositeit ν:

$$\nu = kt \tag{58}$$

Hierin is k de kalibratieconstante van de specifieke Ubbelohde viscometer. Deze moet dus altijd gekalibreerd worden met een vloeistof waarvan de (kinematische) viscositeit bekend is.

Je kunt je waarschijnlijk voorstellen dat de dichtheid van invloed is op hoe snel een vloeistof als gevolg van de zwaartekracht door een buis naar beneden stroomt. De kinematische viscositeit μ is dan ook niks anders dan de dynamische viscositeit gedeeld door de dichtheid van de vloeistof:

$$\nu = \frac{\mu}{\rho}. \tag{59}$$

Mengels van oplossingen

Voor mengsels (Engels: mixtures) van vloeistoffen kan de dynamische viscositeit ook bepaald worden. Een eerste-orde benadering hiervoor is de massa-gewogen som van de individuele viscositeiten:

$$\mu_{\text{mix}} = \sum_i w_i \mu_i = (1-w)\,\mu_1 + w\,\mu_2 \tag{3}$$

Hierbij is w_i de massafractie van vloeistof i met dynamische viscositeit μ_i. Voor twee vloeistoffen kan dit vereenvoudigd worden door te schrijven in termen van het massapercentage w van het mengsel (bijvoorbeeld: $w = 0.1$ voor 10% glycerol in water). Echter, deze benadering werkt in het algemeen niet goed.

De reden hiervoor is dat de individuele moleculen vaak complexe interacties met elkaar aangaan (sterke intermoleculaire interacties, zoals waterstofbruggen, zwavelbruggen, etc.). Voor mengsels (bijvoorbeeld water–glycerol) kan een betere benadering worden gemaakt door te corrigeren voor het molecuulgewicht M_i en door een interactieparameter te introduceren.

De eerste correctie betreft het molecuulgewicht van de individuele vloeistoffen. Voor water geldt $M_w = 18.015\ g/mol$ en voor glycerol $M_g = 92.093\ g/mol$. De molfracties worden dan bepaald via:

$$n_1 = \frac{1-w}{M_w}, \qquad n_2 = \frac{w}{M_g} \tag{4}$$

Vervolgens definiëren we de genormaliseerde fracties:

$$x_1 = \frac{n_1}{n_1 + n_2}, \qquad x_2 = 1 - x_1 \tag{5}$$

De viscositeit kan vervolgens worden bepaald met de Grunberg–Nissan-formule:

$$\ln \mu_{\text{mix}} = x_1 \ln \mu_w + x_2 \ln \mu_g + x_1 x_2 G_{12} \tag{6}$$

Hierbij is μ_w de viscositeit van water, μ_g de viscositeit van glycerol en G_{12} een maat voor de interactiesterkte tussen de moleculen.

Practicum

Experiment deel I: Water

In dit practicum wordt gewerkt met een UBC-2 Ubbelohde Calibrated viscometer. Let op: Ubbelohde-viscometers zijn fragiel en kostbaar, ga hier zorgvuldig mee om.

De onderstaande methode is bewust beknopt geformuleerd. Van jullie wordt verwacht dat je zelfstandig een geschikte meetstrategie opstelt en onderbouwt.

1. Meet de stromingstijd van water minimaal 20 keer en bepaal de kinematische viscositeit ν voor elke meting.

2 Maak een schatting van de meetfouten. Identificeer de dominante foutbron (bijvoorbeeld timing, temperatuur, aflezen).
3 Bepaal de dynamische viscositeit μ van water en bereken de bijbehorende onzekerheid via foutenpropagatie.
4 Spoel de Ubbelohde-viscometer zorgvuldig schoon na afloop van de metingen.

Verwerking deel I

1 Maak een histogram van de gemeten waarden van μ. Kies een passende bin-grootte en bepaal:
   - het gemiddelde van μ
   - de fout op het gemiddelde
2 Onderzoek of de metingen normaal verdeeld zijn, bijvoorbeeld door een Gaussiaanse fit op het histogram toe te passen.
3 Vergelijk de spreiding tussen individuele metingen met de geschatte meetfouten. Welke bijdrage domineert?
4 Noteer de eindwaarde van de dynamische viscositeit van water inclusief onzekerheid.

Experiment deel II: Mengsels

Iedere groep bepaalt de dynamische viscositeit van één mengsel (mixture). De verzamelde data wordt klassikaal gecombineerd voor verdere analyse.

1 Meet de kinematische viscositeit ν van jullie mengsel. Herhaal de meting totdat je een betrouwbare schatting hebt van de gemiddelde waarde en de spreiding.
2 Bepaal de dynamische viscositeit μ van het mengsel en bereken de bijbehorende onzekerheid.
3 Noteer jullie waarde van μ (inclusief onzekerheid) op het bord, zodat deze beschikbaar is voor de gehele groep.
4 Verzamel de viscositeitsgegevens van de andere groepen.
5 Spoel de Ubbelohde-viscometer zorgvuldig schoon na afloop van de metingen.

Verwerking deel II

1 Bepaal de interactieparameter G_{12} (water–glycerol) met bijbehorende onzekerheid, gebruikmakend van de gecombineerde dataset.
2 Leg uit of er bij je metingen sprake is van systematische afwijking(en) en hoe dit je metingen beïnvloed.
3 Verklaar jouw gevonden waarde voor de interactiesterkte met de literatuurwaarde.

Afronding

1 Werk alle resultaten uit in je labjournaal
2 Laat je werk controleren door de begeleider.

Practicum: Digitale viscosimeter en niet-Newtoniaanse vloeistoffen

Leerdoelen

- ☐ Je kan de dynamische viscositeit meten met een digitale viscosimeter.
- ☐ Je kan het verschil tussen Newtoniaanse en niet-Newtoniaanse vloeistoffen bepalen.

Veiligheid

1. De rotor (spindle) en koppeling van de viscosimeter zijn fragiel. Ga hier voorzichtig mee om.
2. Zorg dat de rotor correct gemonteerd is voordat je de meting start.
3. Werk zorgvuldig met vloeistoffen en voorkom morsen.

Introductie

In het vorige practicum heb je de viscositeit bepaald met een Ubbelohde viscometer. In dit practicum gebruik je een digitale viscosimeter, zie Fig. 38, waarmee de viscositeit direct wordt bepaald via een koppelmeting.



Figuur 38: Een digitale viscometer

Het doel van dit practicum is tweeledig:

- het vergelijken van de precisie van beide meetmethodes,
- het bepalen van het vloeistofgedrag van een glycerine-oplossing en een zetmeeloplossing.

Voorbereidingsopdracht

Experiment 1:

1. Bekijk de video over het gebruik van de digitale viscosimeter Link.
2. Bepaal met behulp van de handleiding de juiste instellingen en rotor.
3. Schrijf een meetstrategie uit in je labjournaal.
4. Neem de data van het vorige practicum over in Python.

Experiment 2:

1. Bepaal de waarde van n voor verschillende vloeistoftypes.
2. Voer een dimensie-analyse uit voor k.
3. Zoek het verwachte gedrag van een maizena-oplossing.

Theorie

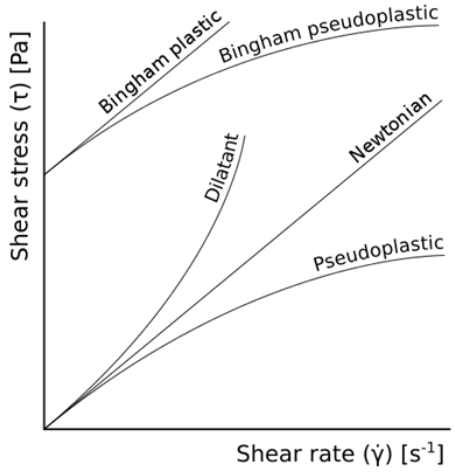
De digitale viscosimeter die we in dit practicum gebruiken werkt anders dan de Ubbelohde-viscometer. De digitale viscosimeter maakt gebruik van een zogeheten spil (Engels: *spindle*), een opzetstuk waarmee de vloeistof in beweging wordt gebracht. Hierbij wordt het koppel M (Engels: *torque*) gemeten. De viscositeit volgt dan uit:

$$\eta = \frac{M}{K} \tag{1}$$

Let op: voor Newtoniaanse vloeistoffen wordt de dynamische viscositeit vaak aangeduid met μ, terwijl voor niet-Newtoniaanse vloeistoffen η wordt gebruikt. In dat geval is de viscositeit afhankelijk van de shear-rate $\dot{\gamma}$ en wordt dit geschreven als $\eta(\dot{\gamma})$.
Hierbij is η de dynamische viscositeit, M het koppel en K de kalibratieconstante van de viscosimeter. Deze kalibratieconstante is afhankelijk van de geometrie (bijvoorbeeld cilindrisch) en de rotatiesnelheid van de spil.
Voor een cilindrische geometrie kan worden afgeleid dat:

$$\eta = \frac{M}{2\pi\left(\frac{R_i^3 L\omega}{R_0 - R_i}\right)} \tag{2}$$

Hierbij is R_i de straal van de binnenste cilinder, R_0 de straal van de buitenste cilinder (spil), L de dompelhoogte van de spil en ω de hoeksnelheid.
De viscosimeter gebruikt een koppelmeter om het moment M te bepalen, waarna de dynamische viscositeit wordt berekend.



Figuur 39: schuifspanning als functie van de vervormingssnelheid voor verschillende typen vloeitstoffen.

Newtoniaanse en niet-Newtoniaanse vloeistoffen

Een fundamenteel onderscheid in de stromingsleer (reologie) is dat tussen Newtoniaanse en niet-Newtoniaanse vloeistoffen. Dit onderscheid is gebaseerd op het verband tussen de schuifspanning τ en de afschuifsnelheid $\dot{\gamma}$ (shear-rate) van een vloeistof:

$$\tau(\dot{\gamma}) = \eta(\dot{\gamma})\,\dot{\gamma} \tag{3}$$

Newtoniaanse vloeistoffen, zoals water en lucht, worden gekenmerkt door een lineaire relatie tussen deze grootheden, waarbij de viscositeit constant is.
Echter, er zijn ook vloeistoffen die hier niet aan voldoen: de niet-Newtoniaanse vloeistoffen. In dit geval is de viscositeit $\eta(\dot{\gamma})$ niet-lineair afhankelijk van de shear-rate $\dot{\gamma}$, zie Fig. 39. Er wordt onderscheid gemaakt tussen twee typen gedrag:

- **Shear-thinning**: de viscositeit daalt bij toenemende shear-rate (bijvoorbeeld ketchup, verf, polymeeroplossingen). Deze worden ook wel pseudoplastisch genoemd.

- **Shear-thickening**: de viscositeit stijgt bij toenemende shear-rate (bijvoorbeeld maizena-oplossingen, silicagel-oplossingen). Deze worden ook wel dilatant genoemd.

Dit gedrag kan worden gemodelleerd met een zogenaamd power-law model:

$$\eta(\dot{\gamma}) = k\,\dot{\gamma}^{\,n-1} \tag{4}$$

Hierbij is η de schijnbare viscositeit, k de consistentie-index (stroperigheidsindex), $\dot{\gamma}$ de shear-rate en n de gedragscoëfficiënt van de vloeistof. De waarde van n bepaalt het type vloeistof:

- $n = 1$: Newtoniaans gedrag
- $n < 1$: shear-thinning
- $n > 1$: shear-thickening

Practicum

Experiment deel I: Digitale viscosimeter

In dit practicum werken we met een digitale viscosimeter van Vevor. Let op: de rotor en de koppeling aan de viscosimeter zijn fragiel en kostbaar. Ga hier dus voorzichtig mee om.

Net als in het vorige practicum meet je de viscositeit van verschillende glycerol-oplossingen. Ditmaal wordt hiervoor echter een digitale viscosimeter gebruikt in plaats van een Ubbelohde-viscometer.

1. Meet de dynamische viscositeit η van dezelfde vloeistoffen als in het vorige practicum.
2. Maak een schatting van de bijbehorende meetfout. Bepaal daarbij welke foutbron waarschijnlijk de grootste bijdrage levert.

Verwerking deel I

1. Maak een grafiek van de viscositeit van de water–glycerol oplossingen.
2. Voeg deze grafiek toe aan de grafiek uit het vorige practicum.
3. Komt dit verband overeen met de waarden die je hebt bepaald met de Ubbelohde-viscometer?
4. Welke viscometer heeft jullie voorkeur en waarom?

Experiment deel II: Niet-Newtoniaanse vloeistoffen

Wij verwachten dat jullie zelfstandiger worden in het opstellen van een passende meetmethode. De methode hieronder is daarom bewust beknopt gehouden. Breng vooraf in kaart welke stappen nodig zijn.

1. Meet de dynamische viscositeit η van één van de water–glycerol oplossingen en bepaal het type vloeistof (bepaal n).
2. Meet de dynamische viscositeit η van de zetmeeloplossing en maak een inschatting van de bijbehorende meetfout.
3. Meet de dynamische viscositeit η van de zetmeeloplossing bij verschillende rotatiesnelheden en maak een inschatting van de bijbehorende meetfout.

Verwerking deel II

Tijdens het practicum noteer je je observaties in je labjournaal.

1. Maak een grafiek van de dynamische viscositeit η als functie van de afschuifsnelheid $\dot{\gamma}$. Gebruik hiervoor:

$$\dot{\gamma} = \frac{2\pi C\,\mathrm{RPM}}{60} \tag{68}$$

Hierbij is C een apparaatconstante, die je gelijk aan 1 mag nemen.

2 Maak een fit met behulp van vergelijking (4).
3 Bereken de waarde van n met bijbehorende onzekerheid. Onder welk type vloeistof valt de water–glycerol oplossing? En hoe zit dit voor de maizena-oplossing?

Resultaten en discussie

1 Vergelijk de precisie van beide meetmethodes.
2 Bepaal het type vloeistof van de zetmeeloplossing.
3 Bespreek systematische fouten.
4 Verklaar verschillen met de verwachting.

Afronding

1 Werk je labjournaal volledig uit.
2 Laat je werk controleren door de begeleider.

Python voor het practicum

Tips en code snippets

Python-tips

In dit onderdeel staan veelgebruikte voorbeelden voor het verwerken, analyseren en visualiseren van meetgegevens met Python. De voorbeelden zijn bedoeld als naslag tijdens de practica.
Wil je direct beginnen? Bij Code snippets staan volledige, zelfstandig uitvoerbare voorbeelden. In dit onderdeel bouwen de korte codefragmenten binnen een onderwerp op elkaar voort.
De belangrijkste packages die we gebruiken zijn:

Package	Veelgebruikt voor	Documentatie
NumPy	Arrays, rekenen en statistiek	NumPy [1]
Matplotlib	Figuren en grafieken	Matplotlib [2]
SciPy	Fits en wetenschappelijke berekeningen	SciPy [3]
Pandas	Tabellen, CSV- en databestanden	Pandas [4]

Packages

Packages worden meestal bovenaan een Python-script of notebook geïmporteerd.

Python
```python
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from scipy.optimize import curve_fit
```

Voor NumPy, Matplotlib en Pandas worden vrijwel altijd de vaste afkortingen `np`, `plt` en `pd` gebruikt. Hierdoor kunnen functies kort worden aangeroepen:

Python
```python
x = np.array([1, 2, 3, 4, 5])

gemiddelde = np.mean(x)

print(gemiddelde)
```

Je hoeft niet altijd een volledig package te importeren. Met

Python
```python
from scipy.optimize import curve_fit
```

wordt bijvoorbeeld alleen de functie `curve_fit` uit de module `scipy.optimize` geïmporteerd. Deze kan daarna direct worden aangeroepen:

Python
```python
popt, pcov = curve_fit(model, xdata, ydata)
```

Versie controleren

Soms is het handig om te controleren welke versie van een package wordt gebruikt:

Python
```python
print(np.__version__)
print(pd.__version__)
```

Voor een overzicht van de geïnstalleerde packages in de actieve conda-environment kun je in de terminal gebruiken:

Terminal / uitvoer

```
conda list
```

ModuleNotFoundError

Krijg je een foutmelding zoals `ModuleNotFoundError`? Controleer dan eerst of in VS Code de juiste Python-environment of Jupyter-kernel is geselecteerd en of het betreffende package daarin is geïnstalleerd.

Datatypes, lijsten & arrays

Python kent verschillende datatypes. Voor het werken met meetgegevens zijn vooral **lijsten** (`list`) en **NumPy-arrays** (`ndarray`) belangrijk.
Een gewone Python-lijst maak je bijvoorbeeld zo:

Python

```python
tijd = [0, 1, 2, 3, 4]
spanning = [0.2, 1.1, 2.0, 3.2, 4.1]
```

Voor numerieke berekeningen gebruiken we meestal een NumPy-array:

Python

```python
import numpy as np

tijd = np.array([0, 1, 2, 3, 4])
spanning = np.array([0.2, 1.1, 2.0, 3.2, 4.1])
```

Het grote voordeel van arrays is dat berekeningen direct op alle waarden kunnen worden uitgevoerd:

Python

```python
spanning = np.array([1.0, 2.0, 3.0])

spanning_kwadraat = spanning**2

print(spanning_kwadraat)
```

Dit geeft:

Terminal / uitvoer

```
[1. 4. 9.]
```

Bij een Python-lijst werkt een vermenigvuldiging anders:

Python

```python
x = [1, 2, 3]

print(x * 2)
```

Dit geeft:

Terminal / uitvoer

```
[1, 2, 3, 1, 2, 3]
```

Terwijl bij een NumPy-array:

Python

```python
x = np.array([1, 2, 3])

print(x * 2)
```

het resultaat is:

Terminal / uitvoer

```
[2 4 6]
```

Voor het rekenen met meetdata gebruiken we daarom meestal **NumPy-arrays**.

Veelgebruikte datatypes

Type	Voorbeeld	Gebruik
Integer `int`	`5`	Gehele getallen
Float `float`	`5.2`	Getallen met decimalen
Boolean `bool`	`True`, `False`	Logische voorwaarden
String `str`	`"spanning"`	Tekst
List `list`	`[1, 2, 3]`	Verzameling Python-objecten
NumPy-array `ndarray`	`np.array([1, 2, 3])`	Numerieke data en berekeningen

Het datatype van een variabele kun je controleren met:

Python

```python
print(type(spanning))
```

Bij NumPy-arrays kun je daarnaast het datatype van de **elementen** bekijken:

Python

```python
print(spanning.dtype)
```

Efficiënte numerieke berekeningen

Gebruik voor numerieke meetgegevens bij voorkeur NumPy-arrays. Hiermee kun je berekeningen direct op volledige datasets uitvoeren zonder voor iedere bewerking een `for`-loop te schrijven.

Functies

Een **functie** is een stukje code dat een bepaalde bewerking uitvoert. Je geeft een functie één of meerdere waarden mee als **argumenten** en de functie geeft meestal een resultaat terug.
Een eenvoudig voorbeeld is:

Python

```python
x = np.sqrt(9)

print(x)
```

Hier is `np.sqrt()` de functie, `9` het argument en `3` de waarde die de functie teruggeeft.
Een functie kan ook meerdere argumenten hebben:

Python

```python
x = np.round(3.14159, 2)

print(x)
```

Hier krijgt `np.round()` twee argumenten mee: het getal `3.14159` en het gewenste aantal decimalen `2`. NumPy bevat daarnaast veel wiskundige functies die op dezelfde manier direct op arrays kunnen worden toegepast:

Berekening	Python	Documentatie
Wortel	`np.sqrt(x)`	numpy.sqrt [5]
Macht	`np.power(x, n)`	numpy.power [6]
Exponentiële functie e^x	`np.exp(x)`	numpy.exp [7]
Natuurlijke logaritme $\ln(x)$	`np.log(x)`	numpy.log [8]
Logaritme $\log_{10}(x)$	`np.log10(x)`	numpy.log10 [9]
Sinus	`np.sin(x)`	numpy.sin [10]
Cosinus	`np.cos(x)`	numpy.cos [11]
Tangens	`np.tan(x)`	numpy.tan [12]
Arcsinus	`np.arcsin(x)`	numpy.arcsin [13]
Arccosinus	`np.arccos(x)`	numpy.arccos [14]
Arctangens	`np.arctan(x)`	numpy.arctan [15]
Absolute waarde	`np.abs(x)`	numpy.absolute [16]
Afronden	`np.round(x, n)`	numpy.round [17]
Graden → radialen	`np.deg2rad(x)`	numpy.deg2rad [18]
Radialen → graden	`np.rad2deg(x)`	numpy.rad2deg [19]

Radialen & graden

De goniometrische functies van NumPy gebruiken hoeken in **radialen**, niet in graden.
Dit gaat dus **niet goed** wanneer `hoek` in graden staat:

Python
```
hoek = 30
y = np.sin(hoek)        # 30 wordt geïnterpreteerd als 30 radialen
```

Zet de hoek eerst om naar radialen:

Python
```
hoek = 30
hoek_rad = np.deg2rad(hoek)

y = np.sin(hoek_rad)    # sin(30°) = 0.5
```

Dit kan ook direct:

Python
```
y = np.sin(np.deg2rad(30))
```

Een volledig overzicht is te vinden in de NumPy Mathematical functions-documentatie [20].

Indexeren, slicen & filteren

Een enkele waarde uit een array kan worden geselecteerd met een **index**:

Python
```
x = np.array([10, 20, 30, 40])

print(x[0])
print(x[2])
```

Python begint met tellen bij **0**. `x[0]` is dus de eerste waarde en `x[2]` de derde waarde.
Met negatieve indices kun je vanaf het einde van een array tellen:

Python
```
print(x[-1])   # laatste waarde
print(x[-2])   # voorlaatste waarde
```

Een deel van een array selecteren

Met een **slice** kun je meerdere opeenvolgende waarden selecteren:

Python
```
x = np.array([10, 20, 30, 40, 50])

print(x[1:4])
```

Dit geeft:

Terminal / uitvoer
```
[20 30 40]
```

Bij `x[1:4]` wordt index `1` **wel** meegenomen, maar index `4` **niet**.
De algemene notatie is:

Terminal / uitvoer
```
x[start:stop:stap]
```

Bijvoorbeeld:

Python
```
print(x[:3])     # eerste drie waarden
print(x[2:])     # vanaf index 2
print(x[::2])    # iedere tweede waarde
print(x[::-1])   # omgekeerde volgorde
```

Selecteren met een voorwaarde

NumPy kan ook waarden selecteren die aan een bepaalde voorwaarde voldoen:

Python
```
x = np.array([1, 2, 3, 4, 5])

selectie = x[x > 3]

print(selectie)
```

Dit geeft:

Terminal / uitvoer
```
[4 5]
```

Ook meerdere voorwaarden kunnen worden gecombineerd:

Python
```
selectie = x[(x >= 2) & (x <= 4)]

print(selectie)
```

Dit geeft:

Terminal / uitvoer
```
[2 3 4]
```

Gebruik bij NumPy-arrays `&` voor **en** en `|` voor **of**. Zet de afzonderlijke voorwaarden tussen haakjes.

Veelgebruikte selecties en vergelijkingen

Bewerking	Python	Betekenis
Eerste waarde	`x[0]`	Element met index 0
Derde waarde	`x[2]`	Element met index 2
Laatste waarde	`x[-1]`	Laatste element
Eerste drie waarden	`x[:3]`	Index 0 t/m 2
Vanaf derde waarde	`x[2:]`	Vanaf index 2
Deel van array	`x[1:4]`	Index 1 t/m 3
Iedere tweede waarde	`x[::2]`	Stapgrootte 2
Array omkeren	`x[::-1]`	Omgekeerde volgorde
Groter dan	`x[x > 3]`	Waarden groter dan 3
Kleiner dan	`x[x < 3]`	Waarden kleiner dan 3
Groter of gelijk	`x[x >= 3]`	Waarden groter dan of gelijk aan 3
Gelijk aan	`x[x == 3]`	Waarden gelijk aan 3
Niet gelijk aan	`x[x != 3]`	Waarden ongelijk aan 3
Tussen twee waarden	`x[(x >= 2) & (x <= 4)]`	Waarden tussen 2 en 4
Aan één van twee voorwaarden	`x[(x < 2) | (x > 4)]`	Logische OF

Vaste selectie

Dezelfde selectie kan ook op een tweede array worden toegepast. Dit is erg handig bij het verwerken van meetdata.

Python

```
tijd = np.array([0, 1, 2, 3, 4])
spanning = np.array([0.1, 1.2, 2.1, 3.2, 4.1])

selectie = tijd >= 2

tijd_selectie = tijd[selectie]
spanning_selectie = spanning[selectie]
```

Zo blijven de bij elkaar horende waarden van `tijd` en `spanning` gekoppeld wanneer je een deel van de meetdata selecteert.
Meer informatie is te vinden in de NumPy-documentatie over indexing [21].

Beschrijvende statistiek

NumPy bevat functies voor veelgebruikte statistische berekeningen. Bijvoorbeeld voor een serie herhaalde metingen:

Python

```python
import numpy as np

metingen = np.array([4.9, 5.1, 5.0, 5.2, 4.8])

gemiddelde = np.mean(metingen)
mediaan = np.median(metingen)
standaardafwijking = np.std(metingen, ddof=1)

N = len(metingen)
standaardfout = standaardafwijking / np.sqrt(N)

print("Gemiddelde:", gemiddelde)
print("Mediaan:", mediaan)
print("Standaardafwijking:", standaardafwijking)
print("Standaardfout:", standaardfout)
```

Bij experimentele meetreeksen gebruiken we voor de standaardafwijking meestal `ddof=1`, met minimaal twee metingen. Voor onafhankelijke herhaalde metingen onder dezelfde omstandigheden geeft de standaardfout de statistische onzekerheid in het gemiddelde. Systematische onzekerheden, zoals een kalibratiefout, zijn hierin niet opgenomen. De standaardfout wordt berekend met

$$s_{\bar{x}} = \frac{s}{\sqrt{N}}.$$

Handige statistische functies

Berekening	Python	Documentatie
Gemiddelde	`np.mean(x)`	numpy.mean [22]
Mediaan	`np.median(x)`	numpy.median [23]
Standaardafwijking	`np.std(x, ddof=1)`	numpy.std [24]
Variantie	`np.var(x, ddof=1)`	numpy.var [25]
Standaardfout	`np.std(x, ddof=1) / np.sqrt(len(x))`	numpy.std [24]
Minimum	`np.min(x)`	numpy.min [26]
Maximum	`np.max(x)`	numpy.max [27]
Bereik	`np.ptp(x)`	numpy.ptp [28]
Som	`np.sum(x)`	numpy.sum [29]
Aantal waarden	`len(x)`	Python len [30]
Percentiel	`np.percentile(x, p)`	numpy.percentile [31]

Een volledig overzicht van de beschikbare statistische functies is te vinden in de NumPy Statistics-documentatie [32].

Figuren maken

Voor het maken van figuren gebruiken we **Matplotlib**. Een figuur bestaat uit een `figure` en één of meerdere assen (`ax`). De meeste instellingen van de grafiek worden vervolgens via `ax` aangepast.

Eenvoudige lijnplot

Een eenvoudige lijnplot maak je met `ax.plot()`:

Python

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.array([0, 1, 2, 3, 4])
y = np.array([0, 1, 4, 9, 16])

fig, ax = plt.subplots(figsize=(6, 4))

ax.plot(x, y)

ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Afstand (m)")

fig.tight_layout()
plt.show()
```

Gebruik bij de assen altijd de **grootheid en eenheid**. Gebruik dus bijvoorbeeld `"Tijd (s)"` en `"Spanning (V)"` in plaats van alleen `"x"` en `"y"`.

Meetdata met foutbalken

Voor experimentele meetdata gebruiken we meestal `ax.errorbar()`. Hiermee kunnen de meetonzekerheden direct als foutbalken worden weergegeven.

Python

```python
import numpy as np
import matplotlib.pyplot as plt

xdata = np.array([1, 2, 3, 4, 5])
ydata = np.array([1.1, 1.9, 3.2, 3.9, 5.1])
yerr = np.array([0.1] * 5)

fig, ax = plt.subplots(figsize=(6, 4))

ax.errorbar(
    xdata,
    ydata,
    yerr=yerr,
    fmt="o",
    capsize=3
)

ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Positie (m)")

fig.tight_layout()
plt.show()
```

Als ook de x-waarden een onzekerheid hebben, kan `xerr` worden toegevoegd:

Python

```python
xerr = np.array([0.05] * 5)

ax.errorbar(
    xdata,
    ydata,
    xerr=xerr,
    yerr=yerr,
    fmt="o",
    capsize=3
)
```

Lijn versus scatter!

Verbind losse meetpunten niet automatisch met lijnen. Een lijn tussen meetpunten suggereert dat je ook iets weet over de waarden tussen de meetpunten. Gebruik voor experimentele meetdata daarom meestal losse punten met foutbalken. Een theoretisch model of fit kan wel als lijn worden weergegeven.

Veelgebruikte soorten plots

Matplotlib bevat verschillende functies voor het visualiseren van data.

Type	Python	Gebruik	Documentatie
Lijnplot	`ax.plot(x, y)`	Continue functie of model	plot [33]
Meetpunten	`ax.plot(x, y, "o")`	Losse datapunten	plot [33]
Foutbalken	`ax.errorbar(x, y, yerr=dy, fmt="o")`	Meetdata met onzekerheden	errorbar [34]
Scatterplot	`ax.scatter(x, y)`	Relatie tussen twee variabelen	scatter [35]
Histogram	`ax.hist(x)`	Verdeling van meetwaarden	hist [36]
Staafdiagram	`ax.bar(x, y)`	Vergelijken van categorieën	bar [37]
Horizontale lijn	`ax.axhline(y)`	Referentiewaarde aangeven	axhline [38]
Verticale lijn	`ax.axvline(x)`	Referentiewaarde aangeven	axvline [39]

Een volledig overzicht van de mogelijkheden is te vinden in de Matplotlib-documentatie [40].

Veelgebruikte instellingen

De eigenschappen van een figuur kunnen via `ax` worden aangepast:

Instelling	Python
Label x-as	`ax.set_xlabel("Tijd (s)")`
Label y-as	`ax.set_ylabel("Spanning (V)")`
Titel	`ax.set_title("Meting")`
Legenda	`ax.legend()`
Raster	`ax.grid()`
x-bereik	`ax.set_xlim(xmin, xmax)`
y-bereik	`ax.set_ylim(ymin, ymax)`
Logaritmische x-as	`ax.set_xscale("log")`
Logaritmische y-as	`ax.set_yscale("log")`
Minor ticks	`ax.minorticks_on()`
Figuur netjes uitlijnen	`fig.tight_layout()`

Bij meerdere datasets of een combinatie van meetdata en een model kan met `label` een legenda worden gemaakt:

Python

```
ax.errorbar(
    xdata,
    ydata,
    yerr=yerr,
    fmt="o",
    capsize=3,
    label="Meetdata"
)

ax.plot(x, y, label="Model")

ax.legend()
```

Een publicatiewaardige figuur

Een figuur voor een verslag, poster of presentatie moet **zelfstandig leesbaar** zijn. De lezer moet kunnen zien welke grootheden zijn weergegeven, welke onzekerheden bij de metingen horen en welke onderdelen meetdata of modelresultaten zijn.
Een goed uitgangspunt is:

Python

```python
import numpy as np
import matplotlib.pyplot as plt

xdata = np.array([1, 2, 3, 4, 5])
ydata = np.array([1.1, 1.9, 3.2, 3.9, 5.1])
yerr = np.array([0.1] * 5)

fig, ax = plt.subplots(figsize=(6, 4))

ax.errorbar(
    xdata,
    ydata,
    yerr=yerr,
    fmt="o",
    markersize=5,
    capsize=3,
    linewidth=1,
    label="Meetdata"
)

ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Positie (m)")

ax.tick_params(
    direction="in",
    top=True,
    right=True
)

ax.minorticks_on()

ax.legend(frameon=False)

fig.tight_layout()
plt.show()
```

Aanbevolen instellingen

Onderdeel	Instelling	Opmerking
Figuurgrootte	`figsize=(6, 4)`	Goed uitgangspunt voor een losse figuur
Meetpunten	`fmt="o"`	Duidelijke losse meetpunten
Puntgrootte	`markersize=5`	Niet te groot en niet te klein
Foutbalken	`capsize=3`	Maakt foutbalken beter leesbaar
Lijndikte	`linewidth=1`	Rustige, duidelijke lijnen
Assenlabels	`set_xlabel()`, `set_ylabel()`	Altijd grootheid én eenheid
Tickrichting	`direction="in"`	Veelgebruikt in wetenschappelijke figuren
Ticks boven/rechts	`top=True`, `right=True`	Geeft een rustig afgesloten assenstelsel
Minor ticks	`minorticks_on()`	Handig voor nauwkeurig aflezen
Legenda	`legend(frameon=False)`	Alleen nodig bij meerdere datasets of modellen
Uitlijning	`tight_layout()`	Voorkomt afgesneden labels

Lettergrootte instellen

Voor een verslag is het handig om de lettergroottes expliciet in te stellen:

Python

```
plt.rcParams.update({
    "font.size": 11,
    "axes.labelsize": 12,
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "legend.fontsize": 10
})
```

Zet dit bij voorkeur bovenaan het script, vóór het maken van de figuur.

Figuur opslaan

Voor een verslag heeft een vectorformaat zoals PDF meestal de voorkeur:

Python

```
fig.savefig(
    "figuur.pdf",
    bbox_inches="tight"
)
```

Voor PNG:

Python

```
fig.savefig(
    "figuur.png",
    dpi=300,
    bbox_inches="tight"
)
```

Gebruik voor een publicatiewaardige rasterafbeelding bij voorkeur minimaal `dpi=300`.

Goede figuren

Een goede wetenschappelijke figuur bevat alleen informatie die nodig is om de data te begrijpen. Gebruik dus geen decoratieve achtergronden, overmatige kleuren, dikke rasterlijnen of 3D-effecten.

Een goede figuur bevat in ieder geval:

1. duidelijk leesbare assen;
2. grootheden en eenheden bij de assen;
3. foutbalken wanneer meetonzekerheden bekend zijn;
4. een legenda wanneer meerdere datasets of modellen worden weergegeven;
5. voldoende grote tekst en symbolen;
6. geen onnodige visuele elementen.

Titels, legenda's en onderschriften

Gebruik een onderschrift om de meting, het model en de betekenis van de foutbalken toe te lichten. Een titel is vaak overbodig wanneer het onderschrift dezelfde informatie geeft. Een legenda is wel nuttig om meerdere datasets of modellen te onderscheiden. Houd assenlabels en eenheden in de figuur zelf.

Een model fitten

Wanneer uit de theorie een wiskundig model volgt voor het verband tussen twee grootheden, kunnen we de onbekende parameters van dat model bepalen door het model aan de meetdata te **fitten**.

Bij een fit zoekt Python naar de waarden van de modelparameters waarvoor het model zo goed mogelijk overeenkomt met de meetgegevens, rekening houdend met de opgegeven meetonzekerheden.

Stel dat we op basis van de theorie een lineair verband verwachten:

$$y = ax + b.$$

Hierbij zijn:

- x en y de gemeten grootheden;
- a de richtingscoëfficiënt;
- b het snijpunt met de y-as.

De parameters a en b zijn dus de onbekende grootheden die we met de fit willen bepalen.

Stap 1 — Het model definiëren

We schrijven het theoretische model eerst als Python-functie:

Python

```python
def linear(x, a, b):
    return a * x + b
```

De eerste invoer `x` is de onafhankelijke variabele. De overige invoeren, hier `a` en `b`, zijn de parameters die door de fit moeten worden bepaald.

Stap 2 — Meetdata invoeren

We gebruiken als voorbeeld de volgende meetgegevens:

```python
import numpy as np
from scipy.optimize import curve_fit

xdata = np.array([1, 2, 3, 4, 5])
ydata = np.array([1.1, 1.9, 3.2, 3.9, 5.1])

yerr = np.array([0.1] * 5)
```

Hier bevat `yerr` de positieve standaardonzekerheid (één sigma) van iedere gemeten y-waarde. We nemen onafhankelijke meetfouten aan. `curve_fit` neemt onzekerheden in de x-waarden niet mee: ook het tekenen van `xerr` verandert dat niet.

Stap 3 — De fit uitvoeren

De fit voeren we uit met `curve_fit()` uit SciPy:

```python
p0 = [1, 0]

popt, pcov = curve_fit(
    linear,
    xdata,
    ydata,
    sigma=yerr,
    p0=p0,
    absolute_sigma=True
)
```

De belangrijkste argumenten zijn:

Argument	Betekenis
`linear`	het model dat wordt gefit
`xdata`	de gemeten waarden van x
`ydata`	de gemeten waarden van y
`sigma=yerr`	de onzekerheden van de y-metingen
`p0=[1, 0]`	eerste schatting van de parameters a en b
`absolute_sigma=True`	zorgt ervoor dat `yerr` als absolute meetonzekerheid wordt gebruikt

De functie geeft twee resultaten terug:

```python
popt
pcov
```

`popt` bevat de gevonden optimale waarden van de fitparameters. In dit voorbeeld geldt:

```python
a = popt[0]
b = popt[1]
```

`pcov` is de covariantiematrix van de fitparameters. Hieruit kunnen we de onzekerheden van a en b bepalen.
Bij een niet-lineair model is deze onzekerheidsschatting gebaseerd op een lokale lineaire benadering. Zonder `absolute_sigma=True` schaalt SciPy de covariantiematrix met de gereduceerde chi-kwadraat. Zie de documentatie van curve_fit [41].

Volgorde van de parameters

De volgorde in `popt` is dezelfde als de volgorde van de parameters in de modeldefinitie.
Bij

Python
```
def linear(x, a, b):
    return a * x + b
```

geldt dus:

Python
```
popt[0]   # a
popt[1]   # b
```

Onzekerheid van de fitparameters

Een fit levert niet alleen de beste waarden van de parameters op, maar ook informatie over hun onzekerheid.
De standaardonzekerheden van de fitparameters volgen uit de diagonaalelementen van `pcov`:

Python
```
perr = np.sqrt(np.diag(pcov))

a = popt[0]
b = popt[1]

a_err = perr[0]
b_err = perr[1]

print("a =", a, "+-", a_err)
print("b =", b, "+-", b_err)
```

Voor deze meetdata vinden we ongeveer:

$$a = 1.000 \pm 0.032$$

en

$$b = 0.040 \pm 0.105.$$

Het resultaat van de fit rapporteer je dus inclusief de onzekerheid van de gevonden parameters.

Onzekerheid in x

`curve_fit()` verwerkt met `sigma=` alleen onzekerheden in de y-waarden.
Als de onzekerheid in x niet verwaarloosbaar is, is deze eenvoudige aanpak niet voldoende.

Fit en meetdata samen plotten

Om de fit te beoordelen is het belangrijk om zowel de meetdata als het gefitte model te bekijken.
De meetdata tekenen we als losse punten met foutbalken. Voor het model maken we een fijn verdeelde reeks x-waarden, zodat de fit als een vloeiende lijn wordt weergegeven.

Python

```python
import matplotlib.pyplot as plt

xfit = np.linspace(
    np.min(xdata),
    np.max(xdata),
    200
)

yfit = linear(xfit, *popt)

fig, ax = plt.subplots(figsize=(6, 4))

ax.errorbar(
    xdata,
    ydata,
    yerr=yerr,
    fmt="o",
    capsize=3,
    label="Meetdata"
)

ax.plot(
    xfit,
    yfit,
    label="Lineaire fit"
)

ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Positie (m)")

ax.tick_params(
    direction="in",
    top=True,
    right=True
)

ax.minorticks_on()
ax.legend(frameon=False)

fig.tight_layout()
plt.show()
```

De uitdrukking

Python

```python
linear(xfit, *popt)
```

is een korte schrijfwijze voor:

Python

```python
linear(xfit, popt[0], popt[1])
```

Het sterretje `*` pakt de waarden uit `popt` uit en geeft ze afzonderlijk door aan de functie.

Meetpunten en fitlijn

De 200 waarden in `xfit` zijn **geen extra meetpunten**.
Ze worden alleen gebruikt om het wiskundige model als een vloeiende lijn weer te geven.

De kwaliteit van een fit beoordelen

Het feit dat Python een fit kan uitvoeren betekent nog niet automatisch dat het gekozen model de meetdata goed beschrijft.
Een eerste stap is daarom altijd:

1. bekijk de meetpunten en foutbalken;
2. bekijk de fitlijn;
3. controleer of er systematische afwijkingen zichtbaar zijn;
4. bereken eventueel de chi-kwadraat en bekijk de residuen.

Chi-kwadraat

De chi-kwadraat vergelijkt de afwijking tussen iedere meetwaarde en het model met de onzekerheid van die meting:

$$\chi^2 = \sum_i \left(\frac{y_i - y_{\text{model},i}}{\sigma_i} \right)^2.$$

Voor ieder meetpunt wordt dus gekeken naar:

$$\frac{y_i - y_{\text{model},i}}{\sigma_i} = \frac{\text{afwijking tussen meting en model}}{\text{onzekerheid van de meting}}.$$

Een afwijking van bijvoorbeeld 0.2 is veel belangrijker wanneer de meetonzekerheid 0.01 is dan wanneer de meetonzekerheid 0.5 is.
In Python:

Python

```
y_model = linear(xdata, *popt)

chi2 = np.sum(
    ((ydata - y_model) / yerr)**2
)

print("chi^2 =", chi2)
```

Gereduceerde chi-kwadraat

De gewone χ^2 hangt onder andere af van het aantal meetpunten. Daarom gebruiken we vaak de **gereduceerde chi-kwadraat**:

$$\chi^2_{\text{red}} = \frac{\chi^2}{N - p}.$$

Hierbij is:

- N het aantal meetpunten;
- p het aantal vrije fitparameters;
- $N - p$ het aantal vrijheidsgraden.

Voor het lineaire model zijn er twee fitparameters, namelijk a en b. Bij vijf meetpunten geldt dus:

$$N - p = 5 - 2 = 3.$$

In Python:

Python

```
dof = len(xdata) - len(popt)

chi2_red = chi2 / dof

print("Vrijheidsgraden =", dof)
print("chi^2_red =", chi2_red)
```

Voor de voorbeelddata vinden we ongeveer:

Grootheid	Waarde
a	1.000 ± 0.032
b	0.040 ± 0.105
χ^2	7.20
Vrijheidsgraden $N - p$	3
χ^2_{red}	2.40

Bij onafhankelijke, normaal verdeelde meetfouten met correct opgegeven standaardonzekerheden en een geschikt model verwachten we een waarde van χ^2_{red} van **orde 1**. Hiervoor moet $N > p$ gelden. Bij weinig vrijheidsgraden kan de waarde sterk fluctueren; voor een lineair model is de standaardafwijking van deze grootheid $\sqrt{2/(N-p)}$.
Een veel grotere waarde kan bijvoorbeeld betekenen dat:

- het gekozen model de meetdata niet goed beschrijft;
- de meetonzekerheden te klein zijn geschat;
- er systematische effecten aanwezig zijn die niet in het model zijn opgenomen.

Een veel kleinere waarde kan bijvoorbeeld betekenen dat de meetonzekerheden ruim zijn geschat.

Geen automatische goed/fout-test

Gebruik χ^2_{red} niet als automatische grens tussen een goede en slechte fit.
Bekijk altijd ook de meetpunten, foutbalken, fitlijn en residuen.

Residuen bekijken

De **residuen** zijn de verschillen tussen de gemeten waarden en de waarden die het model voorspelt:

$$\epsilon_i = y_i - y_{\text{model},i}.$$

In Python:

Python

```
y_model = linear(xdata, *popt)

residuen = ydata - y_model

print(residuen)
```

Een positief residu betekent dat de meetwaarde **boven** het model ligt. Een negatief residu betekent dat de meetwaarde **onder** het model ligt.
De residuen kunnen ook worden geplot:

Python

```python
fig, ax = plt.subplots(figsize=(6, 3))

ax.errorbar(
    xdata,
    residuen,
    yerr=yerr,
    fmt="o",
    capsize=3
)

ax.axhline(
    0,
    linewidth=1
)

ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Residu (m)")

ax.tick_params(
    direction="in",
    top=True,
    right=True
)

ax.minorticks_on()

fig.tight_layout()
plt.show()
```

Bij een geschikt model verwacht je dat de residuen zonder duidelijk systematisch patroon rond nul verspreid liggen.
Bijvoorbeeld:

- willekeurig boven en onder nul → model kan geschikt zijn;
- een kromming in de residuen → mogelijk is het gekozen model te eenvoudig;
- residuen steeds aan één kant van nul → mogelijk is er een systematische afwijking.

Een fit beoordelen

Kijk bij het beoordelen van een fit dus niet alleen naar de gevonden fitparameters.
Gebruik in ieder geval:
meetdata + foutbalken → fitlijn → residuen → χ^2_{red}.

Data inlezen uit een CSV-bestand

Meetgegevens worden vaak opgeslagen in een **CSV-bestand**. Met Pandas kan zo'n bestand eenvoudig worden ingelezen:

Python

```python
import pandas as pd

data = pd.read_csv("metingen.csv")
```

Een kolom kan vervolgens worden geselecteerd met:

Python
```
tijd = data["tijd"]
spanning = data["spanning"]
```

Instellingen voor `read_csv()`

Met onderstaande instellingen geef je aan **hoe het bestand is opgebouwd**:

Instelling	Betekenis	Voorbeeld
`sep`	Scheidingsteken tussen kolommen	`sep=";"`
`decimal`	Decimaalteken	`decimal=","`
`header`	Welke regel de kolomnamen bevat	`header=0`
`names`	Zelf kolomnamen opgeven	`names=["tijd", "spanning"]`
`skiprows`	Aantal regels bovenaan overslaan	`skiprows=2`
`usecols`	Alleen bepaalde kolommen inlezen	`usecols=["tijd", "spanning"]`
`na_values`	Waarden die als ontbrekende data moeten worden gezien	`na_values=["-", "N/A"]`
`comment`	Teken waarmee commentaarregels beginnen	`comment="#"`

Bekijk voor alle beschikbare instellingen de documentatie van Pandas [42].
Een CSV-bestand uit Nederlandse software gebruikt bijvoorbeeld vaak een **puntkomma als scheidingsteken** en een **komma als decimaalteken**:

Terminal / uitvoer
```
tijd;spanning;onzekerheid
0;0,12;0,02
1;1,05;0,02
2;2,08;0,03
```

Dit lees je in met:

Python
```
data = pd.read_csv(
    "metingen.csv",
    sep=";",
    decimal=","
)
```

Controleer de data

Controleer na het inlezen altijd kort of de kolommen en getallen correct zijn ingelezen:

Python
```
print(data.head())
print(data.dtypes)
```

Worden getallen niet als numerieke waarden herkend? Controleer dan eerst `sep` en `decimal`.

Veelvoorkomende fouten

Een foutmelding in Python bevat meestal informatie over **wat er fout gaat en waar de fout optreedt**. Kijk daarom altijd eerst naar de laatste regel van de foutmelding. Hier staat meestal het type fout, bijvoorbeeld `NameError`, `ValueError` of `FileNotFoundError`.

`NameError`

Bijvoorbeeld:

Terminal / uitvoer

```
NameError: name 'np' is not defined
```

Python kent de naam `np` niet. Waarschijnlijk is NumPy nog niet geïmporteerd:

Python

```python
import numpy as np
```

Een `NameError` kan ook ontstaan door een typefout in een variabelenaam:

Python

```python
spanning = 5.0

print(spaning)  # typefout
```

NameError

Controleer bij een `NameError` of de variabele of functie al is gedefinieerd en of de naam correct is gespeld.

`ModuleNotFoundError`

Bijvoorbeeld:

Terminal / uitvoer

```
ModuleNotFoundError: No module named 'scipy'
```

Python kan het package niet vinden. Controleer eerst of in VS Code de juiste **Python-environment of Jupyter-kernel** is geselecteerd.
Wanneer SciPy nog niet in de gebruikte conda-environment staat, installeer je het vanuit de terminal met:

Terminal / uitvoer

```
conda install scipy
```

Je kunt controleren welke packages in de actieve environment zijn geïnstalleerd met:

Terminal / uitvoer

```
conda list
```

`FileNotFoundError`

Bijvoorbeeld:

Terminal / uitvoer

```
FileNotFoundError: [Errno 2] No such file or directory: 'metingen.csv'
```

Python kan het opgegeven bestand niet vinden.
Controleer:

- of de bestandsnaam correct is;
- of de extensie klopt, bijvoorbeeld `.csv`;
- of het bestand in de juiste map staat;
- of het juiste bestandspad is opgegeven.

Een relatief bestandspad wordt geïnterpreteerd ten opzichte van de **huidige werkmap**, niet automatisch de map van je script. Bekijk deze met `from pathlib import Path` en `print(Path.cwd())`.
Staat het bestand in de huidige werkmap:

Python
```
data = pd.read_csv("metingen.csv")
```

Staat het bestand in een submap `data`:

Python
```
data = pd.read_csv("data/metingen.csv")
```

`KeyError`

Bijvoorbeeld:

Terminal / uitvoer
```
KeyError: 'spanning'
```

Dit komt bij Pandas vaak voor wanneer je een kolom probeert te selecteren die niet bestaat:

Python
```
spanning = data["spanning"]
```

Bekijk de beschikbare kolomnamen met:

Python
```
print(data.columns)
```

Controleer daarbij ook op hoofdletters en spaties. `"Spanning"` en `"spanning"` zijn voor Python verschillende namen.

`TypeError`

Bijvoorbeeld:

Terminal / uitvoer
```
TypeError: can't multiply sequence by non-int of type 'float'
```

Een `TypeError` betekent dat een bewerking wordt uitgevoerd op een datatype waarvoor die bewerking niet is toegestaan.
Controleer het datatype met:

Python
```
print(type(x))
```

Bij meetgegevens kan het bijvoorbeeld nodig zijn om gegevens om te zetten naar een numeriek datatype:

Python
```
x = np.array(x, dtype=float)
```

`ValueError`

Bijvoorbeeld:

Terminal / uitvoer
```
ValueError: could not convert string to float
```

Python verwacht een getal, maar vindt bijvoorbeeld tekst.
Dit komt bij meetbestanden onder andere voor wanneer getallen verkeerd worden ingelezen. Controleer dan bijvoorbeeld het **decimaalteken** en **scheidingsteken**:

Python
```
data = pd.read_csv(
    "metingen.csv",
    sep=";",
    decimal=","
)
```

Controleer eventueel de datatypes:

Python
```
print(data.dtypes)
```

`IndexError`

Bijvoorbeeld:

Terminal / uitvoer
```
IndexError: index 5 is out of bounds
```

Je probeert een element uit een array op te vragen dat niet bestaat.
Bijvoorbeeld:

Python
```
x = np.array([10, 20, 30])

print(x[3])
```

Deze array heeft drie elementen, maar de indices zijn:

Terminal / uitvoer
```
0, 1, 2
```

Python begint bij index `0`.

Arrays hebben verschillende lengtes

Bij het maken van een figuur kan bijvoorbeeld de volgende fout optreden:

Terminal / uitvoer
```
ValueError: x and y must have same first dimension
```

De arrays met x- en y-waarden hebben dan niet hetzelfde aantal elementen.
Controleer dit met:

Python
```
print(len(xdata))
print(len(ydata))
```

Voor iedere x-waarde moet er een bijbehorende y-waarde zijn.
Als je per meetpunt een foutbalk opgeeft, controleer dan ook:

Python
```
print(len(xdata))
print(len(ydata))
print(len(yerr))
```

Voor gelijke foutbalken mag `yerr` ook één getal zijn, bijvoorbeeld `yerr=0.1`. In dat geval is `len(yerr)` niet van toepassing.

`SyntaxError`

Bijvoorbeeld:

Terminal / uitvoer

```
SyntaxError: invalid syntax
```

Python kan de geschreven code niet interpreteren. Veelvoorkomende oorzaken zijn:

Python

```python
print("Hallo"       # haakje ontbreekt

x = [1, 2, 3       # blokhaakje ontbreekt

if x > 2           # dubbele punt ontbreekt
```

Controleer bij een `SyntaxError` vooral de regel die Python aanwijst én de regel direct daarvoor.

Foutmeldingen lezen

Kijk bij een foutmelding eerst naar de **laatste regel**. Daar staat meestal de belangrijkste informatie. Bijvoorbeeld:

Terminal / uitvoer

```
FileNotFoundError: [Errno 2] No such file or directory: 'metingen.csv'
```

Hier vertelt Python zowel **welk type fout** optreedt (`FileNotFoundError`) als **wat het probleem is** (`metingen.csv` kan niet worden gevonden).

Documentatie voor op papier

De nummers bij de links verwijzen naar onderstaande webadressen.

1. https://numpy.org/doc/stable/
2. https://matplotlib.org/stable/
3. https://docs.scipy.org/doc/scipy/
4. https://pandas.pydata.org/docs/
5. https://numpy.org/doc/stable/reference/generated/numpy.sqrt.html
6. https://numpy.org/doc/stable/reference/generated/numpy.power.html
7. https://numpy.org/doc/stable/reference/generated/numpy.exp.html
8. https://numpy.org/doc/stable/reference/generated/numpy.log.html
9. https://numpy.org/doc/stable/reference/generated/numpy.log10.html
10. https://numpy.org/doc/stable/reference/generated/numpy.sin.html
11. https://numpy.org/doc/stable/reference/generated/numpy.cos.html
12. https://numpy.org/doc/stable/reference/generated/numpy.tan.html
13. https://numpy.org/doc/stable/reference/generated/numpy.arcsin.html
14. https://numpy.org/doc/stable/reference/generated/numpy.arccos.html
15. https://numpy.org/doc/stable/reference/generated/numpy.arctan.html
16. https://numpy.org/doc/stable/reference/generated/numpy.absolute.html
17. https://numpy.org/doc/stable/reference/generated/numpy.round.html
18. https://numpy.org/doc/stable/reference/generated/numpy.deg2rad.html
19. https://numpy.org/doc/stable/reference/generated/numpy.rad2deg.html
20. https://numpy.org/doc/stable/reference/routines.math.html
21. https://numpy.org/doc/stable/user/basics.indexing.html
22. https://numpy.org/doc/stable/reference/generated/numpy.mean.html
23. https://numpy.org/doc/stable/reference/generated/numpy.median.html
24. https://numpy.org/doc/stable/reference/generated/numpy.std.html
25. https://numpy.org/doc/stable/reference/generated/numpy.var.html
26. https://numpy.org/doc/stable/reference/generated/numpy.min.html
27. https://numpy.org/doc/stable/reference/generated/numpy.max.html
28. https://numpy.org/doc/stable/reference/generated/numpy.ptp.html

29 https://numpy.org/doc/stable/reference/generated/numpy.sum.html
30 https://docs.python.org/3/library/functions.html#len
31 https://numpy.org/doc/stable/reference/generated/numpy.percentile.html
32 https://numpy.org/doc/stable/reference/routines.statistics.html
33 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.plot.html
34 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.errorbar.html
35 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.scatter.html
36 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.hist.html
37 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.bar.html
38 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.axhline.html
39 https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.axvline.html
40 https://matplotlib.org/stable/plot_types/index.html
41 https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html
42 https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

Code snippets

Kopieer één volledig codeblok naar een nieuw Python-bestand of een lege notebookcel en voer het uit. Elk voorbeeld bevat eigen imports en voorbeelddata; je hoeft eerdere blokken niet uit te voeren. Vervang daarna de data en eenheden door die van jouw experiment.
Installeer zo nodig de packages in je actieve practicum-environment, via de terminal:

Terminal / uitvoer

```
conda install numpy matplotlib scipy pandas
```

Zie Python-tips voor uitleg en Python installeren [1] voor de werkomgeving.

Meetpunten plotten en een figuur opslaan

Dit voorbeeld slaat `meetpunten.png` op in de huidige werkmap. Een bestaand bestand met die naam wordt vervangen. Sla de figuur op vóór `plt.show()`.

Python

```python
import numpy as np
import matplotlib.pyplot as plt

tijd = np.array([0, 1, 2, 3, 4])
spanning = np.array([0.1, 1.2, 1.9, 3.1, 4.0])

fig, ax = plt.subplots()
ax.plot(tijd, spanning, "o")
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Spanning (V)")
fig.tight_layout()
fig.savefig("meetpunten.png", dpi=300)
plt.show()
```

Meetpunten met foutbalken

De foutbalken zijn hier standaardonzekerheden (één sigma). `xerr` kan op dezelfde manier worden toegevoegd voor horizontale foutbalken.

Python

```python
import numpy as np
import matplotlib.pyplot as plt

tijd = np.array([0, 1, 2, 3, 4])
positie = np.array([0.1, 1.2, 1.9, 3.1, 4.0])
sigma = np.array([0.1, 0.2, 0.1, 0.2, 0.1])

fig, ax = plt.subplots()
ax.errorbar(tijd, positie, yerr=sigma, fmt="o", capsize=3)
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Positie (m)")
fig.tight_layout()
plt.show()
```

Een rechte lijn fitten

Het model is `positie = a * tijd + b`. We nemen onafhankelijke meetfouten en verwaarloosbare onzekerheid in de tijd aan. `sigma` bevat positieve standaardonzekerheden in de positie. `absolute_sigma=True` gebruikt deze schaal ook voor de parameteronzekerheden.

Python

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

def lineair(t, a, b):
    return a * t + b

tijd = np.array([1, 2, 3, 4, 5], dtype=float)
positie = np.array([1.1, 1.9, 3.2, 3.9, 5.1])
sigma = np.full(5, 0.1)

popt, pcov = curve_fit(lineair, tijd, positie,
                       sigma=sigma, absolute_sigma=True)
perr = np.sqrt(np.diag(pcov))
print(f"a = {popt[0]:.3f} +/- {perr[0]:.3f} m/s")
print(f"b = {popt[1]:.3f} +/- {perr[1]:.3f} m")

t = np.linspace(tijd.min(), tijd.max(), 200)
fig, ax = plt.subplots()
ax.errorbar(tijd, positie, yerr=sigma, fmt="o", label="Meetdata")
ax.plot(t, lineair(t, *popt), label="Lineaire fit")
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Positie (m)")
ax.legend()
fig.tight_layout()
plt.show()
```

Je vindt ongeveer `a = 1.000 +/- 0.032 m/s` en `b = 0.040 +/- 0.105 m`. De onzekerheden zijn standaardonzekerheden, geen gegarandeerde grenzen. `curve_fit` neemt onzekerheden in de x-waarden niet mee; alleen `xerr` tekenen verandert de fit niet.

Een exponentiële afname fitten

Dit model beschrijft een spanning die naar nul afneemt: `U(t) = U0 * exp(-t / tau)`. De tijdconstante `tau` is positief. De startwaarden `p0` helpen bij deze niet-lineaire fit.

Python

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

def afname(t, U0, tau):
    return U0 * np.exp(-t / tau)

tijd = np.array([0, 1, 2, 3, 4, 5], dtype=float)
spanning = np.array([5.02, 3.01, 1.86, 1.10, 0.69, 0.40])
sigma = np.full(6, 0.05)

popt, pcov = curve_fit(afname, tijd, spanning, p0=[5, 2],
                       bounds=([0, 0.01], [np.inf, np.inf]),
                       sigma=sigma, absolute_sigma=True)
perr = np.sqrt(np.diag(pcov))
print(f"U0 = {popt[0]:.3f} +/- {perr[0]:.3f} V")
print(f"tau = {popt[1]:.3f} +/- {perr[1]:.3f} s")

t = np.linspace(0, 5, 200)
fig, ax = plt.subplots()
ax.errorbar(tijd, spanning, yerr=sigma, fmt="o", label="Meetdata")
ax.plot(t, afname(t, *popt), label="Exponentiële fit")
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Spanning (V)")
ax.legend()
fig.tight_layout()
plt.show()
```

Residuen en gereduceerde chi-kwadraat

Residuen zijn `meting - model`. Bekijk of ze een systematisch patroon hebben. Voor de interpretatie van chi-kwadraat nemen we onafhankelijke, normaal verdeelde meetfouten met bekende standaardonzekerheden aan. Er moeten meer meetpunten dan vrije fitparameters zijn.

Python

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

def lineair(t, a, b):
    return a * t + b

tijd = np.array([1, 2, 3, 4, 5], dtype=float)
positie = np.array([1.1, 1.9, 3.2, 3.9, 5.1])
sigma = np.full(5, 0.1)
popt, pcov = curve_fit(lineair, tijd, positie,
                       sigma=sigma, absolute_sigma=True)
residuen = positie - lineair(tijd, *popt)
dof = len(tijd) - len(popt)
chi2 = np.sum((residuen / sigma)**2)
print(f"chi2 = {chi2:.2f}; vrijheidsgraden = {dof}")
print(f"chi2_red = {chi2 / dof:.2f}")

fig, ax = plt.subplots()
ax.errorbar(tijd, residuen, yerr=sigma, fmt="o", capsize=3)
ax.axhline(0, color="black", linewidth=1)
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Residu (m)")
fig.tight_layout()
plt.show()
```

Hier is `chi2_red = 2.40`. Een waarde rond 1 is een verwachting onder de aannames, geen automatische goedkeuring van het model. Bij weinig vrijheidsgraden zijn grote schommelingen mogelijk. De foutbalken tonen hier de oorspronkelijke meetonzekerheden; gefitte residuen zijn onderling gecorreleerd.

Gemiddelde en standaardfout

Voor onafhankelijke herhalingen onder dezelfde omstandigheden, met minimaal twee metingen. De standaardfout omvat geen systematische onzekerheid, zoals een kalibratiefout.

Python

```python
import numpy as np

metingen = np.array([4.9, 5.1, 5.0, 5.2, 4.8])  # in volt
gemiddelde = np.mean(metingen)
spreiding = np.std(metingen, ddof=1)
standaardfout = spreiding / np.sqrt(len(metingen))
print(f"Standaardafwijking = {spreiding:.3f} V")
print(f"Gemiddelde = {gemiddelde:.3f} +/- {standaardfout:.3f} V")
```

CSV-data inlezen en plotten

Dit voorbeeld gebruikt CSV-tekst in het geheugen en heeft dus geen extern bestand nodig. Voor je eigen bestand vervang je `StringIO(csv_tekst)` door `"metingen.csv"`. Dat pad is relatief aan de huidige werkmap.

Python

```python
from io import StringIO
import pandas as pd
import matplotlib.pyplot as plt

csv_tekst = """tijd;spanning
0;0,12
1;1,05
2;2,08
"""
data = pd.read_csv(StringIO(csv_tekst), sep=";", decimal=",")
print(data)

fig, ax = plt.subplots()
ax.plot(data["tijd"], data["spanning"], "o")
ax.set_xlabel("Tijd (s)")
ax.set_ylabel("Spanning (V)")
fig.tight_layout()
plt.show()
```

Documentatie

- Matplotlib: plottypen [2]
- SciPy: curve_fit en parameteronzekerheden [3]
- NumPy: standaardafwijking [4]
- Pandas: read_csv [5]

Documentatie voor op papier

De nummers bij de links verwijzen naar onderstaande webadressen.

1. https://hva-tn.github.io/Website-Practicum/periode%201/python/
2. https://matplotlib.org/stable/plot_types/index.html
3. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html
4. https://numpy.org/doc/stable/reference/generated/numpy.std.html
5. https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html